



DATA MANAGEMENT 2

PL/SQL programming Techniques





WHAT IS PL/SQL?

PL/SQL - PROCEDURAL LANGUAGE/STRUCTURED QUERY LANGUAGE

- A powerful extension of SQL.
- Used for developing database applications.
- Its constructs express the intent of the code clearly.
- PL/SQL is a straightforward language to learn.
- PL/SQL is a standard and portable language for Oracle Database development. If you develop program that executes on an Oracle Database, you can quickly move it to another compatible Oracle Database without any changes.
- Combines SQL for data manipulation with procedural constructs for logic.

KEY FEATURES OF PL/SQL

Procedural Constructs

Block Structure

Variables

Cursors

Exception Handling

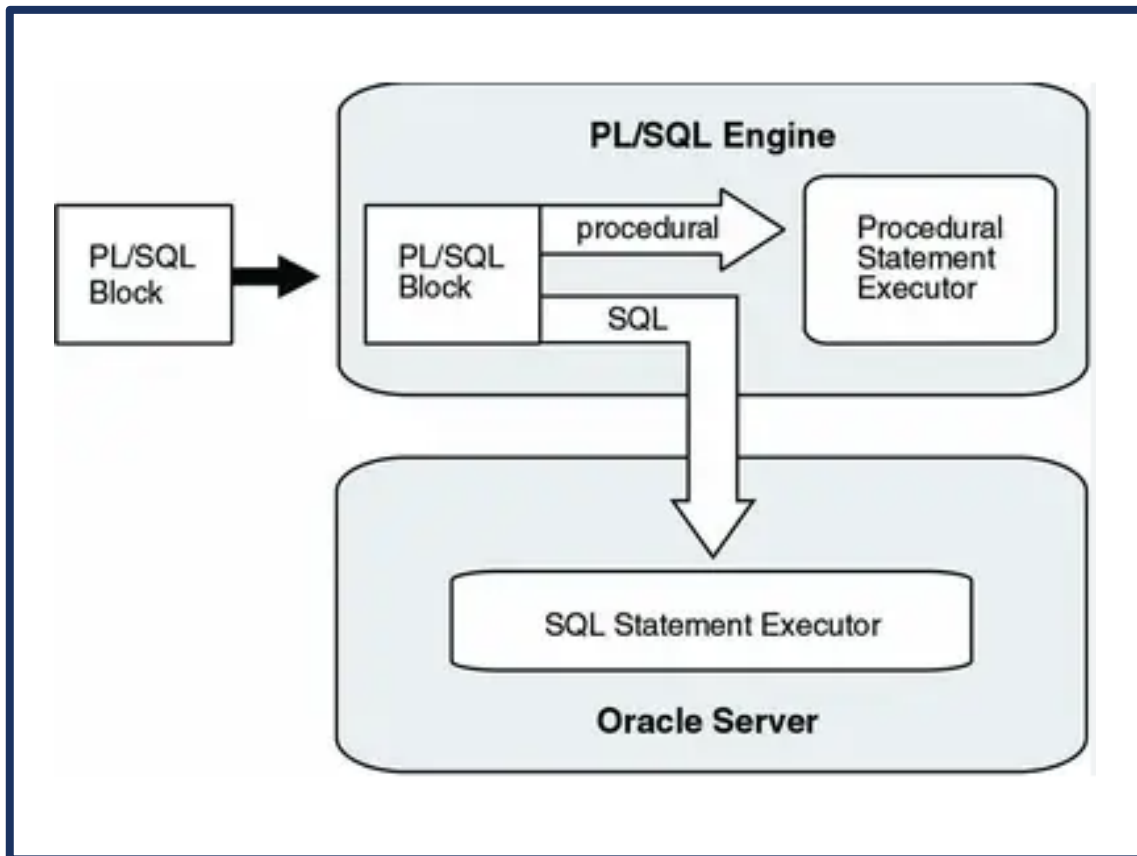
Integration

Portability

DIFFERENCES BETWEEN SQL AND PL/SQL

SQL	PL/SQL
SQL is a single query that is used to perform DML and DDL operations.	PL/SQL is a block of codes that used to write the entire program blocks/ procedure/ function, etc.
It is declarative, that defines what needs to be done, rather than how things need to be done.	PL/SQL is procedural that defines how the things needs to be done.
Execute as a single statement.	Execute as a whole block.
Mainly used to manipulate data.	Mainly used to create an application.
Cannot contain PL/SQL code in it.	It is an extension of SQL, so it can contain SQL inside it.

PL/SQL ARCHITECTURE



PL/SQL Engine:

- The core component responsible for executing PL/SQL code.
- It interprets and compiles PL/SQL programs into an intermediate form called "p-code" or bytecode before execution, which can improve performance.
- Handles the control flow, variable management, and exception handling within PL/SQL programs.

CONT

PL/SQL Blocks:

- A PL/SQL block consists of the following sections:
 - **DECLARE:** Declarations of variables, constants, and cursors used within the block.
 - **BEGIN:** Executable statements, including SQL statements and procedural logic.
 - **EXCEPTION:** Exception handling, where you define how the program should respond to errors.

CONT

Database Server (SQL Statement Executor):

- When a PL/SQL program includes SQL statements, the PL/SQL Engine communicates with the database server to perform the requested database actions.

```
DECLARE
```

```
    <declarations section>
```

```
BEGIN
```

```
    <executable command(s)>
```

```
EXCEPTION
```

```
    <exception handling>
```

```
END;
```

BASIC STRUCTURE

BASIC STRUCTURE : DECLARATION SECTION

Common declarations in this section include:

- Variable and constant declarations (`DECLARE v_variable NUMBER;`)
- Cursor declarations (`DECLARE CURSOR c_employee IS SELECT * FROM employees;`)
- Exception declarations (`DECLARE invalid_input EXCEPTION;`)

BASIC STRUCTURE : EXECUTABLE SECTION

- The executable section contains the actual PL/SQL statements and logic that perform tasks, manipulate data, and make decisions.
- The executable section begins with the **BEGIN** keyword and ends with the **END** keyword.

Common elements in this section include:

- Conditional statements (IF, CASE)
- Looping constructs (FOR, WHILE, LOOP)
- SQL statements for database operations (SELECT, INSERT, UPDATE, DELETE)
- Procedural statements (e.g., assignments, function calls)

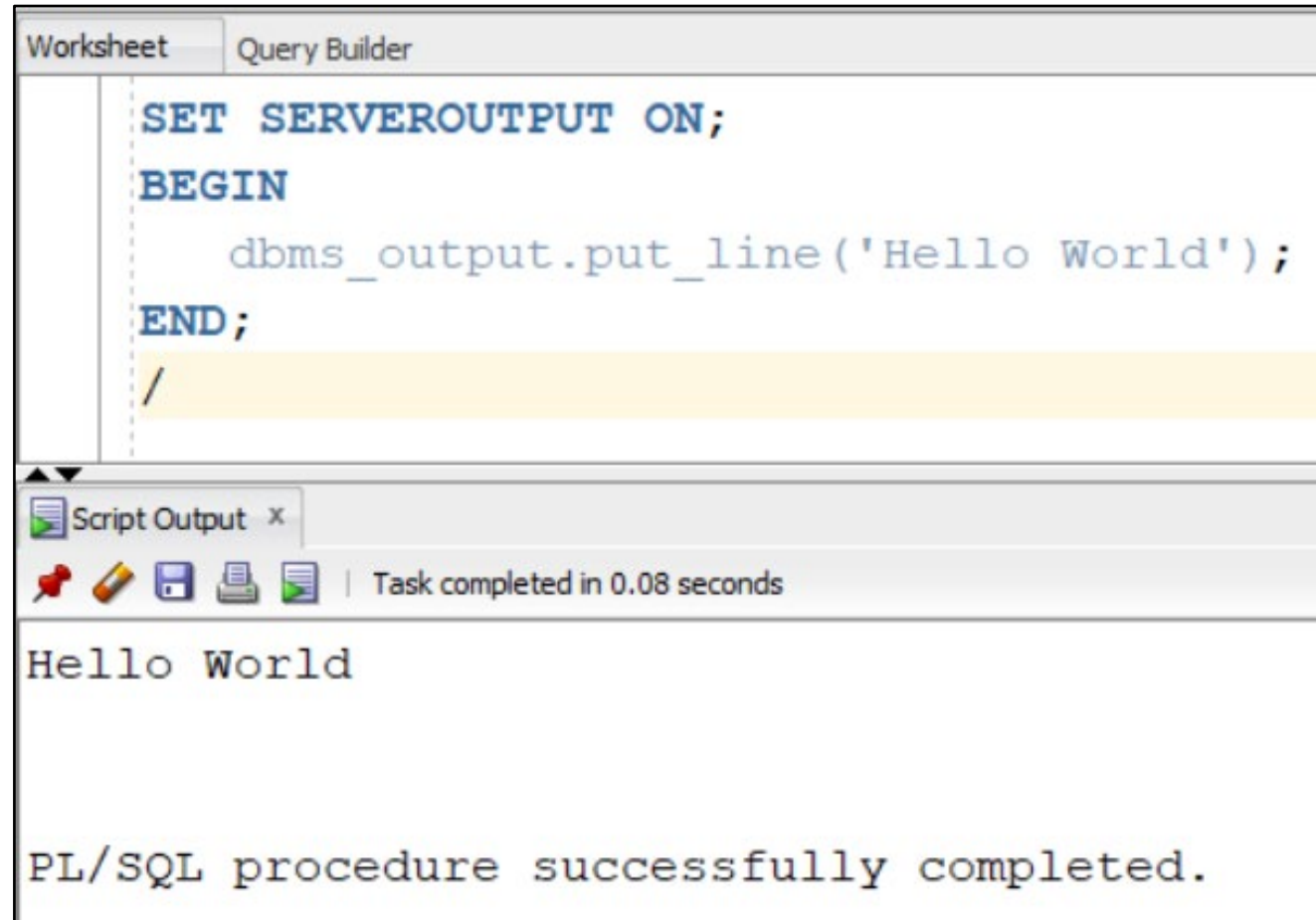
BASIC STRUCTURE : EXCEPTION-HANDLING SECTION

- The exception-handling section is used to manage exceptions (errors) that might occur during the execution of the PL/SQL block.
- Exception handling begins with the EXCEPTION keyword and can include one or more exception handlers.

Common elements in this section include:

- WHEN clauses that specify conditions under which exception handling should occur (WHEN NO_DATA_FOUND, WHEN OTHERS)
- Handling code, which can include logging, custom error messages, and recovery actions

EXAMPLE



The screenshot shows a software interface with two main panes. The top pane, titled 'Query Builder', contains a PL/SQL script. The script starts with 'SET SERVEROUTPUT ON;', followed by a 'BEGIN' block containing an indented call to 'dbms_output.put_line('Hello World');'. The block ends with 'END;' and a forward slash on a new line. The bottom pane, titled 'Script Output', shows the results of executing the script. It displays 'Hello World' on the first line and 'PL/SQL procedure successfully completed.' on the second line. Above the output text, there is a status bar with icons for a pin, a pencil, a save disk, a printer, and a document, followed by the text 'Task completed in 0.08 seconds'.

```
Worksheet | Query Builder
SET SERVEROUTPUT ON;
BEGIN
    dbms_output.put_line('Hello World');
END;
/

Script Output x
Task completed in 0.08 seconds

Hello World

PL/SQL procedure successfully completed.
```



PL/SQL DATA TYPES

PL/SQL DATA TYPES

- PL/SQL has two kinds of data types: scalar and composite.
- The scalar types are types that store single values such as number, Boolean, character, and datetime
- The composite types are types that store multiple values, for example, record and collection.

NUMBER

Represents numeric data, including integers and floating-point numbers.

You can specify precision and scale to control the number of digits before and after the decimal point.

Example: `DECLARE num_var NUMBER(10, 2);`

VARCHAR2:

Stores character strings of variable length.

Example: `DECLARE name_var VARCHAR2(50);`

CHAR:

Stores fixed-length character strings.

Padding with spaces is common to ensure a fixed length.

Example: `DECLARE code CHAR(10);`

DATE:

Stores date and time values.

Example: `DECLARE birthdate DATE;`

COMMON SCALAR DATA TYPES IN PL/SQL

BOOLEAN:

Represents Boolean values: TRUE, FALSE, or NULL.

Used for logical comparisons and conditions.

Example: DECLARE is_valid BOOLEAN := TRUE;

BINARY_INTEGER:

Used for integer values, especially as loop counters and array indexing.

Example: DECLARE counter BINARY_INTEGER := 0;

PLS_INTEGER:

Similar to BINARY_INTEGER but with automatic range checking.

Generally more efficient for integer arithmetic.

Example: DECLARE index PLS_INTEGER := 1;

COMMON SCALAR DATA TYPES IN PL/SQL



PL/SQL VARIABLES

PL/SQL VARIABLES

- PL/SQL variables are used to store and manipulate data within PL/SQL programs.
- They act as containers for holding values of various data types.

Declaring Variable

In PL/SQL, you declare variables in the DECLARE section of a PL/SQL block. The syntax for declaring variables is as follows:

```
DECLARE
    -- Declaration of variables
    variable_name datatype [NOT NULL] [:= initial_value];
    -- Additional variable declarations
BEGIN
    -- PL/SQL statements using variables
    -- ...
END;
```

DECLARING VARIABLES

```
DECLARE
```

```
emp_name VARCHAR2(50); -- Declaration of a VARCHAR2 variable
```

```
emp_age NUMBER := 30; -- Declaration of a NUMBER variable with an initial
```

```
is_active BOOLEAN; -- Declaration of a BOOLEAN variable
```

```
pi CONSTANT NUMBER := 3.14159; -- Declaration of a constant
```

```
BEGIN
```

```
-- PL/SQL statements using variables
```

```
-- ...
```

```
END;
```

NAMING RULES FOR PL/SQL VARIABLES

- **Start with a Letter**
- **Contain Alphanumeric Characters and Underscore.**
- **Maximum Length:** The maximum length for a variable name is typically 30 characters.
- **Case Insensitive:** Variable names are not case-sensitive in PL/SQL
- **Avoid Reserved Words**
- **Meaningful and Descriptive**

NAMING RULES FOR PL/SQL VARIABLES

- **Use Camel Case or Underscore Notation**
 - **Camel Case:** `employeeName`, `annualSalary`.
 - **Underscore Notation:** `employee_name`, `annual_salary`.
- **Avoid Single-Letter Variables:**

Examples of valid variable names adhering to these rules include:

employeeName

annualSalary

total_amount

is_active

emp_count

PL/SQL VARIABLES

INITIALIZING VARIABLES

- Initializing variables is essential as it assigns an initial state to the variable, which can be useful for various purposes.
 - 1) Initializing Variables at Declaration
 - 2) Initializing Variables in the Execution Section
 - 3) Initializing Variables in a Query
 - 4) Conditional Initialization

1) INITIALIZING VARIABLES AT DECLARATION

- This initial value is assigned to the variable when the block starts executing.

```
SET SERVEROUTPUT ON;

DECLARE
    emp_name VARCHAR2(50) := 'John Doe';
    emp_salary NUMBER := 50000;
BEGIN
    dbms_output.put_line(emp_name);
    dbms_output.put_line(emp_salary);
END;
/
```



2) INITIALIZING VARIABLES IN THE EXECUTION SECTION

- This is useful when you need to determine the initial value dynamically.

```
SET SERVEROUTPUT ON;

DECLARE
    emp_name VARCHAR2(50);
    emp_salary NUMBER;
BEGIN
    -- Determine values dynamically
    emp_name := 'Jane Smith';
    emp_salary := 60000;
    dbms_output.put_line(emp_name);
    dbms_output.put_line(emp_salary);
END;
/
```

3) INITIALIZING VARIABLES IN A QUERY

- Initialize variables by selecting values from the database or by using query results.

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
    emp_name VARCHAR2(50);
```

```
BEGIN
```

```
    SELECT first_name INTO emp_name FROM employees WHERE employee_id = 101;
```

```
    dbms_output.put_line('First Name : ' || emp_name );
```

```
END;
```

4) CONDITIONAL INITIALIZATION

- Conditionally initialize variables based on certain conditions within your PL/SQL code.

```
SET SERVEROUTPUT ON;
DECLARE
    emp_name VARCHAR2(50);
    emp_salary NUMBER;
    emp_id NUMBER:=101;
BEGIN
    IF emp_id !=101 THEN
        emp_name := 'John Doe';
        emp_salary := 50000;
    ELSE
        emp_name := 'Jane Smith';
        emp_salary := 60000;
    END IF;

    dbms_output.put_line('Name : ' || emp_name );
END;
```

VARIABLE ATTRIBUTE

- When you declare a PL/SQL variable to hold the column values, it must be of correct data types and precision, otherwise error will occur on execution.
- Rather than hard coding the data type and precision of a variable.
- PL/SQL provides the facility to declare a variable without having to specify a particular data type using %TYPE and %ROWTYPE attributes.
- These two attributes allow us to specify a variable and have that variable data type be defined by a table/view column or a PL/SQL package variable.
- A % sign serves as the attribute indicator. This method of declaring variables has an advantage as the user is not concerned with writing and maintaining code.

VARIABLE ATTRIBUTE : %TYPE

- The %TYPE attribute is used to declare variables according to the already declared variable or database column.
- It is used when you are declaring an individual variable, not a record.
- The data type and precision of the variable declared using %TYPE attribute is the same as that of the column that is referred from a given table.
- This is particularly useful when declaring variables that will hold database values.
- When using the %TYPE keyword, the name of the columns and the table to which the variable will correspond must be known to the user.
- These are then prefixed with the variable name. If some previously declared variable is referred then prefix that variable name to the %TYPE attribute.

```
<var_name> <tab_name>.<column_name>%TYPE;
```

VARIABLE ATTRIBUTE : %ROWTYPE

- The %ROWTYPE attribute is used to declare a record type that represents a row in a table.
- The record can store an entire row or some specific data selected from the table.
- A column in a row and corresponding fields in a record have the same name and data types.

```
<var_name> <table_name>%ROWTYPE;
```



CURSORS

CURSORS

- In Oracle Database, a cursor is a database object that allows you to retrieve and manipulate rows of data in a result set.
- Cursors are primarily used in SQL and PL/SQL to work with query results or to perform row-level operations on a set of data.

There are two main types of cursors in Oracle Database:

- Implicit Cursors
- Explicit Cursors

IMPLICIT CURSORS

- These cursors are automatically created by the Oracle database engine whenever a SQL statement is executed in a PL/SQL block.
- Implicit cursors are used for single-row queries and don't require explicit declaration or management.
- Examples of implicit cursors include the SQL%ROWCOUNT and SQL%FOUND attributes, which provide information about the outcome of the most recently executed SQL statement.

EXPLICIT CURSORS

- Explicit cursors are user-defined and require explicit declaration, opening, fetching, and closing operations.
- They are typically used for more complex queries that return multiple rows.
- Explicit cursors provide more control and flexibility over the result set and allow you to iterate through the rows one at a time.
- You can declare explicit cursors using the `CURSOR` keyword and use them in PL/SQL code to fetch and process data.

1) DECLARE THE CURSOR

- To declare an explicit cursor, you use the CURSOR declaration. You specify the SQL query that defines the result set the cursor will work with.

sql

```
CURSOR cursor_name IS  
    SELECT column1, column2  
    FROM table_name  
    WHERE conditions;
```

Example:

sql

```
CURSOR employee_cursor IS  
    SELECT employee_id, first_name, last_name  
    FROM employees  
    WHERE department_id = 10;
```

2) OPEN THE CURSOR:

- Before you can fetch data from the cursor, you need to open it using the OPEN statement:

sql

```
OPEN cursor_name;
```

Example:

sql

```
OPEN employee_cursor;
```

3) FETCH DATA

- You can use the FETCH statement to retrieve rows from the cursor one at a time into variables. You typically use a LOOP construct to iterate through the result set.

sql

```
FETCH cursor_name INTO variable1, variable2, ...;
```

Example:

sql

```
FETCH employee_cursor INTO emp_id, emp_first_name, emp_last_name;
```

4) PROCESSING DATA:

- You can use the FETCH statement to retrieve rows from the cursor one at a time into variables. You typically use a LOOP construct to iterate through the result set.

```
-- Process the data, e.g., print it
DBMS_OUTPUT.PUT_LINE('Employee ID: ' || emp_id || ',
                      Name: ' || emp_first_name || ' ' || emp_last_name);
```

5) EXIT THE LOOP

- You typically use a condition to exit the loop when there are no more rows to fetch. One common condition is to check the %NOTFOUND attribute of the cursor.

```
EXIT WHEN cursor_name%NOTFOUND;
```

6) CLOSE THE CURSOR:

- After you've finished processing the result set, you should close the cursor to release the associated resources.

sql

```
CLOSE cursor_name;
```

Example:

sql

```
CLOSE employee_cursor;
```



OPERATORS IN PL/SQL

OPERATORS IN PL/SQL

- PL/SQL supports a variety of operators for performing operations on data, including arithmetic, comparison, logical, and concatenation operators.

Arithmetic Operators:

- + (Addition): Adds two numbers.
- - (Subtraction): Subtracts the right operand from the left operand.
- * (Multiplication): Multiplies two numbers.
- / (Division): Divides the left operand by the right operand.
- % (Modulus): Returns the remainder when the left operand is divided by the right operand.

OPERATORS IN PL/SQL

Comparison Operators:

- = (Equal to): Compares two values for equality.
- != or <> (Not Equal to): Compares two values for inequality.
- < (Less Than): Checks if the left operand is less than the right operand.
- > (Greater Than): Checks if the left operand is greater than the right operand.
- <= (Less Than or Equal to): Checks if the left operand is less than or equal to the right operand.
- >= (Greater Than or Equal to): Checks if the left operand is greater than or equal to the right operand.

OPERATORS IN PL/SQL

Logical Operators:

- AND: Logical AND operator.
- OR: Logical OR operator.
- NOT: Logical NOT operator.

Concatenation Operator:

|| (Double Pipe): Concatenates two strings or values together.

Others:

IS NULL: Checks if a value is NULL.

IS NOT NULL: Checks if a value is not NULL.



CONDITIONAL CONTROL STRUCTURES

CONDITIONAL CONTROL STRUCTURES

- Conditional control structures in PL/SQL allow you to make decisions based on specified conditions and execute different blocks of code accordingly.
- These structures help you control the flow of your program by branching to different code segments based on the evaluation of conditions.
- Here are the primary conditional control structures in PL/SQL
 - IF-THEN Statement
 - IF-THEN-ELSE Statement
 - IF-THEN-ELSIF-ELSE Statement
 - CASE Statement

IF-THEN STATEMENT

- The basic form of conditional control, used for simple conditional branching.

```
IF condition THEN
    -- Code to execute if the condition is true
END IF;
```

IF-THEN STATEMENT

- Extends the basic IF-THEN structure to handle an alternative code block when the condition is false..

```
IF condition THEN
    -- Code to execute if the condition is true
ELSE
    -- Code to execute if the condition is false
END IF;
```

IF-THEN-ELSIF-ELSE STATEMENT:

- Allows you to evaluate multiple conditions sequentially and execute the block of code associated with the first true condition.

```
IF condition1 THEN
    -- Code to execute if condition1 is true
ELSIF condition2 THEN
    -- Code to execute if condition2 is true
ELSE
    -- Code to execute if none of the conditions are true
END IF;
```

CASE STATEMENT

- Provides a way to perform multiple conditional checks in a concise manner.
- Useful when you have multiple conditions to test.

```
CASE
  WHEN condition1 THEN
    -- Code to execute if condition1 is true
  WHEN condition2 THEN
    -- Code to execute if condition2 is true
  ELSE
    -- Code to execute if none of the conditions are
END CASE;
```



LOOP CONTROL STRUCTURES

LOOP CONTROL STRUCTURES

- PL/SQL (Procedural Language for SQL) loops are control structures that allow you to repeatedly execute a block of code based on specific conditions or for a defined number of iterations.
- Loops are essential in programming for performing repetitive tasks, processing data, and making decisions based on changing conditions.
- In PL/SQL, there are several types of loops:
 - Basic Loop / Exit Loop
 - While Loop
 - For Loop
 - Cursor For Loop

BASIC LOOP / EXIT LOOP

- PL/SQL exit loop is used when a set of statements is to be executed at least once before the termination of the loop.
- The LOOP statement creates an unconditional loop that repeats a block of code indefinitely until it's manually terminated using an EXIT statement or some other condition within the loop.
- LOOP is often used when you want to create an infinite loop and break out of it when a certain condition is met.

```
LOOP
  -- Code to execute repeatedly
  EXIT WHEN condition; -- Exit the loop when a condition is met
END LOOP;
```

```
DECLARE
  num NUMBER := 1;
BEGIN
  LOOP
    DBMS_OUTPUT.PUT_LINE('Number: ' || num);
    num := num + 1;
    EXIT WHEN num > 5;
  END LOOP;
END;
/
```

WHILE LOOP

- WHILE: The WHILE loop executes a block of code as long as a specified condition is true.
- It checks the condition before each iteration, and if the condition becomes false, the loop terminates.

```
WHILE condition LOOP
    -- Code to execute repeatedly while the condition is true
END LOOP;
```

```
DECLARE
    counter NUMBER := 1;
BEGIN
    WHILE counter <= 5 LOOP
        DBMS_OUTPUT.PUT_LINE('Counter: ' || counter);
        counter := counter + 1;
    END LOOP;
END;
/
```

FOR LOOP

- FOR LOOP: The FOR loop is used when you know the number of iterations in advance.
- It iterates over a range of values, such as numbers or elements in a collection, and is commonly used for iterating a specific number of times.

```
FOR iterator IN range LOOP
    -- Code to execute for each iteration
END LOOP;
```

```
DECLARE
    total NUMBER := 0;
BEGIN
    FOR i IN 1..5 LOOP
        total := total + i;
    END LOOP;
    DBMS_OUTPUT.PUT_LINE('Sum of numbers 1 to 5: ' || total);
END;
/
```

CURSOR FOR LOOP

- CURSOR FOR LOOP: This loop is specifically designed for iterating through the result set of a cursor. It simplifies the process of fetching and processing rows from a query result.

```
FOR row_variable IN cursor_name LOOP
    -- Code to process each row fetched by the cursor
END LOOP;
```

```
SET SERVEROUTPUT ON;

DECLARE
    CURSOR emp_cursor IS
        SELECT employee_id, first_name FROM employees;
    emp_rec emp_cursor%ROWTYPE;
BEGIN
    FOR emp_rec IN emp_cursor LOOP
        DBMS_OUTPUT.PUT_LINE('Employee ID: ' || emp_rec.employee_id || ', Name: ' || emp_rec.first_name);
    END LOOP;
END;
```

/



PL/SQL PROCEDURE

PL/SQL PROCEDURE

- A Procedure in PL/SQL is a subprogram unit that consists of a group of PL/SQL statements that can be called by name.
- Each procedure in PL/SQL has its own unique name by which it can be referred to and called. This subprogram unit in the Oracle database is stored as a database object.

```
CREATE [OR REPLACE] PROCEDURE procedure_name
    [ (parameter [,parameter]) ]

IS
    [declaration_section]

BEGIN
    executable_section

[EXCEPTION
    exception_section]

END [procedure_name];
```

PL/SQL PROCEDURE

Based on procedure purpose parameters are classified as,

IN Parameter

- This parameter is used for giving input to the subprograms.
- It is a read-only variable inside the subprograms. Their values cannot be changed inside the subprogram.
- In the calling statement, these parameters can be a variable or a literal value or an expression, for example, it could be the arithmetic expression like '5*8' or 'a/b' where 'a' and 'b' are variables.
- By default, the parameters are of IN type.

PL/SQL PROCEDURE : IN PARAMETER

IN parameters are used to pass values from the calling program (the caller) to the called program (the callee).

```
CREATE OR REPLACE PROCEDURE calculate_square(number_to_square IN NUMBER)
AS
    result_square NUMBER;
BEGIN
    -- Calculate the square of the input number
    result_square := number_to_square * number_to_square;
    DBMS_OUTPUT.PUT_LINE('Square of ' || number_to_square || ' is ' || result_square);
END calculate_square;
/
```

Callee

2

Caller

```
DECLARE
    number_to_square_value NUMBER := 7;
    -- Specify the number for which you want to calculate the square.
BEGIN
    -- Call the procedure, passing the number_to_square as an IN parameter
    calculate_square(number_to_square_value);
END;
/
```

1

PL/SQL PROCEDURE : OUT PARAMETER

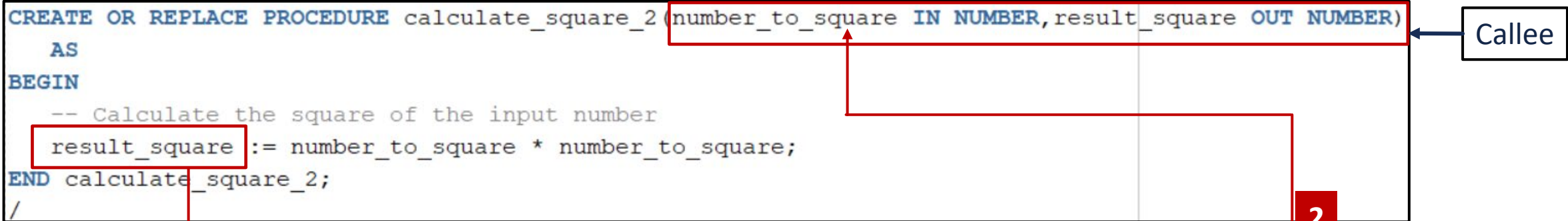
OUT Parameter

- This parameter is used for getting output from the subprograms.
- It is a read-write variable inside the subprograms. Their values can be changed inside the subprograms.
- In the calling statement, these parameters should always be a variable to hold the value from the current subprograms.

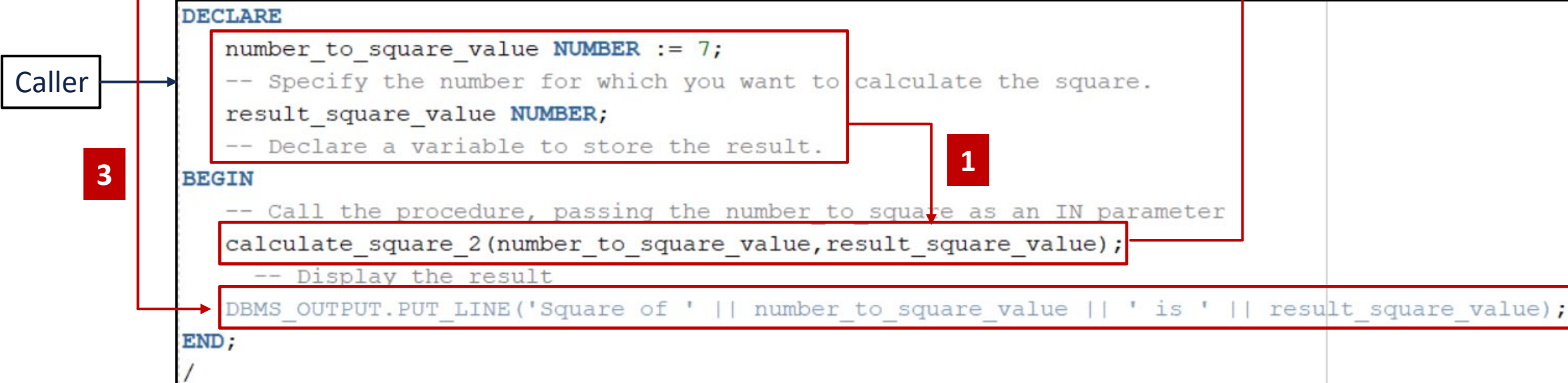
PL/SQL PROCEDURE : OUT PARAMETER

OUT parameters are used to return values from the called program (callee) back to the calling program (caller).

```
CREATE OR REPLACE PROCEDURE calculate_square_2(number_to_square IN NUMBER,result_square OUT NUMBER)
AS
BEGIN
    -- Calculate the square of the input number
    result_square := number_to_square * number_to_square;
END calculate_square_2;
/
```



```
DECLARE
    number_to_square_value NUMBER := 7;
    -- Specify the number for which you want to calculate the square.
    result_square_value NUMBER;
    -- Declare a variable to store the result.
BEGIN
    -- Call the procedure, passing the number to square as an IN parameter
    calculate_square_2(number_to_square_value,result_square_value);
    -- Display the result
    DBMS_OUTPUT.PUT_LINE('Square of ' || number_to_square_value || ' is ' || result_square_value);
END;
/
```



PL/SQL PROCEDURE : IN OUT PARAMETER

IN OUT Parameter

- This parameter is used for both giving input and for getting output from the subprograms.
- It is a read-write variable inside the subprograms. Their values can be changed inside the subprograms.
- In the calling statement, these parameters should always be a variable to hold the value from the subprograms.

PL/SQL PROCEDURE : IN OUT PARAMETER

- IN OUT parameters combine the features of both IN and OUT parameters.

```
CREATE OR REPLACE PROCEDURE calculate_square_3(s_number IN OUT NUMBER)
AS
BEGIN
    -- Calculate the square of the input number
    s_number := s_number * s_number;
END calculate_square_3;
/
```

3

4

```
DECLARE
    number_to_square_value NUMBER := 7;
    -- Specify the number for which you want to calculate the square.
BEGIN
    -- Call the procedure, passing the number to square as an IN parameter
    calculate_square_3(number_to_square_value);
    -- Display the result
    DBMS_OUTPUT.PUT_LINE('Square - ' || number_to_square_value);
END;
/
```

1

2

5

EXECUTING A PL/SQL PROCEDURE

The following shows the syntax for executing a procedure:

```
procedure_name( arguments);
```

```
procedure_name( arguments);
```

PL/SQL DROP PROCEDURE

```
DROP PROCEDURE procedure_name;
```

```
DROP PROCEDURE UpdateCourse;
```



PL/SQL FUNCTION

PL/SQL FUNCTION

- Function can be used as a part of SQL expression i.e. we can use them with select/update/merge commands.
- One most important characteristic of a function is that, unlike procedures, it must return a value.

```
CREATE [OR REPLACE] FUNCTION function_name
    [ (parameter [,parameter]) ]

    RETURN return_datatype

IS | AS

    [declaration_section]

BEGIN
    executable_section

[EXCEPTION
    exception_section]

END [function_name];
```

PL/SQL FUNCTION

```
SET SERVEROUTPUT ON;
```

```
CREATE OR REPLACE FUNCTION calculate_square(number_to_square IN NUMBER)
```

2

```
RETURN NUMBER
```

```
IS
```

```
    result NUMBER;
```

```
BEGIN
```

```
    result := number_to_square * number_to_square ;
```

3

```
    RETURN result;
```

4

```
END;
```

```
/
```

```
DECLARE
```

```
    answer number(2);
```

```
BEGIN
```

1

```
    answer := calculate_square(5);
```

5

```
    dbms_output.put_line('Square is: ' || answer);
```

```
END;
```

```
/
```

Aspect	Procedure	Function
Return Value	Does not return a value directly.	Always returns a single value.
Usage in SQL	Typically used for actions, not in SQL.	Can be used directly in SQL queries.
Return Statement	Not required, as they don't return.	Requires a RETURN statement.
Modifying Data	Commonly used for data modification.	Used for calculations, no data change.
Parameters	Can accept input parameters.	Can also accept input parameters.
Calling Syntax	Called using CALL or PL/SQL block.	Called like any SQL expression.

PL/SQL PROCEDURE VS. FUNCTION: KEY DIFFERENCES



PL/SQL TRIGGER

PL/SQL TRIGGER

- Like a stored procedure, a trigger is a named PL/SQL unit that is stored in the database and can be invoked repeatedly.
- Unlike a stored procedure, you can enable and disable a trigger, but you cannot explicitly invoke it.
- While a trigger is enabled, the database automatically invokes it—that is, the trigger fires—whenever its triggering event occurs.
- While a trigger is disabled, it does not fire.
- Triggers are written to be executed in response to any of the following events.
 - A database manipulation (DML) statement (DELETE, INSERT, or UPDATE).
 - A database definition (DDL) statement (CREATE, ALTER, or DROP).

PL/SQL TRIGGER

```
CREATE [OR REPLACE] TRIGGER trigger_name
{BEFORE | AFTER } triggering_event ON table_name
[FOR EACH ROW]
[FOLLOWS | PRECEDES another_trigger]
[ENABLE / DISABLE ]
[WHEN condition]
DECLARE
    declaration statements
BEGIN
    executable statements
EXCEPTION
    exception_handling statements
END;
```

- CREATE [OR REPLACE] TRIGGER: This is the SQL statement used to create a new trigger or replace an existing one. If you use OR REPLACE, it means that if a trigger with the same name already exists, it will be replaced by the new trigger.
- trigger_name: This is the name of the trigger you want to create.
- {BEFORE | AFTER } triggering_event: Specifies whether the trigger should execute before or after the triggering event occurs. The triggering event can be one of the following: INSERT, UPDATE, DELETE, or TRUNCATE.
- ON table_name: Specifies the table or view on which the trigger should be created. The trigger will be associated with this table or view.
- [FOR EACH ROW]: This optional clause specifies whether the trigger should execute once for each row affected by the triggering event (FOR EACH ROW) or once for the entire SQL statement (FOR EACH STATEMENT). If omitted, the trigger defaults to FOR EACH STATEMENT.
- [FOLLOWS | PRECEDES another_trigger]: Also optional, this clause is used to specify the firing order of triggers on the same table. You can indicate that one trigger should execute before or after another trigger.
- [ENABLE / DISABLE]: Allows you to enable or disable the trigger. Disabled triggers won't execute until they are enabled.
- [WHEN condition]: An optional condition that determines whether the trigger should execute. If the condition evaluates to TRUE, the trigger fires; otherwise, it doesn't.
- DECLARE: This is an optional section where you can declare local variables, cursors, or other PL/SQL constructs that you might need within the trigger body.
- BEGIN: Marks the beginning of the trigger body, where you write the PL/SQL code that will execute when the trigger is fired. This section typically contains the logic that should occur in response to the triggering event.
- EXCEPTION: This is also an optional section where you can handle exceptions that may occur within the trigger body. You can include specific error-handling logic here.
- END;: Marks the end of the trigger.



VIEW

VIEW

- An Oracle VIEW, in essence, is a virtual table that does not physically exist.
- Rather, it is created by a query joining one or more tables.
- A view derives its data from the tables on which it is based.
- These tables are called base tables.
- Views can be based on actual tables or another view also.
- Whatever DML operations you performed on a view they actually affect the base table of the view.
- You can treat views same as any other table.
- You can Query, Insert, Update and delete from views, just as any other table.
- Views are very powerful and handy since they can be treated just like any other table but do not occupy the space of a table.

VIEW

- The syntax for the CREATE VIEW Statement in Oracle/PLSQL is

```
CREATE [OR REPLACE] VIEW view_name AS  
SELECT column1, column2, ...  
FROM table_name  
WHERE condition;
```

view_name

- The name of the Oracle VIEW that you wish to create.

WHERE conditions

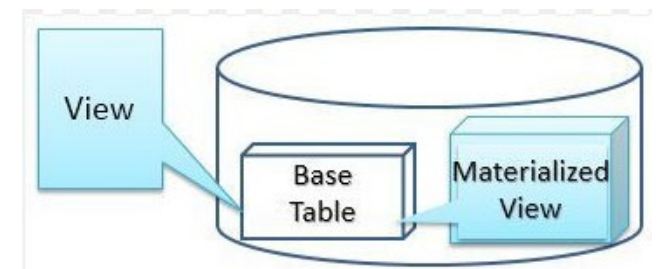
- Optional. The conditions that must be met for the records to be included in the VIEW.



MATERIALIZED VIEW

MATERIALIZED VIEW

- A Materialized View is a database object that contains the results of a query.
- It is different from a regular view in that the query result is cached as a concrete (“materialized”) table that may be updated from the original base tables from time to time.
- This enables much more efficient access, at the cost of extra storage and of some data being potentially out-of-date.
- **Performance improvement:** Querying a materialized view commonly performs better than querying the source table for the same aggregation function(s).
- **Freshness:** A materialized view query always returns the most up-to-date results, independent of when materialization last took place.
- **Cost reduction:** Querying a materialized view consumes fewer resources from the cluster than doing the aggregation over the source table



DIFFERENCE BETWEEN MATERIALIZED VIEW AND VIEW

View	Materialized Views(Snapshots)
View is nothing but the logical structure of the table which will retrieve data from 1 or more table.	Materialized views(Snapshots) are also logical structure, but data is physically stored in database.
Data access is not as fast as materialized views	Data retrieval is fast as compared to simple view because data is accessed from directly physical location
There are 2 types of views: 1.Simple View 2.Complex view	There are following types of Materialized views: 1.Refresh on Auto 2.Refresh on demand
In Application-level views are used to restrict data from database	Materialized Views are used in Data Warehousing.

MATERIALIZED VIEW

- The syntax for creating a normal view in an Oracle database is as follows:

```
CREATE VIEW view_name AS SELECT ... FROM ... WHERE ...;
```

- To create a materialized view instead of a normal view you add the MATERIALIZED keyword:

```
CREATE MATERIALIZED VIEW view_name AS SELECT ... FROM ... WHERE ...;
```

MATERIALIZED VIEW

- When creating a materialized view, you should think about and decide on three aspects of materialized views:
 - The populated method,
 - The refresh method,
 - The refresh interval,

THE POPULATED METHOD

This option determines how the materialized view is initially populated.

- **IMMEDIATE:** The materialized view is populated immediately after it's created.
- **DEFERRED:** The materialized view is not populated immediately. You can populate it later using the REFRESH command.

```
CREATE MATERIALIZED VIEW view_name  
  BUILD [IMMEDIATE | DEFERRED]  
  AS SELECT ...;
```

THE REFRESH METHOD

- The refresh method determines how the data in the materialized view is updated or refreshed to reflect changes in the base tables.
- **COMPLETE:** This one completely rebuilds the materialized view from scratch. It drops the existing contents of the materialized view and then re-executes the query to populate it with fresh data. This method can be resource-intensive and slow, especially for large materialized views.
- **FAST:** Updates only the rows in the materialized view that have changed since the last refresh. It uses the materialized view logs on the base tables to identify the changed rows and then applies the changes to the materialized view. It can be much faster than a complete refresh, especially if there are only a few changes to the data.
- **FORCE:** Tries to perform a fast refresh if possible, but falls back to a complete refresh if necessary. This method is useful if you want to try to perform a fast refresh, but you're not sure if it will be possible due to the nature of the data or the query.

THE REFRESH METHOD

You can specify the refresh method when creating the materialized view using the REFRESH keyword:

```
CREATE MATERIALIZED VIEW view_name  
  BUILD [IMMEDIATE | DEFERRED]  
  REFRESH [COMPLETE | FAST | FORCE]  
  AS SELECT ...;
```

If you do not specify a refresh mode FORCE is the default.

THE REFRESH INTERVAL

- The refresh interval controls how often the materialized view is automatically refreshed. It determines how frequently the materialized view is updated to reflect changes in the underlying data.

Some refresh interval options in Oracle are:

- **ON COMMIT:** The materialized view is refreshed automatically every time a transaction that modifies the underlying data is committed. This interval is useful when you need to keep the materialized view up-to-date in near real-time.
- **ON DEMAND:** The materialized view is refreshed only when you explicitly request a refresh using the `DBMS_MVIEW.REFRESH`.

THE REFRESH INTERVAL

- You can specify the refresh interval when creating the materialized view by adding it to the REFRESH clause when creating the view:

```
CREATE MATERIALIZED VIEW view_name  
  BUILD [IMMEDIATE | DEFERRED]  
  REFRESH [COMPLETE | FAST | FORCE]  
  ON [COMMIT | DEMAND]  
  AS SELECT ...;
```



EXCEPTION HANDLING

EXCEPTION HANDLING - SYSTEM-DEFINED EXCEPTIONS

- Exception handling is a crucial aspect of PL/SQL (Procedural Language/Structured Query Language) programming in Oracle Database.
- It allows you to gracefully handle errors and exceptions that can occur during the execution of your PL/SQL code.
- Proper exception handling helps ensure that your application behaves predictably and doesn't terminate unexpectedly due to errors.

There are two type of exceptions:

- System-defined Exceptions
- User-defined Exceptions

EXCEPTION HANDLING- SYSTEM-DEFINED EXCEPTIONS

```
BEGIN
    -- Code in the TRY block
EXCEPTION
    WHEN exception_name1 THEN
        -- Code to handle exception_name1
    WHEN exception_name2 THEN
        -- Code to handle exception_name2
    ...
    WHEN OTHERS THEN
        -- Code to handle all other exceptions
END;
```

EXCEPTION HANDLING - SYSTEM-DEFINED EXCEPTIONS

Exception	Description
ZERO_DIVIDE	It is raised when an attempt is made to divide a number by zero.
TOO_MANY_ROWS	It is raised when a SELECT INTO statement returns more than one row.
NO_DATA_FOUND	It is raised when a select into statement returns no rows.
INVALID_CURSOR	It is raised when attempts are made to make a cursor operation that is not allowed, such as closing an unopened cursor.
INVALID_NUMBER	It is raised when the conversion of a character string into a number fails because the string does not represent a valid number.
LOGIN_DENIED	It is raised when a program attempts to log on to the database with an invalid username or password.

EXCEPTION HANDLING - USER-DEFINED EXCEPTIONS

```
DECLARE
    user_define_exception_name EXCEPTION;
BEGIN
    statement(s);
    IF condition THEN
        RAISE user_define_exception_name;
    END IF;
EXCEPTION
    WHEN user_define_exception_name THEN
        User defined statement (action) will be taken;
END;
```

Thank
You 😊

DATA MANAGEMENT- 2

Oracle for Database administration





USER MANAGEMENT



CREATE A NEW USER

CREATE A NEW USER : MANUAL SETTINGS

- 🔒 Using the Oracle “CREATE USER” statement you can create and configure new database user.
- 🔒 By using this database user, you can log on to your database.

Syntax of The Oracle Create User statement

```
CREATE USER username IDENTIFIED BY password/externally/globally  
DEFAULT TABLESPACE tablespace_name TEMPORARY TABLESPACE tablespace_name  
QUOTA size/unlimited ON tablespace_name  
PROFILE profile_name  
PASSWORD expire  
ACCOUNT lock/unlock;
```

CREATE A NEW USER : MANUAL SETTINGS

- 🔒 **CREATE** and **USER** both are the keywords followed by the username.
- 🔒 As **username**, you can give any name of your choice.
- 🔒 The **IDENTIFIED BY** clause which helps the Oracle engine in authenticating the user.
- 🔒 Password clause lets you create a local user and indicates that the user must specify a password to log on to the database.

Default Tablespace

- 🔒 In the default tablespace, you have to specify the name of tablespace where this user will create its object.
- 🔒 This is an optional clause, if you omit this clause then all the objects created by this user gets stored in the database default tablespace which is “users” tablespace.

CREATE A NEW USER : MANUAL SETTINGS

Temporary Tablespace

- 🏠 All the temporary segments created by this user get stored inside Temporary Tablespace .
- 🏠 This is an optional clause so if you omit this clause then all the temporary objects created by this user get stored in the database default temporary tablespace which is “temp” tablespace.

Quota Clause

- 🏠 Quota clause helps you in specifying the **maximum amount of space that the user can allocate in the tablespace.**
- 🏠 You can either specify a limited size such as 50M or 100M, M stands for Megabyte here or just write UNLIMITED for allocating space without any limits.
- 🏠 After the size, you have to specify the name of tablespace using ON clause.

CREATE A NEW USER : MANUAL SETTINGS

Profile Clause

- 🌐 Using profile clause you can specify the profile you want to assign to the user. By Assigning profile you can limit the amount of database resources the user can use.
- 🌐 It's also an optional clause hence if you omit this clause database will assign a default profile to your user.

Password Expire Clause

- 🌐 This is the kind of settings which force your user to change the password on the first login attempt or say before the user can log in to the database.
- 🌐 This is also an optional clause.

CREATE A NEW USER : MANUAL SETTINGS

Account Clause

- 🔒 Using Account clause you can set the status of your user account.
- 🔒 If you set the status on the lock then Oracle will create your user account but disable its access.
- 🔒 This means that you will have your user account but you cannot use it.
- 🔒 If you set the status on UNLOCK then you are all free to use it. Unlock means status will be open.
- 🔒 By default its set on UNLOCK option. It's again optional if you omit this clause.

```
CREATE USER nibm_user1 IDENTIFIED BY nibm1
DEFAULT TABLESPACE users TEMPORARY TABLESPACE temp
QUOTA 20M ON users
PROFILE default
ACCOUNT UNLOCK;
```

CREATE A NEW USER : MANUAL SETTINGS

```
CREATE USER TESTUSER1 IDENTIFIED BY u1  
DEFAULT TABLESPACE users TEMPORARY TABLESPACE temp  
QUOTA 20M ON users  
ACCOUNT UNLOCK;
```

 You can create a new user account with all oracle default settings by using just a small very simple query.

```
CREATE USER TESTUSER1 IDENTIFIED BY u1;
```

CREATE A NEW USER : MANUAL SETTINGS

🔒 Grant the CREATE SESSION system privilege to this user account – testuser1.

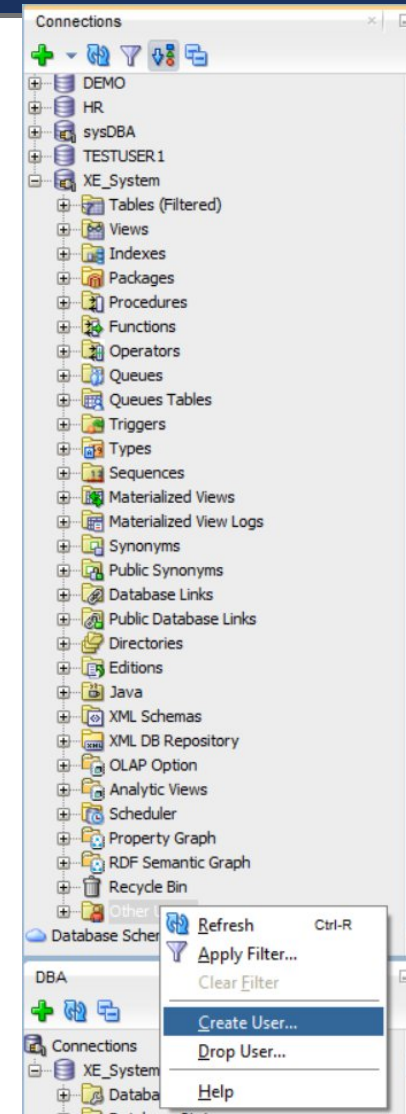
```
GRANT CREATE SESSION TO TESTUSER1;
```

🔒 Grant ALL privileges TO TESTUSER1

```
GRANT ALL PRIVILEGES TO TESTUSER1;
```

CREATE A NEW USER : SQL DEVELOPER

- 🌐 Login with SYSTEM or SYS user
- 🌐 Select Other Users 🌙 Create User



CREATE A NEW USER : SQL DEVELOPER

🔒 Enter UserName : TESTUSER2

🔒 Password : u2

Create User

User | Granted Roles | System Privileges | Quotas | SQL

User Name: TESTUSER2

New Password: ..

Confirm Password: ..

☒ Password Expired (user must change next login)

☐ Operating System User

☐ Account is Locked

☐ Edition Enabled

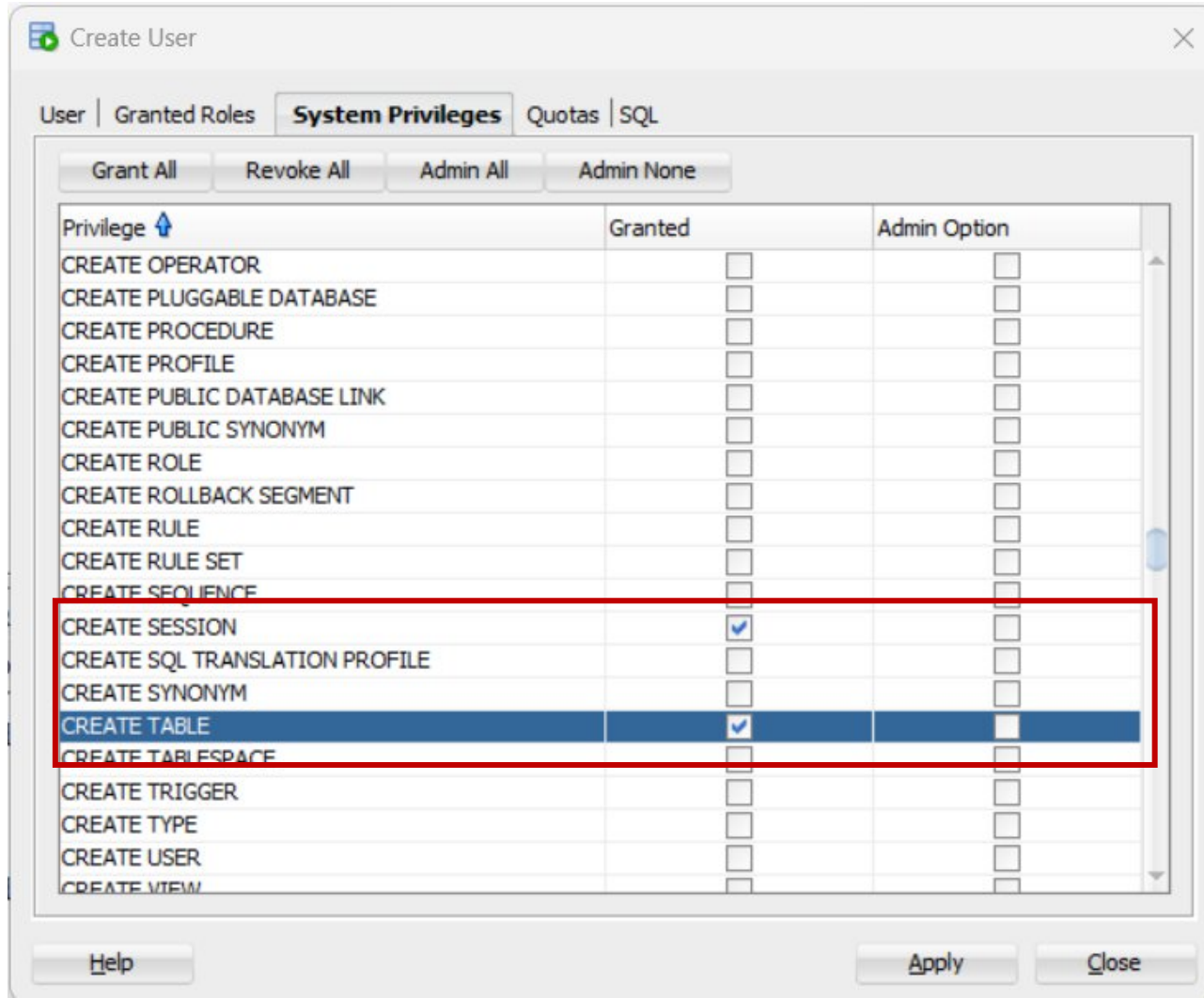
Default Tablespace: USERS

Temporary Tablespace: TEMP

Help Apply Close

CREATE A NEW USER : SQL DEVELOPER

Assign system privileges

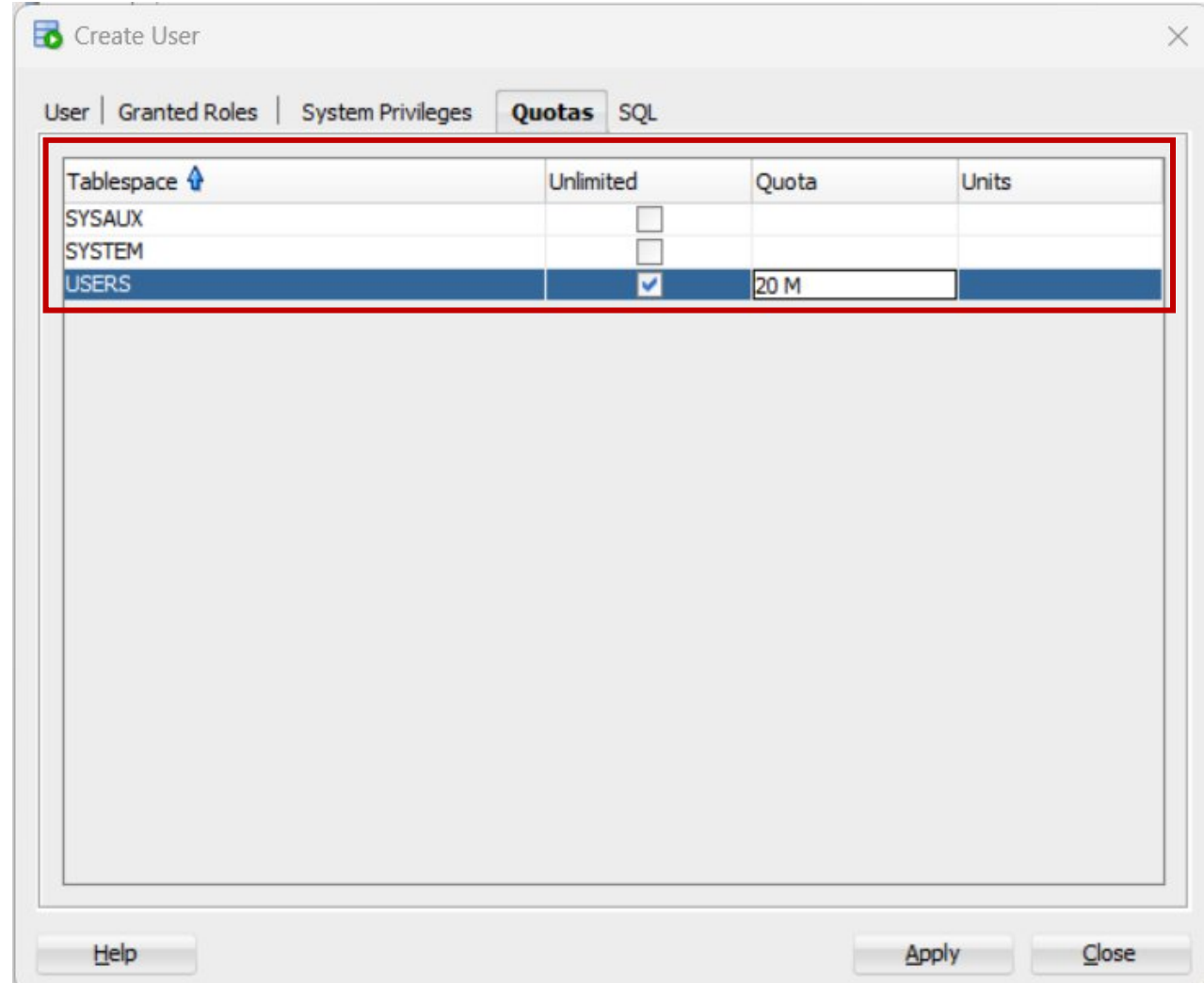


The screenshot shows the 'Create User' dialog box with the 'System Privileges' tab selected. The dialog has four tabs: 'User', 'Granted Roles', 'System Privileges', and 'Quotas'. Below the tabs are four buttons: 'Grant All', 'Revoke All', 'Admin All', and 'Admin None'. A table lists various system privileges with columns for 'Privilege', 'Granted', and 'Admin Option'. The 'CREATE TABLE' privilege is highlighted with a red box, and its 'Granted' checkbox is checked. The 'CREATE SESSION' privilege is also checked in the 'Granted' column.

Privilege	Granted	Admin Option
CREATE OPERATOR	☐	☐
CREATE PLUGGABLE DATABASE	☐	☐
CREATE PROCEDURE	☐	☐
CREATE PROFILE	☐	☐
CREATE PUBLIC DATABASE LINK	☐	☐
CREATE PUBLIC SYNONYM	☐	☐
CREATE ROLE	☐	☐
CREATE ROLLBACK SEGMENT	☐	☐
CREATE RULE	☐	☐
CREATE RULE SET	☐	☐
CREATE SEQUENCE	☐	☐
CREATE SESSION	☒	☐
CREATE SQL TRANSLATION PROFILE	☐	☐
CREATE SYNONYM	☐	☐
CREATE TABLE	☒	☐
CREATE TABLESPACE	☐	☐
CREATE TRIGGER	☐	☐
CREATE TYPE	☐	☐
CREATE USER	☐	☐
CREATE VIEW	☐	☐

CREATE A NEW USER : SQL DEVELOPER

 Assign Quotas



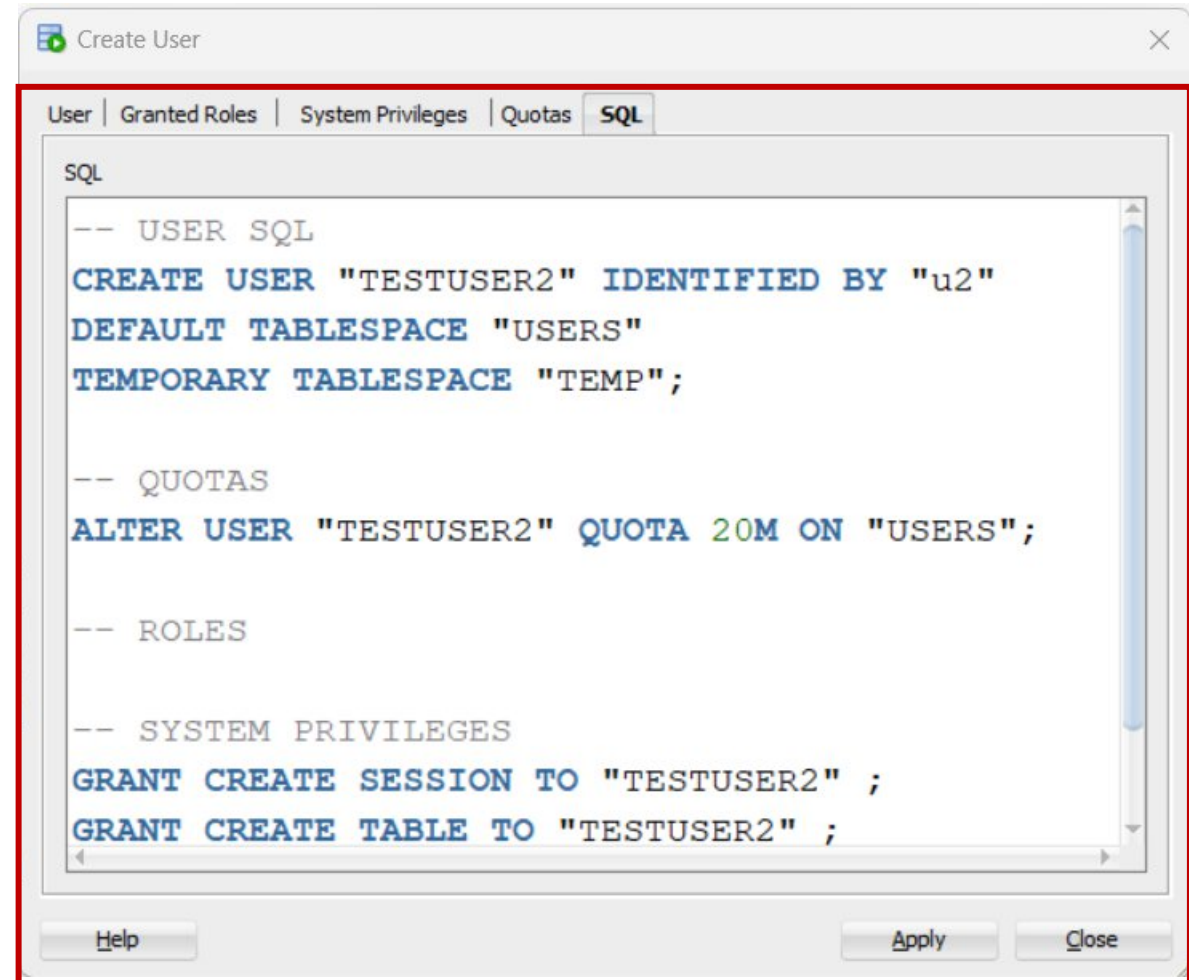
The screenshot shows the 'Create User' dialog box with the 'Quotas' tab selected. The dialog has tabs for 'User', 'Granted Roles', 'System Privileges', 'Quotas', and 'SQL'. The 'Quotas' tab contains a table with the following data:

Tablespace	Unlimited	Quota	Units
SYSAUX	☐		
SYSTEM	☐		
USERS	☒	20 M	

At the bottom of the dialog are buttons for 'Help', 'Apply', and 'Close'.

CREATE A NEW USER : SQL DEVELOPER

🔧 SQL command related to the settings





DROP USER

DROP USER

- 🌐 DROP USER statement in Oracle is used to permanently remove a database user, along with all of their associated objects and privileges.
- 🌐 When you drop a user, it effectively deletes the user account, all the tables, views, indexes, triggers, and other database objects owned by that user, and revokes all of their privileges.
- 🌐 To drop the user from database can do using following methods.
 - Drop user system privillages
 - Login with SYSTEM or sys user as sysdba

DROP USER

```
DROP USER username [CASCADE];
```

- 🌐 **username** is the name of the user you want to drop.
- 🌐 The optional **CASCADE** keyword is used to automatically drop all objects owned by the user. If you include CASCADE, Oracle will delete all objects and then drop the user. If you omit CASCADE, Oracle will not drop the user if there are any objects still associated with them, and you'll need to manually drop those objects first.



USER PRIVILEGES

USER PRIVILEGES

- 🌐 Oracle database needs some rights to perform any task.
- 🌐 These tasks can be anything, for example to execute any SQL statement or to access the objects owned by any other user or to run any PL/SQL package and many more.

There are two types of user privileges in Oracle Database.

- 🌐 System Privilege
- 🌐 Object Privilege

SYSTEM PRIVILEGE

- 🌐 System privileges are some powerful special rights given to a user to perform standard administrator tasks in the database.
- 🌐 These tasks can be any action on any schema objects for example create and drop a user or tablespace, flashback or lock any table and export as well as import the database and many more.
- 🌐 The System privileges are quite powerful set of rights therefore when in wrong hands they could result in catastrophic events.
- 🌐 Hence these should be granted very responsibly and only when absolutely necessary.

WHO CAN GRANT OR REVOKE SYSTEM PRIVILEGES

1. The most powerful administrative user in the database sys can grant and revoke any privilege from any user in the database.
2. A user with the system privilege GRANT ANY PRIVILEGE and
3. Any user who was granted a specific system privilege with the ADMIN OPTION

 All of these can grant and revoke any privilege.

 In oracle database we have three options which we can use along with the Grant statement and add some flexibility to this privilege management. These 3 options are:

- With Admin Option
- With Grant Option
- With Hierarchy Option

GRANT OR REVOKE SYSTEM PRIVILEGES

With Admin Option

With Admin Option enables the grantee to –

- Grant the privilege or role to another user or role, unless the role is a GLOBAL role
- Revoke the privilege or role from another user or role
- Alter the privilege or role to change the authorization needed to access it
- Drop the privilege or role

If you grant a system privilege or role to a user without specifying `WITH ADMIN OPTION`, and then subsequently grant the privilege or role to the user `WITH ADMIN OPTION`, then the user has the `ADMIN OPTION` on the privilege or role.

OBJECT PRIVILEGE

- 🏠 Object privileges are the rights given to a user to access and perform some actions on the database objects or the objects owned by some other user in the database.
- 🏠 By objects it means tables, views, sequences, procedures etc. and by Action it means Alter, Delete, Execute, Select, Update, References, Insert and Index.
- 🏠 Object privileges vary from object to object and an owner of the object has all the privileges on it.

An object's owner has following advantages:

- 🏠 The owner has every object privilege on the Object owned by it.
- 🏠 The privileges enjoyed by the Object owner cannot be revoked.
- 🏠 The owner can grant any number of object privileges for that particular object to other database users.

OBJECT PRIVILEGES

The syntax for Grant DCL statement for granting Object Privilege is:

```
GRANT  object privilege name(s)  ON  object name  TO  user name(s)
```

- 🔒 The statement is started with the keyword GRANT and then followed with the name of the OBJECT PRIVILEGE.
- 🔒 In the latter case you can either specify a single object privilege or give the entire list of every valid object privilege.
- 🔒 Thereafter we have another keyword ON which is followed by OBJECT NAME which can be assigned the privilege by you.
- 🔒 Then you have another keyword TO that is followed by USERNAME to which you can then assign the grant privilege.

OBJECT PRIVILEGES

DML object privilege for the Tables

- 🔒 The PRIVILLAGE_USER1 wants to select the data from the employees table owned by HR user.
- 🔒 In this case PRIVILLAGE_USER1 can either ask user HR to grant him the SELECT object privilege on employees table or ask the DBA for the same.
- 🔒 In both the case query for granting SELECT object privilege will be the same.

```
GRANT SELECT ON HR.employees TO PRIVILLAGE_USER1;
```

OBJECT PRIVILEGES

Grant multiple object privilege to a user

- 🔒 We can also grant UPDATE, INSERT, DELETE, INDEX and REFERENCES object privileges on database Object Tables.

```
GRANT UPDATE, INSERT, DELETE, INDEX ON HR.employees TO PRIVILLAGE_USER1;
```

Grant All data control language statement

- 🔒 Replace list of Object privileges by ALL keyword for example

```
GRANT ALL ON HR.employees TO PRIVILLAGE_USER1;
```

OBJECT PRIVILEGES

Object Privilege on column level

- 🌐 Oracle allows you to grant object privileges on column level also but you can only grant INSERT, UPDATE, and REFERENCES object privilege on column level.

```
GRANT UPDATE (first_name) ON HR.employees TO PRIVILLAGE_USER1;
```

WITH GRANT OPTION

- 🌐 Just like we use ADMIN OPTION flag with system privileges similarly we use Grant option flag with object privileges.
- 🌐 GRANT OPTION flag allows the user to grant an object privilege to another user.

```
GRANT UPDATE ON HR.employees TO PRIVILLAGE_USER2 WITH GRANT OPTION;
```



USER ROLES

USER ROLES

- 🔒 Usage of roles makes the managing and controlling of privileges an easy to handle process.
- 🔒 Roles are named group of privileges that you can assign to users or other roles.
- 🔒 Roles are the easiest and quickest way of granting permission to users.
- 🔒 Every role within a database has to be unique with exclusive user names as well as role names.
- 🔒 Roles are often created by Database administrators for a given database application.
- 🔒 In order for the application to run, you need to grant all necessary privileges to a secure application role.

USER ROLES

Create a NOT IDENTIFIED Role

- 🌐 Name of the role is user defined thus you can give whatever name you want.
- 🌐 Then at the end of the statement we have NOT IDENTIFIED clause, NOT IDENTIFIED clause indicates that this role is authorized by the database and no password is required to enable the role.

```
CREATE ROLE role1 NOT IDENTIFIED;
```

Role with Identified By Password clause

- 🌐 IDENTIFIED BY clause indicates that a role must be authorized by the specified method.
- 🌐 In our case the specific method is password. Followed by Identified By clause we have our password.

```
CREATE ROLE role2 IDENTIFIED BY role123;
```

ASSIGN PRIVILEGES TO THE ROLES

How to grant System Privilege to a role

Grant Create table system privilege to the role1

```
GRANT CREATE TABLE TO role1 WITH ADMIN OPTION;
```

How to grant Object Privilege to a Role.

Grant SELECT object privilege on employees table of HR user to the role1

```
GRANT SELECT ON hr.employees TO role1;
```

ASSIGN PRIVILEGES TO THE ROLES

How to Grant a Role to another Role

Role1 has “Create table” system privilege and “Select” object privilege on employees table of HR user.

Grant the Role1 to Role2

```
Grant role1 TO role2;
```

How to Grant a Role to another user

To grant a role to any user or to any role we use GRANT DCL statement.

Such as, say you want to grant role1 to PRIVILLAGE_USER1.

```
GRANT role1 TO PRIVILLAGE_USER1;
```



BACKUP [DATA PUMP]

DATABASE BACKUP

In Oracle Database, a backup is the process of creating a copy of your database or specific database components to safeguard your data against data loss, corruption, or system failures.

Logical backups:

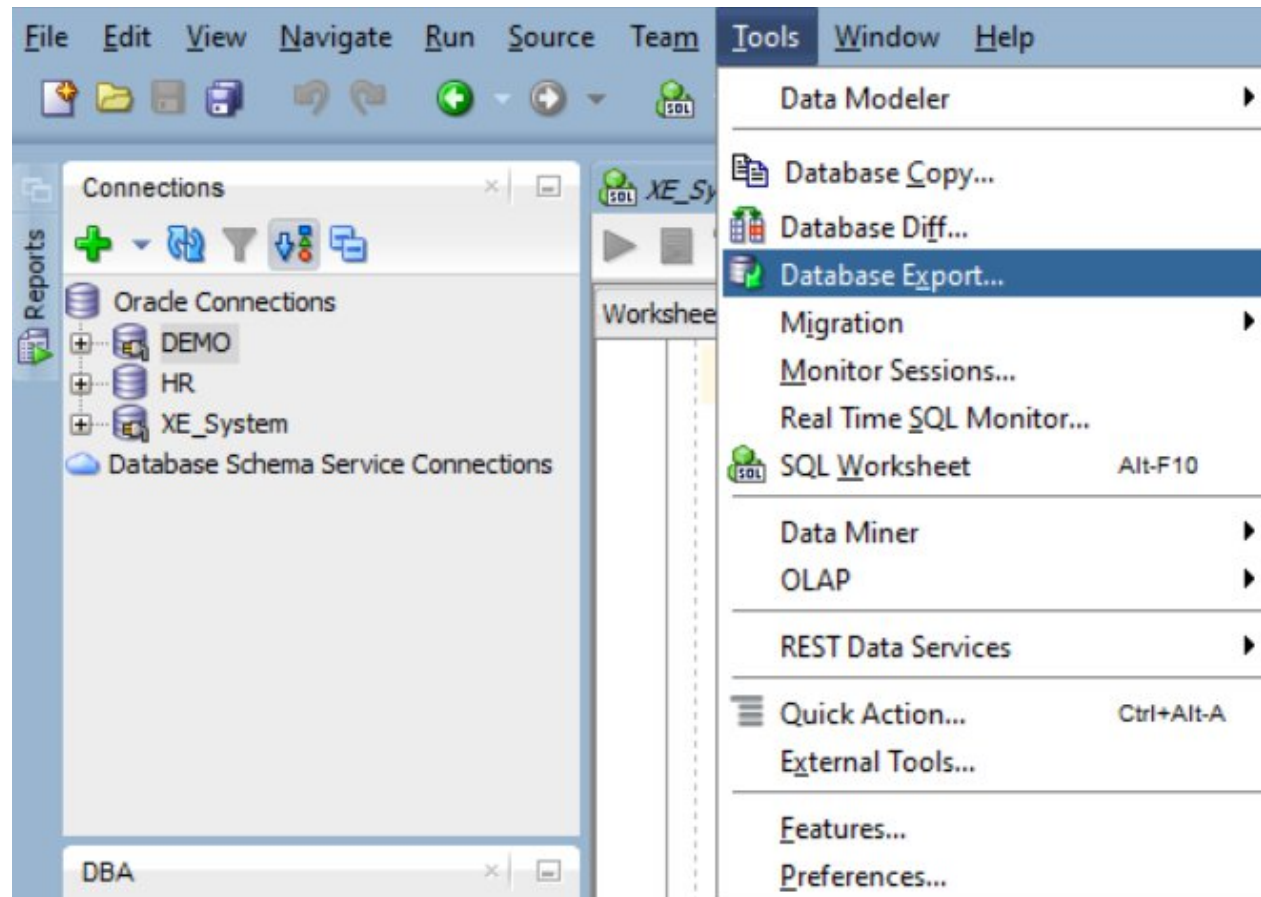
- 🗄️ These backups export database objects and data in a format that can be imported back into the database.
- 🗄️ The oracle provides different utilities to take logical backup.
 - Export / Import utility
 - Datapump utility

Physical backups:

- 🗄️ Physical backups are copies of the physical files used in storing and recovering a database.
- 🗄️ These files include data files, control files, and archived redo logs.
- 🗄️ Physical backups are the foundation of any sound backup and recovery strategy.

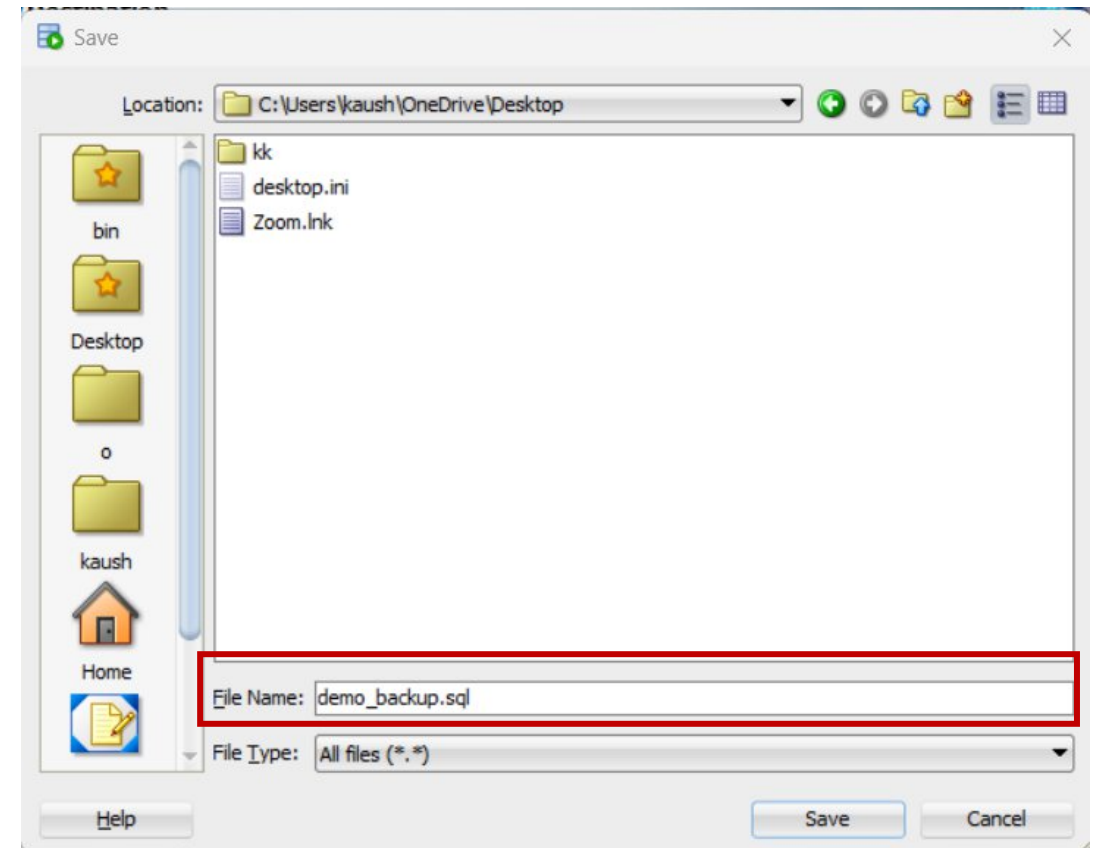
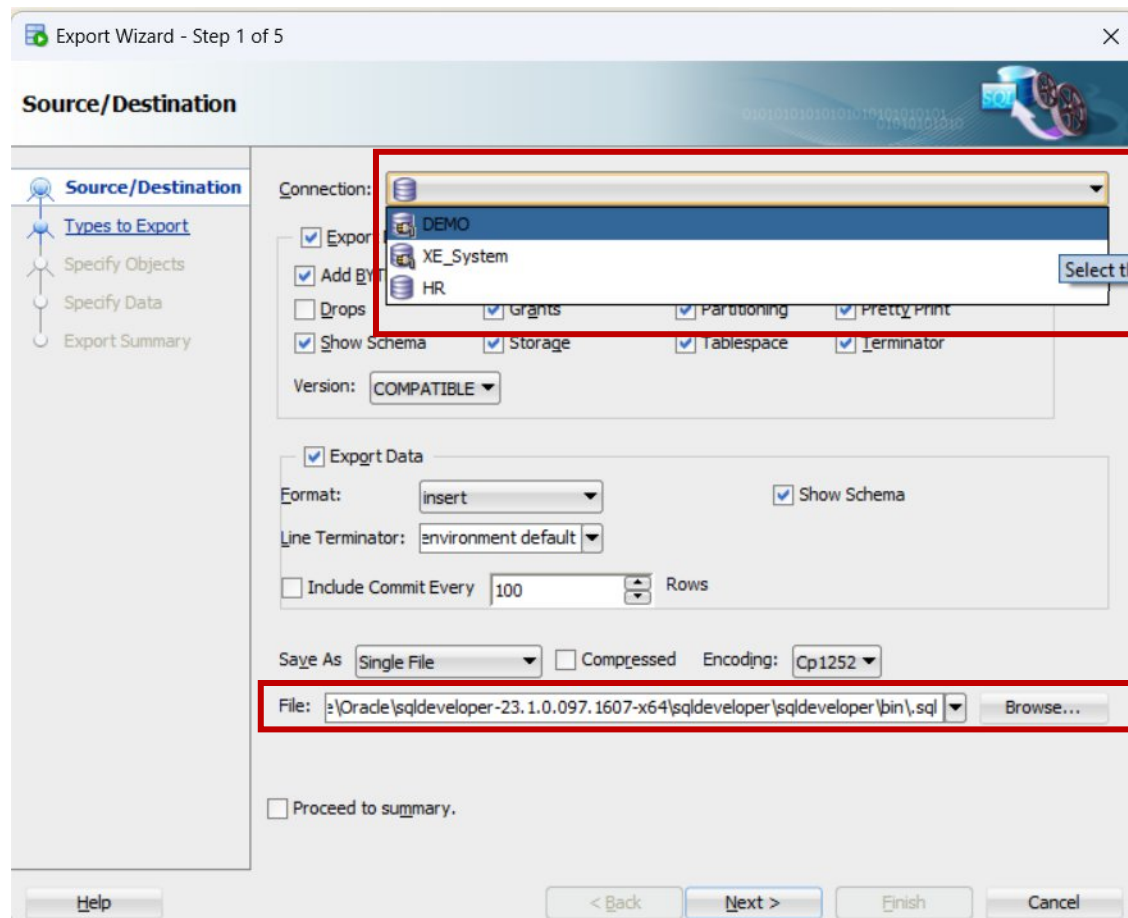
EXPORT UTILITY

1. Select Tool ☾ Database Export



EXPORT UTILITY

2. Select the connection needed to export and select path of file destination.



EXPORT UTILITY

3. Click on Next.

Export Wizard - Step 2 of 5

Types to Export

Limit the scope of your process by selecting the types of objects to include. If you do not further limit your process by selecting objects, all objects for the types selected here will be processed.

Standard Object Types

☒ Toggle All Standard

☒ Tables	☒ Types	☒ Package Spec
☒ Views	☒ Sequences	☒ Package Body
☒ Indexes	☒ Synonyms	☒ Procedures
☒ Triggers	☒ Database Links	☒ Functions
☒ Constraints	☒ Queues	☒ Operators
☒ Referential Constraints	☒ Queue Tables	
☒ Materialized Views		
☒ Materialized View Logs		

Extension Object Types

☐ Toggle All Extension

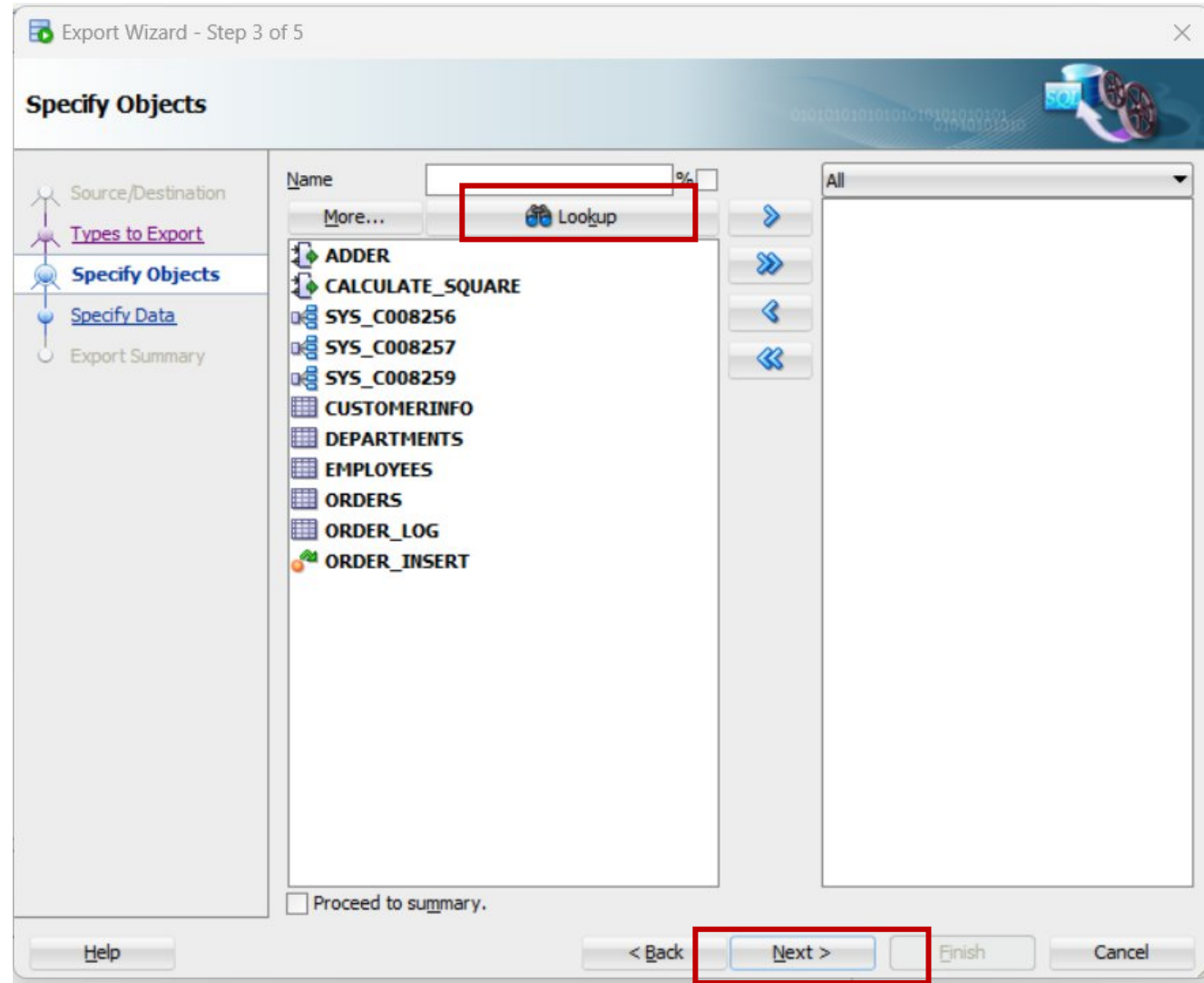
☐ Attribute Dimension	☐ Hierarchy	☐ Analytic View
--	------------------------------------	--

☐ Proceed to summary.

Help < Back Next > Finish Cancel

EXPORT UTILITY

4. Click on Next.



EXPORT UTILITY

6. Click on Next.

Export Wizard - Step 4 of 5

Specify Data

Source/Destination
Types to Export
Specify Objects
Specify Data
Export Summary

Name: % ☐

More... **Lookup**

- CUSTOMERINFO
- DEPARTMENTS
- EMPLOYEES
- ORDERS
- ORDER_LOG

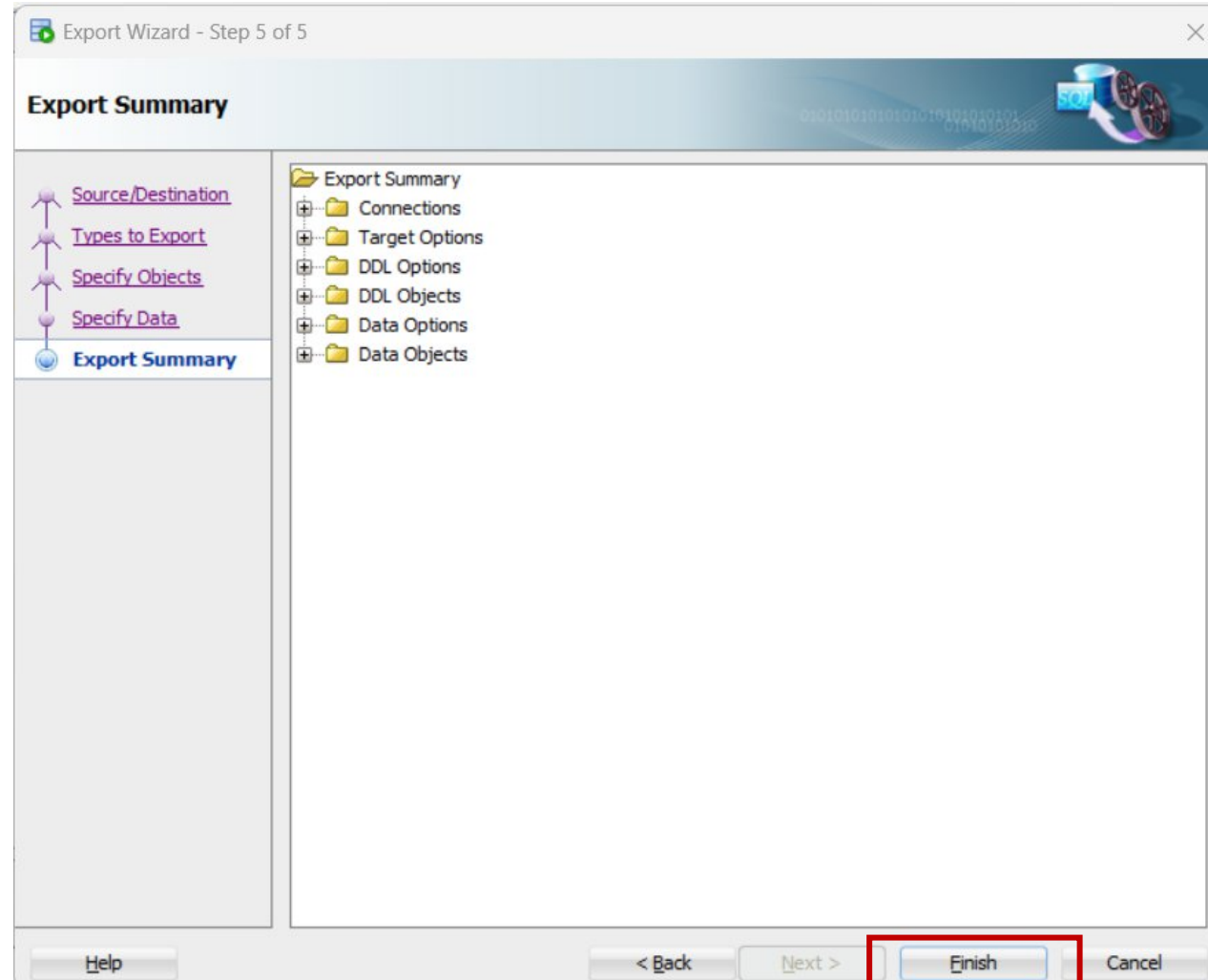
Database Object | Columns | Object Where

Global Where:

Help < Back Next > Finish Cancel

EXPORT UTILITY

7. Click on Finish.





RESTORING

IMPORT UTILITY

1. Create a new connection to import data.

New / Select Database Connection

Connection Name	Connection Details
DEMO	DEMO1@//localh...
DEMO_BACKUP	DEMO1@//localh...
HR	hr@//localhost:1...
sysDBA	sys@//localhost:...
XE_System	system@//localh...

Name: DEMO_BACKUP

Database Type: Oracle

User Info: Proxy User

Authentication Type: Default

Username: DEMO1

Password:

Role: default

Save Password: ☐

Connection Type: Basic

Details: Advanced

Hostname: localhost

Port: 1521

☐ SID: xe

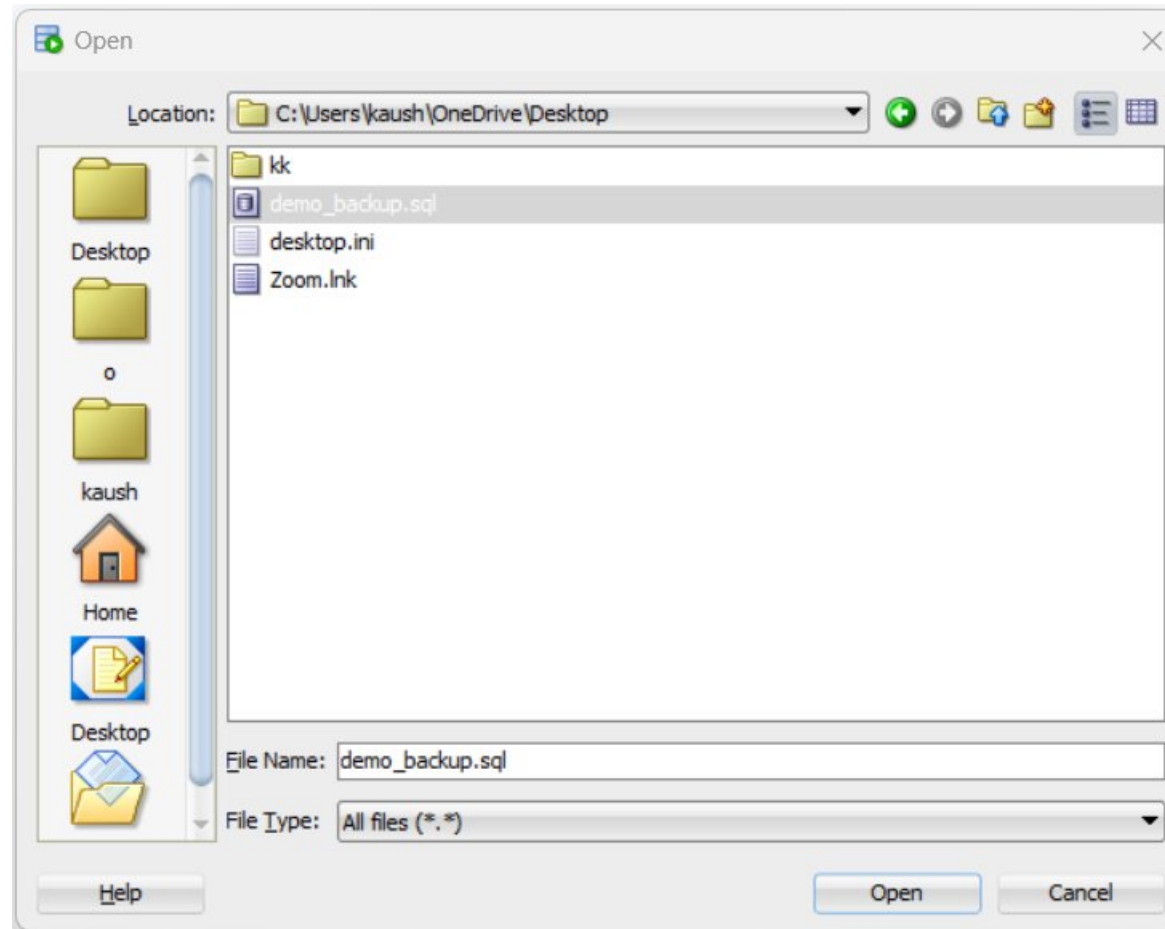
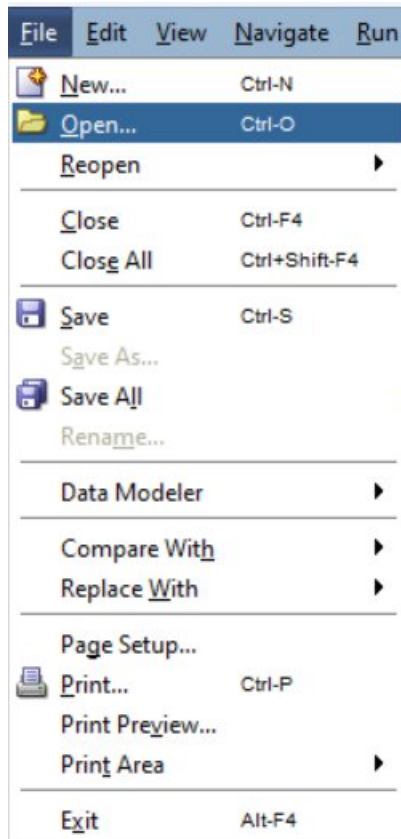
☒ Service name: XEPDB1

Status: Success

Buttons: Help, Save, Clear, Test, Connect, Cancel

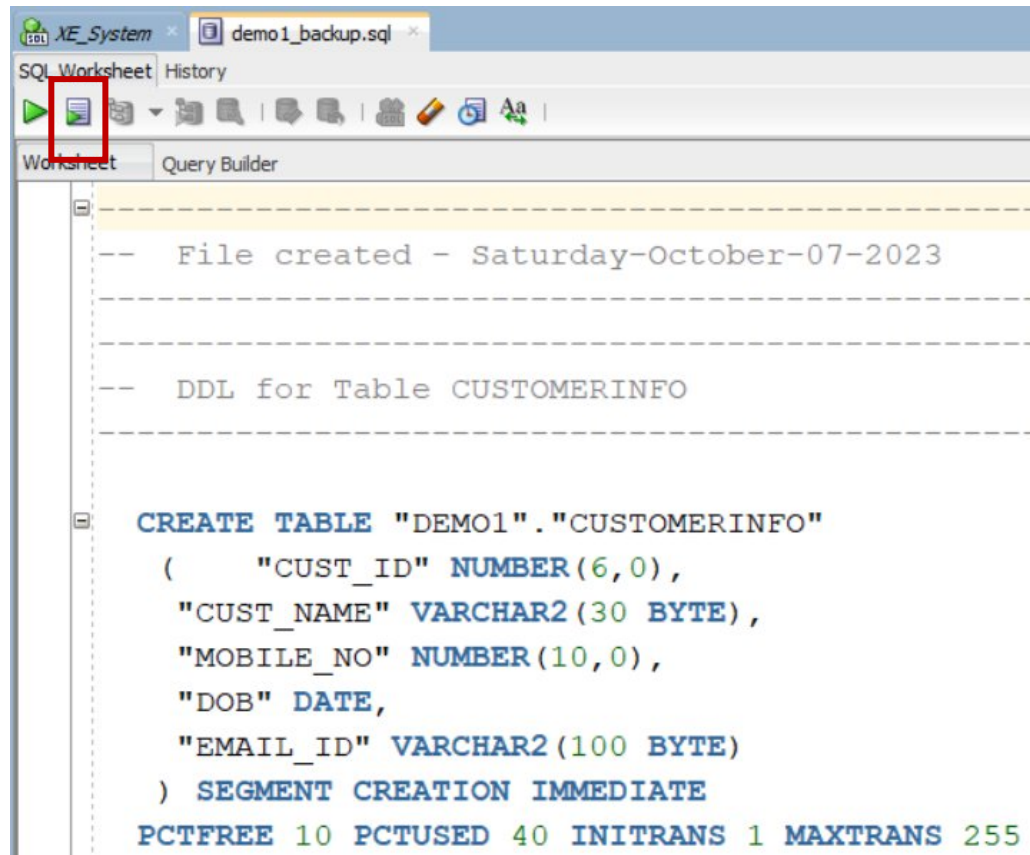
IMPORT UTILITY

3. Open export sql file.



IMPORT UTILITY

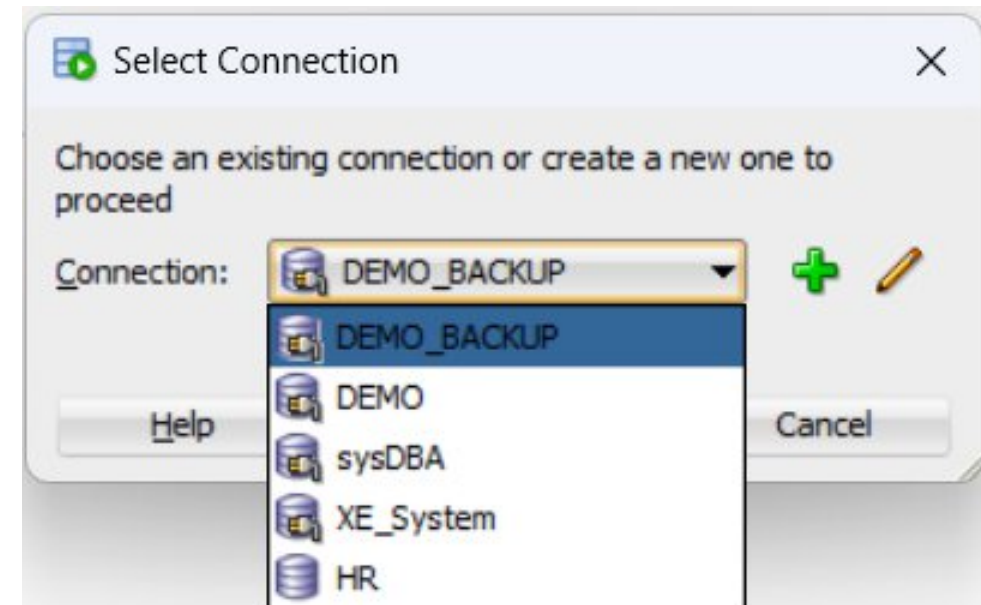
3. Run script.



```
-- File created - Saturday-October-07-2023

-- DDL for Table CUSTOMERINFO

CREATE TABLE "DEMO1"."CUSTOMERINFO"
(
  "CUST_ID" NUMBER(6,0),
  "CUST_NAME" VARCHAR2(30 BYTE),
  "MOBILE_NO" NUMBER(10,0),
  "DOB" DATE,
  "EMAIL_ID" VARCHAR2(100 BYTE)
) SEGMENT CREATION IMMEDIATE
PCTFREE 10 PCTUSED 40 INITRANS 1 MAXTRANS 255
```



DATA MANAGEMENT 2

Non-Relational Databases - MongoDB





NON-RELATIONAL DATABASES

NON-RELATIONAL DATABASES

- 🌱 Non-relational databases, often referred to as NoSQL databases (which stands for "not only SQL").
- 🌱 NoSQL is a category of database system designed to store and manage data in ways that deviate from the traditional relational database management system (RDBMS) model.
- 🌱 These databases are used to handle large volumes of structured, semi-structured, or unstructured data, and they offer more flexibility, scalability, and performance for certain use cases.

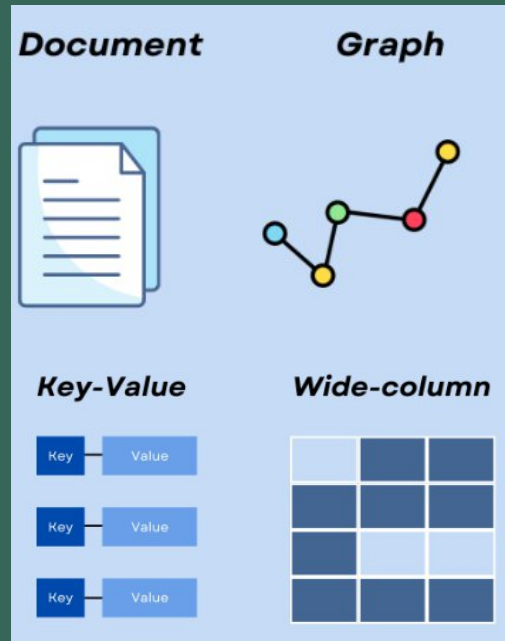
KEY FEATURES OF NON-RELATIONAL DATABASES

-  **Schema-less or Flexible Schema:** Unlike relational databases, which require a fixed schema, NoSQL databases allow you to store data without a predefined structure. This flexibility is particularly useful when dealing with dynamic or evolving data models.
-  **Scalability:** NoSQL databases are often designed to scale horizontally, which means you can add more servers or nodes to handle increased data and traffic, providing a high degree of scalability.
-  **Variety of Data Models:** NoSQL databases can handle various data models, including document-oriented, key-value, column-family, and graph data. This allows you to choose the best data model for your specific use case.
-  **High Performance:** NoSQL databases are often optimized for performance and can handle high volumes of data and complex queries. They are suitable for real-time, high-velocity data processing.

KEY FEATURES OF NON-RELATIONAL DATABASES

-  **Distributed Architecture:** Many NoSQL databases are distributed and offer built-in data replication and partitioning, ensuring high availability and fault tolerance.
-  **Support for Big Data:** NoSQL databases are well-suited for big data scenarios, where data sets are too large to be efficiently processed by traditional databases.
-  **Use Cases:** NoSQL databases are commonly used in applications such as content management systems, social networks, real-time analytics, Internet of Things (IoT), and more. Each NoSQL database type serves different use cases.

PRIMARY TYPES OF NOSQL DATABASES



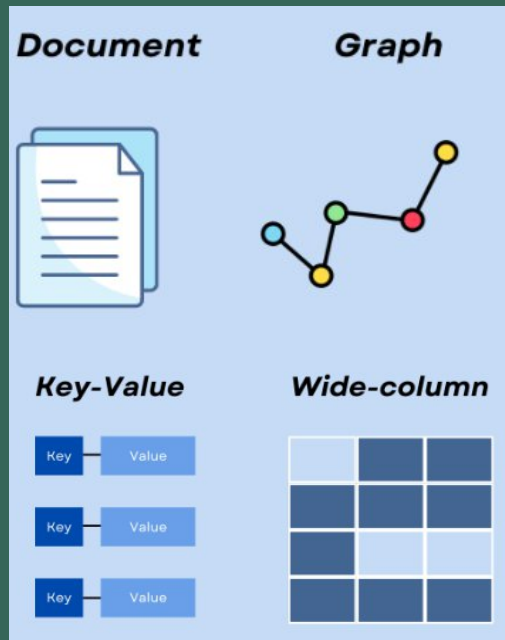
Document-Oriented Databases:

- 🌱 These databases store data in semi-structured documents, typically using formats like JSON or XML.
- 🌱 Examples include MongoDB and CouchDB.

Key-Value Stores:

- 🌱 Data is stored as key-value pairs.
- 🌱 These databases are highly performant for simple data retrieval and storage.
- 🌱 Examples include Redis and Amazon DynamoDB.

PRIMARY TYPES OF NOSQL DATABASES



Column-Family Databases:

- 🌱 Data is stored in column families, which are similar to tables in relational databases.
- 🌱 These databases are designed for high write and read throughput.
- 🌱 Examples include Apache Cassandra and HBase.

Graph Databases:

- 🌱 These databases are used for managing highly interconnected data.
- 🌱 They use graph structures to represent and query data.
- 🌱 Examples include Neo4j and Amazon Neptune.



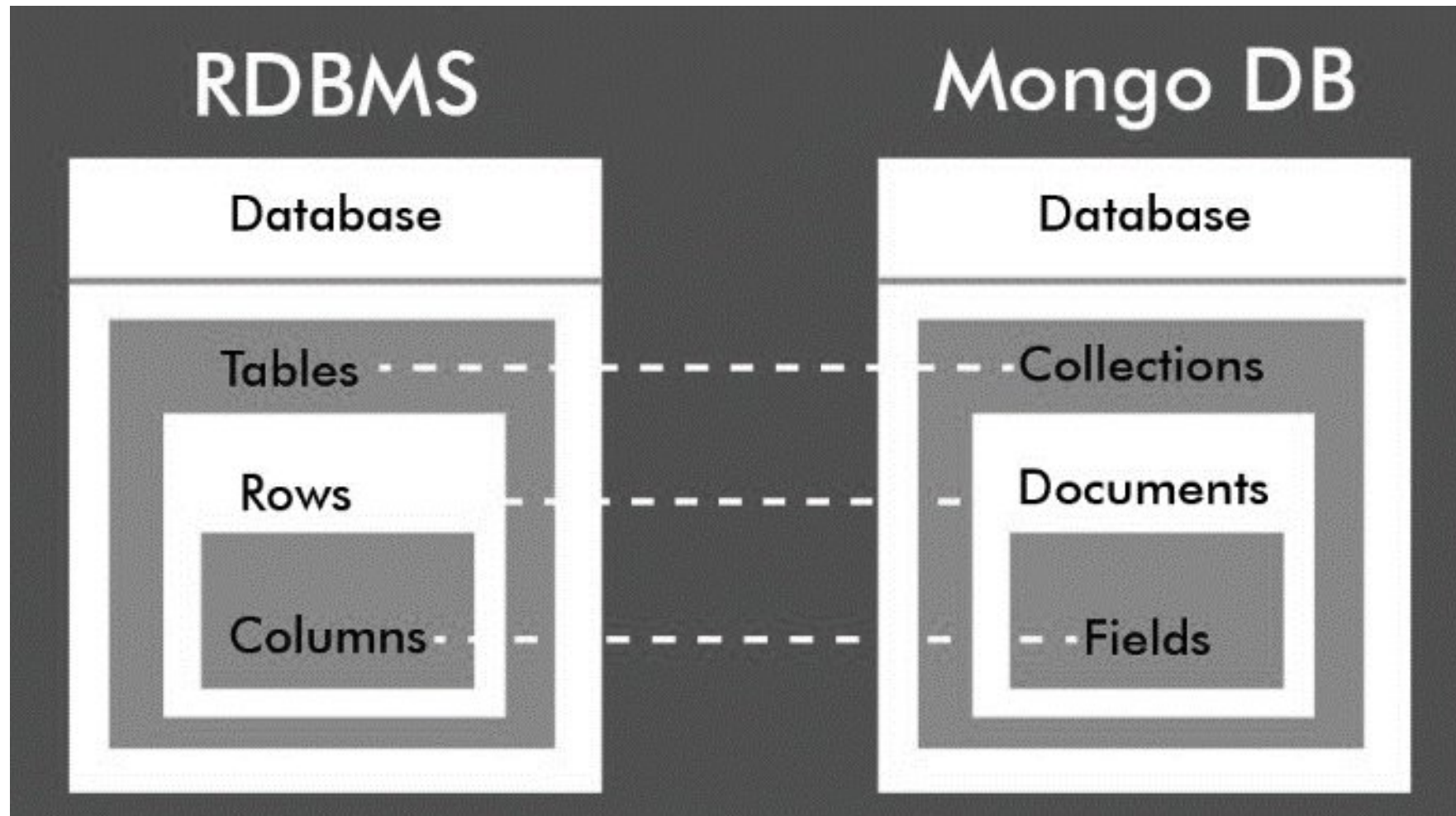
INTRODUCTION



MONGODB

- 🌱 MongoDB is a document-oriented NoSQL database used for high volume data storage.
- 🌱 Instead of using tables and rows as in the traditional relational databases, MongoDB makes use of collections and documents.
- 🌱 Documents consist of key-value pairs which are the basic unit of data in MongoDB.
- 🌱 Collections contain sets of documents and function which is the equivalent of relational database tables.
- 🌱 MongoDB is a database which came into light around the mid-2000s.

MONGODB



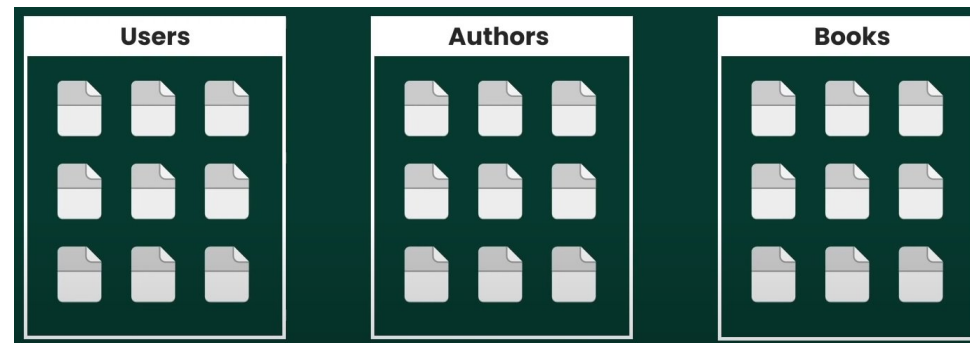
KEY COMPONENTS OF MONGODB ARCHITECTURE

Database:

- 🧑 In simple words, We can call it the physical container for data.
- 🧑 In mongoDB, A database contains the collections of documents.
- 🧑 Therefore, one can create multiple databases on the mongoDB server.

Collection:

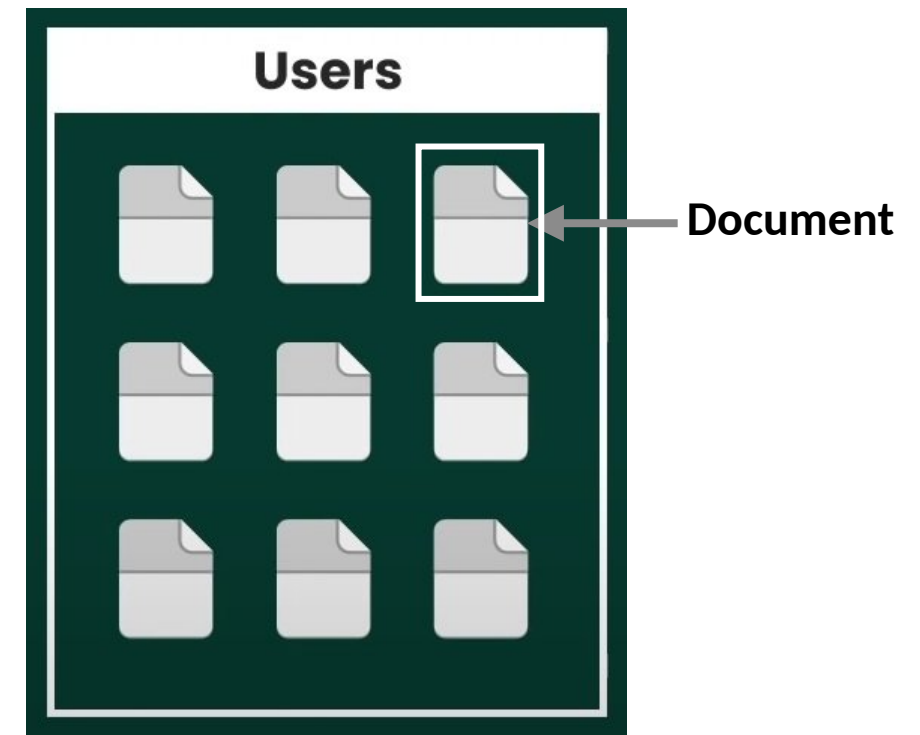
- 🧑 It's a group of mongoDB documents, similar to a table in RDBMS like Oracle, MySQL.
- 🧑 This collection doesn't enforce any structure.
- 🧑 As a result, schema-less mongoDB is so popular.



KEY COMPONENTS OF MONGODB ARCHITECTURE

Document:

- 🌐 A record in a mongoDB collection is called a document.
- 🌐 This document consists of a key-value pair.
- 🌐 This document is basically a BSON(Binary JSON) document.
- 🌐 Documents are associated with dynamic schemas.
- 🌐 Accordingly, the benefit of having dynamic schemas is that a document in a single collection does not have to possess the same structure or fields.



KEY COMPONENTS OF MONGODB ARCHITECTURE

Field –

- 🌱 A name-value pair in a document.
- 🌱 A document has zero or more fields.
- 🌱 Fields are analogous to columns in relational databases.
- 🌱 The following diagram shows an example of Fields with Key value pairs.
- 🌱 So in the example below rating and 9 is one of the key value pair's defined in the document.

```
{  
  "title": "Name of the Wind",  
  "rating": 9,  
  "pages": 400,  
  "author": {  
    "first_name": "..."  
    "last_name": "..."  
  }  
}
```

KEY COMPONENTS OF MONGODB ARCHITECTURE

_id -

- 🌱 This is a field required in every MongoDB document. The _id field represents a unique value in the MongoDB document.
- 🌱 The _id field is like the document's primary key.
- 🌱 If you create a new document without an _id field, MongoDB will automatically create the field.
- 🌱 So for example, if we see the example of the above users table, Mongo DB will add a 24 digit unique identifier to each document in the collection.

Cursor -

- 🌱 This is a pointer to the result set of a query.
- 🌱 Clients can iterate through a cursor to retrieve results.

JSON -

- 🌱 This is known as JavaScript Object Notation.
- 🌱 This is a human-readable, plain text format for expressing structured data.
- 🌱 JSON is currently supported in many programming languages.

MONGODB FEATURES

- 🌱 Each database contains collections which contains documents.
- 🌱 Each document can be different with a varying number of fields.
- 🌱 The size and content of each document can be different from each other.
- 🌱 The document structure is more in line with how developers construct their classes and objects in their respective programming languages.
- 🌱 Developers will often say that their classes are not rows and columns but have a clear structure with key-value pairs.
- 🌱 The rows (or documents as called in MongoDB) doesn't need to have a schema defined beforehand.
- 🌱 Scalability – The MongoDB environments are very scalable. Companies across the world have defined clusters with some of them running 100+ nodes with around millions of documents within the database

DIFFERENCE BETWEEN MONGODB & RDBMS

RDBMS	MongoDB	Difference
Table	Collection	In RDBMS, the table contains the columns and rows which are used to store the data whereas, in MongoDB, this same structure is known as a collection. The collection contains documents which in turn contains Fields, which in turn are key-value pairs.
Row	Document	In RDBMS, the row represents a single, implicitly structured data item in a table. In MongoDB, the data is stored in documents.
Column	Field	In RDBMS, the column denotes a set of data values. These in MongoDB are known as Fields.
Joins	Embedded documents	In RDBMS, data is sometimes spread across various tables and in order to show a complete view of all data, a join is sometimes formed across tables to get the data. In MongoDB, the data is normally stored in a single collection, but separated by using Embedded documents. So there is no concept of joins in MongoDB.



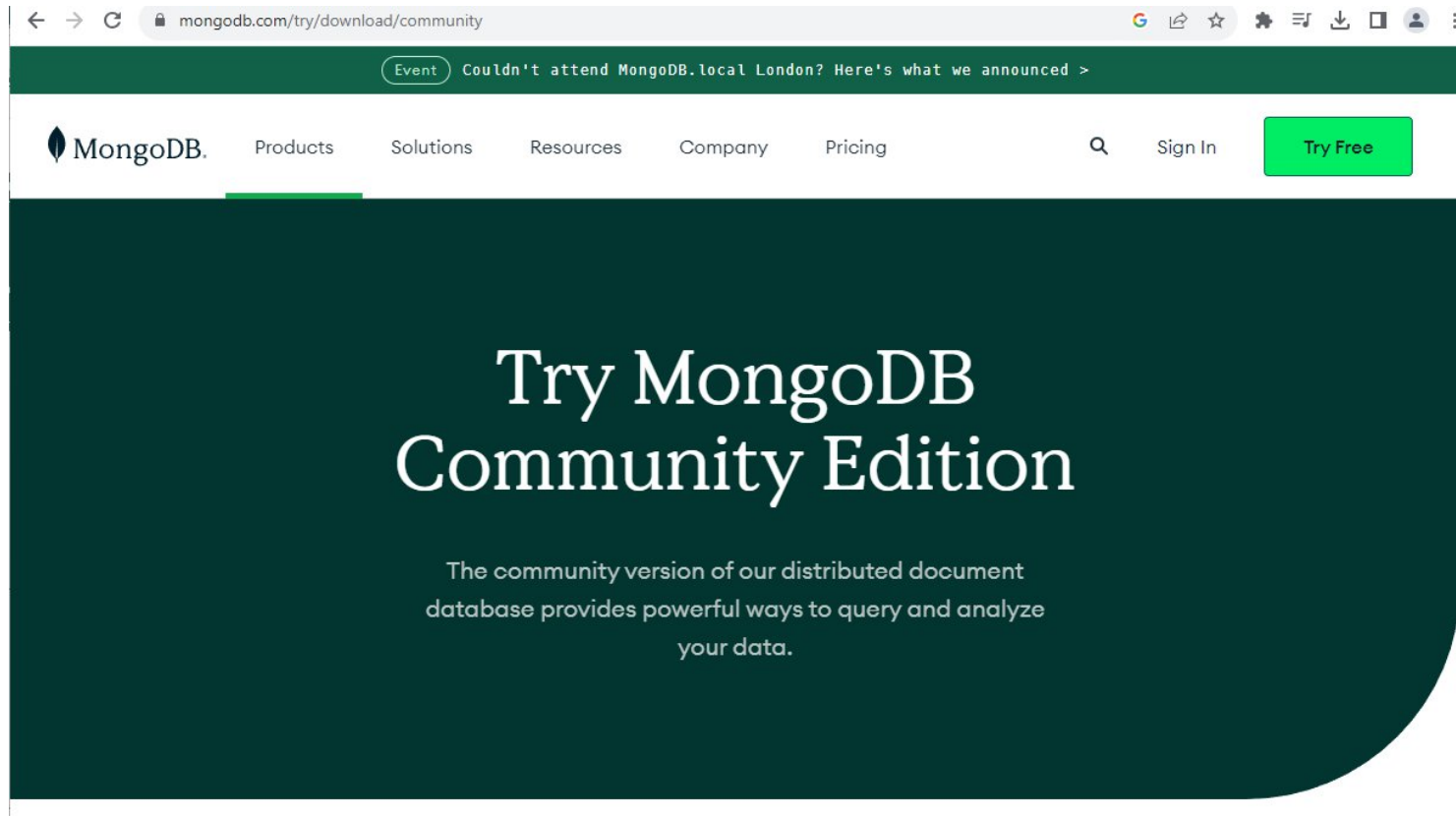
INSTALLING MONGODB



INSTALL MONGODB

Use below link to download the setup file based on your operating system.

<https://www.mongodb.com/try/download/community>



INSTALL MONGODB

Download MongoDB Community Server based on your operating system.

[Products](#)[Solutions](#)[Resources](#)[Company](#)[Pricing](#)



Try Free

MongoDB Atlas

MongoDB Enterprise Advanced

MongoDB Community Edition

MongoDB Community Server

MongoDB Community Kubernetes Operator

Tools

Atlas SQL Interface

Mobile & Edge



MONGODB COMMUNITY SERVER

MongoDB Community Server Download

Community Server

The Community version of our distributed database offers a flexible document data model along with support for:

- Ad-hoc queries
- Secondary indexing
- Real-time aggregations to provide powerful ways to access and analyze your data

Select package

Try MongoDB Atlas

The database is also offered as a fully-managed service with [MongoDB Atlas](#). Get access to advanced functionality such as:

- Auto-scaling
- Serverless instances
- Full-text search, and data distribution across regions and clouds
- Multi-region and multi-cloud support

Version

7.0.2 (current)

Platform

Windows x64

Package

msi

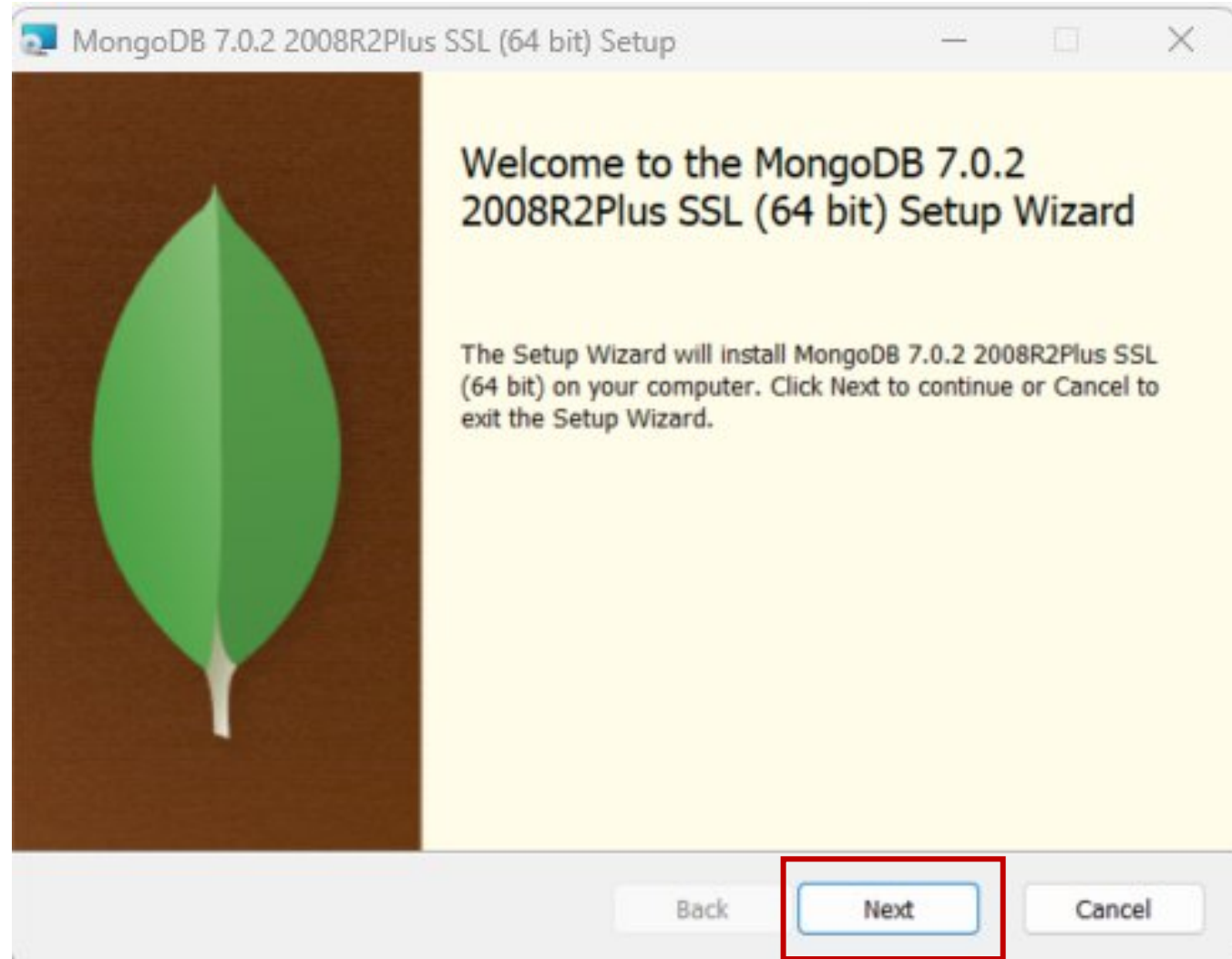
Download 

 Copy link

More Options 

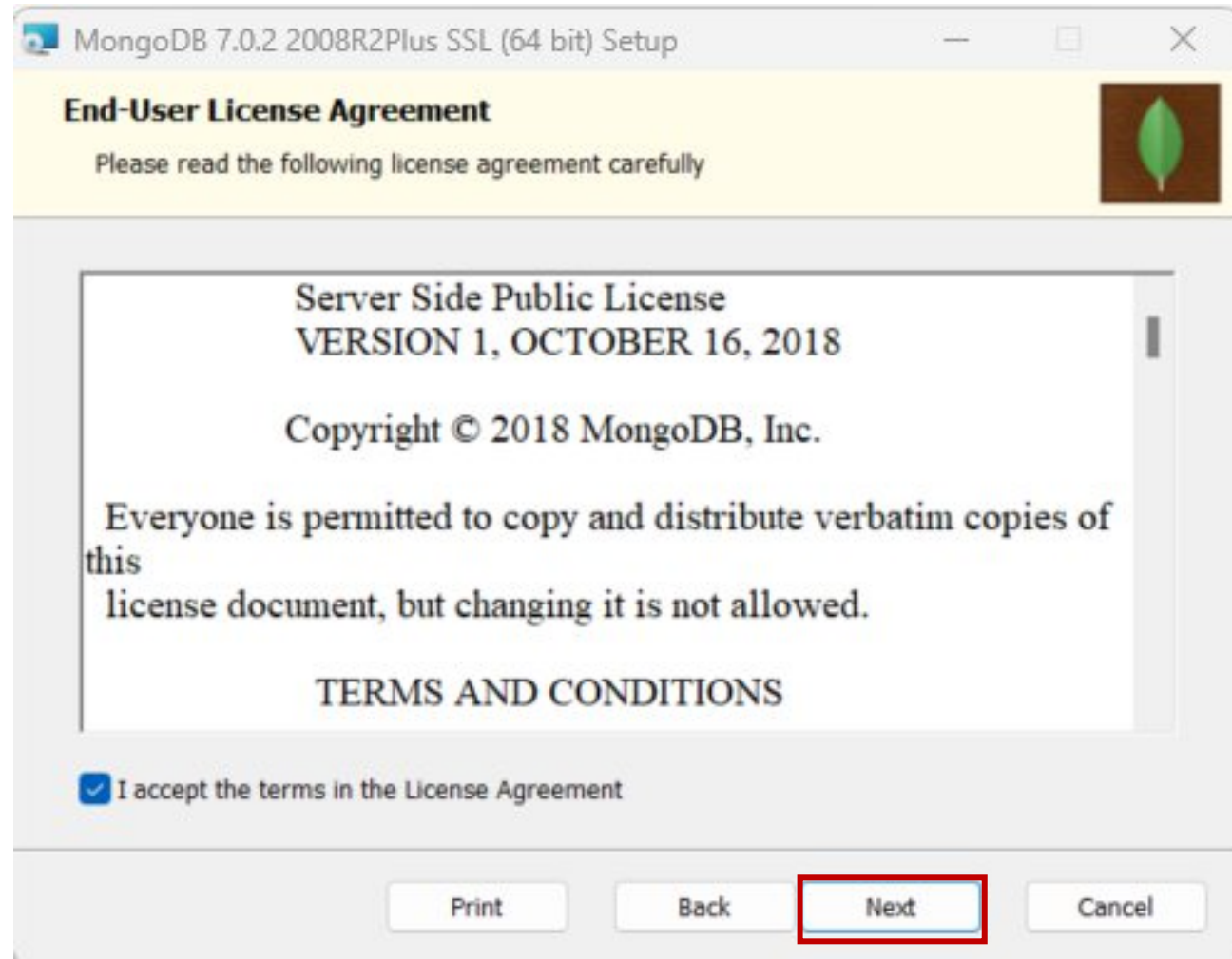
INSTALL MONGODB

 Click on Next



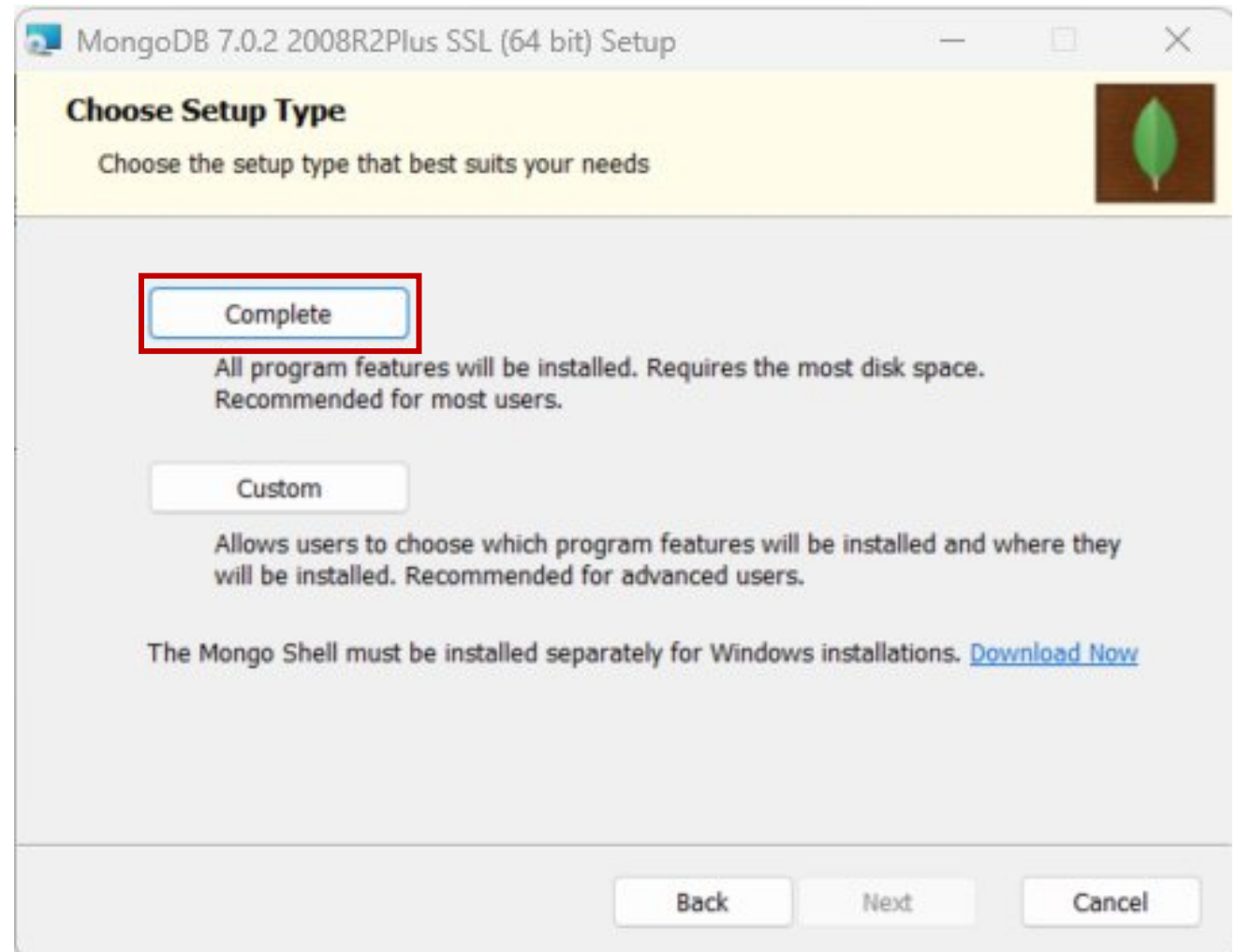
INSTALL MONGODB

 Click on Next



INSTALL MONGODB

 Click on Complete



INSTALL MONGODB

 Click on Next

MongoDB 7.0.2 2008R2Plus SSL (64 bit) Service Customization

Service Configuration

Specify optional settings to configure MongoDB as a service.

☒ Install MongoD as a Service

☒ Run service as Network Service user

☐ Run service as a local or domain user:

Account Domain:

Account Name:

Account Password:

Service Name:

Data Directory:

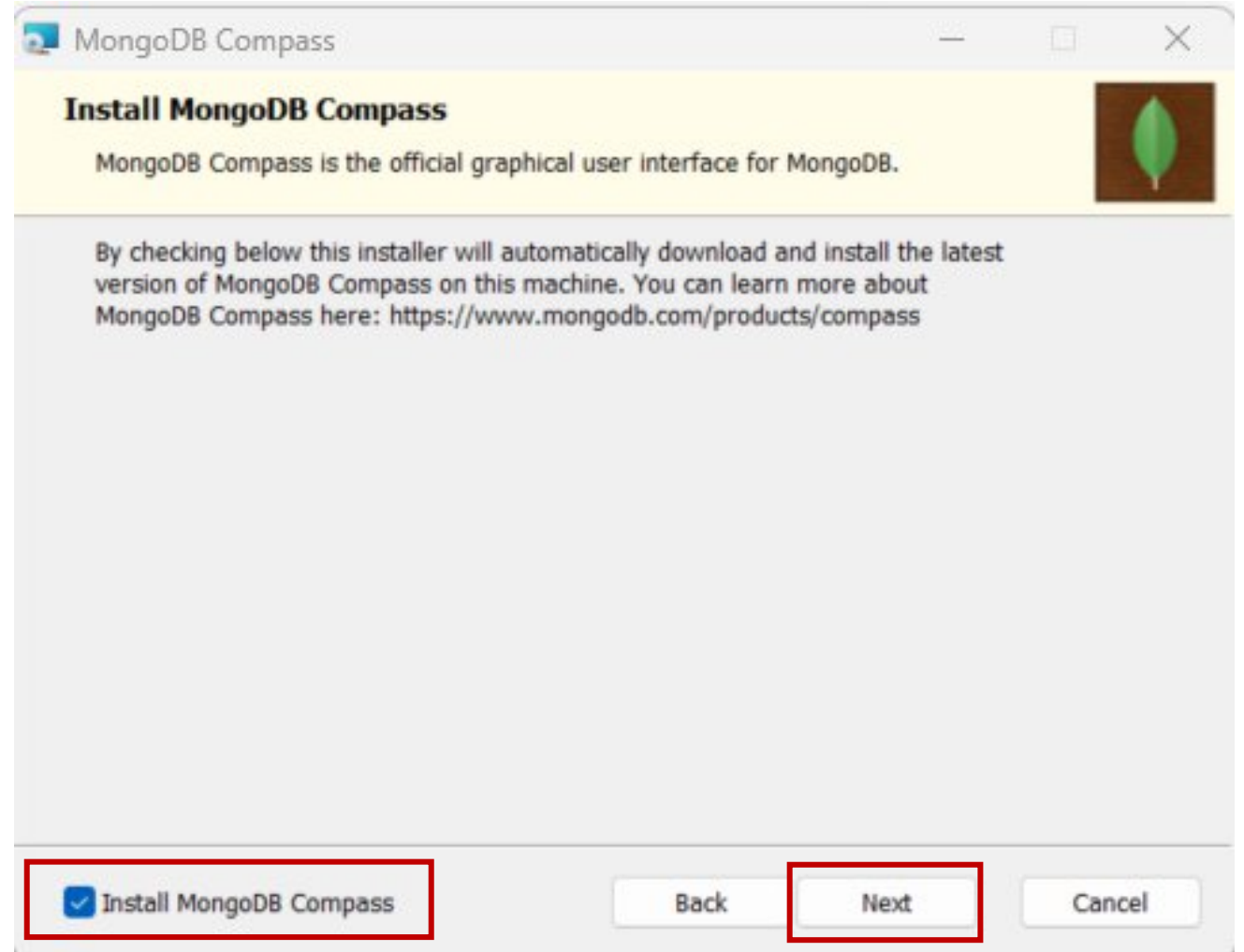
Log Directory:

< Back **Next >** Cancel

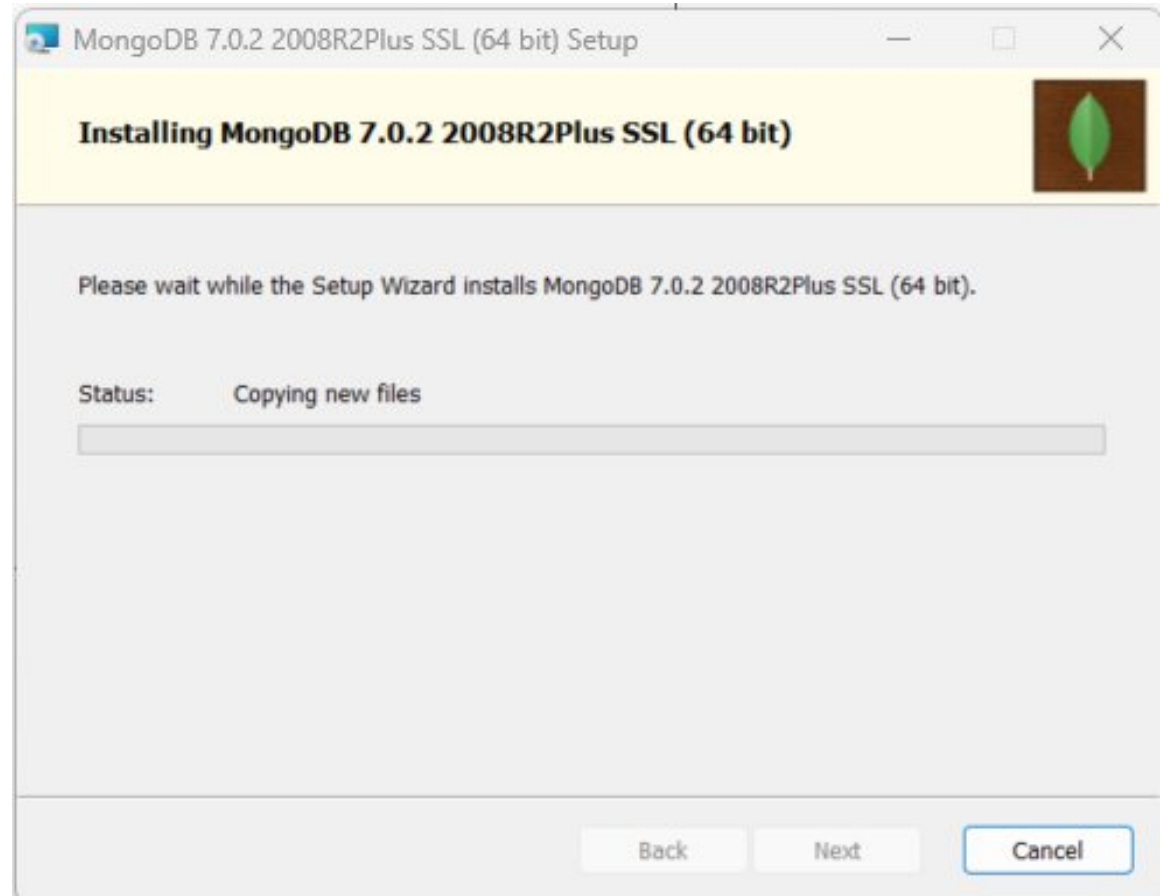
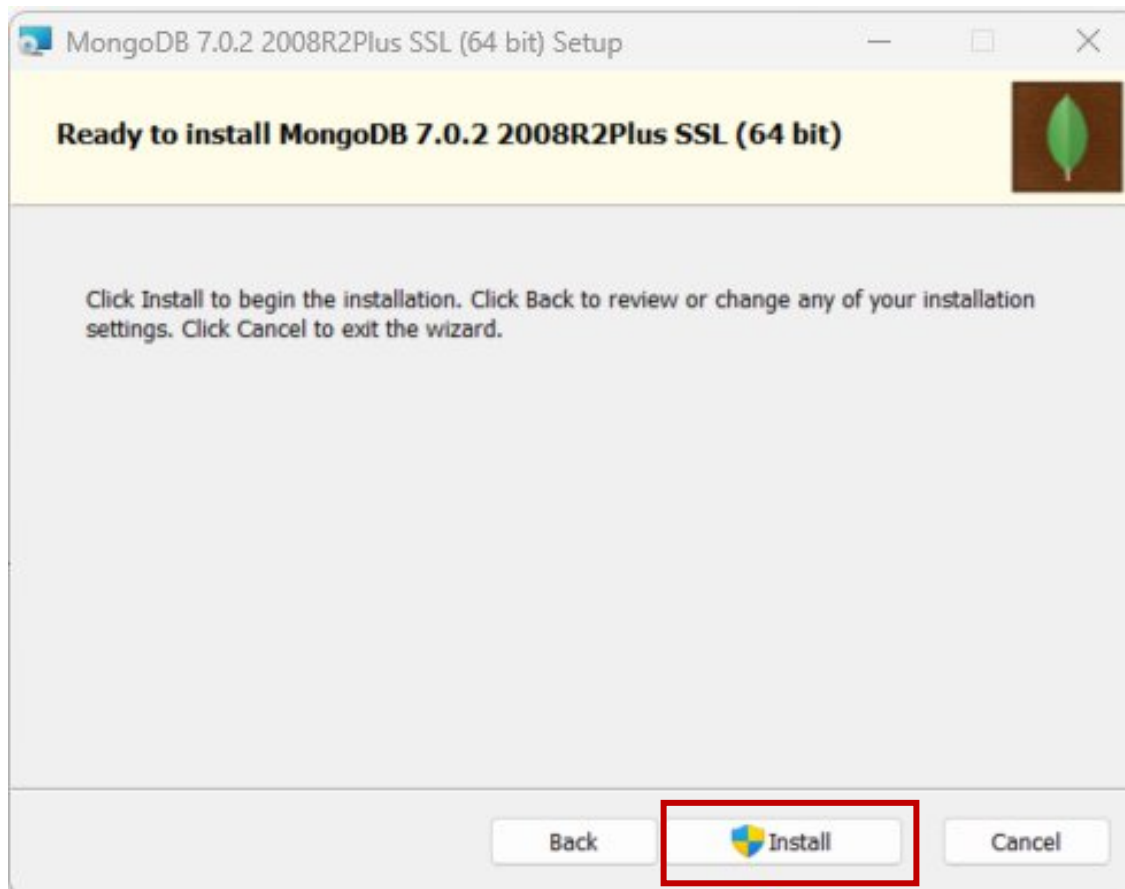
INSTALL MONGODB

🌱 Install MongoDB Compass.

🌱 Click on Next

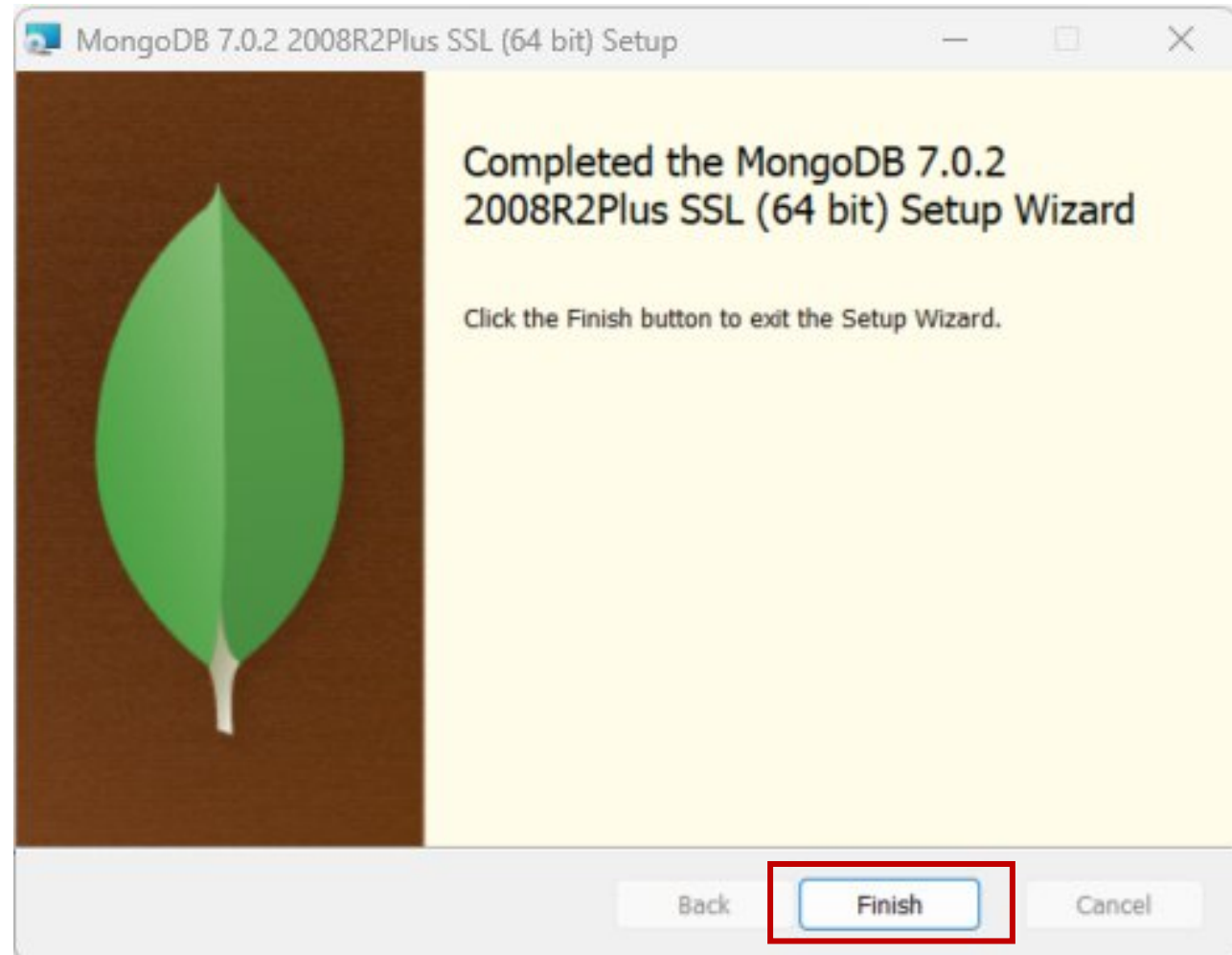


INSTALL MONGODB



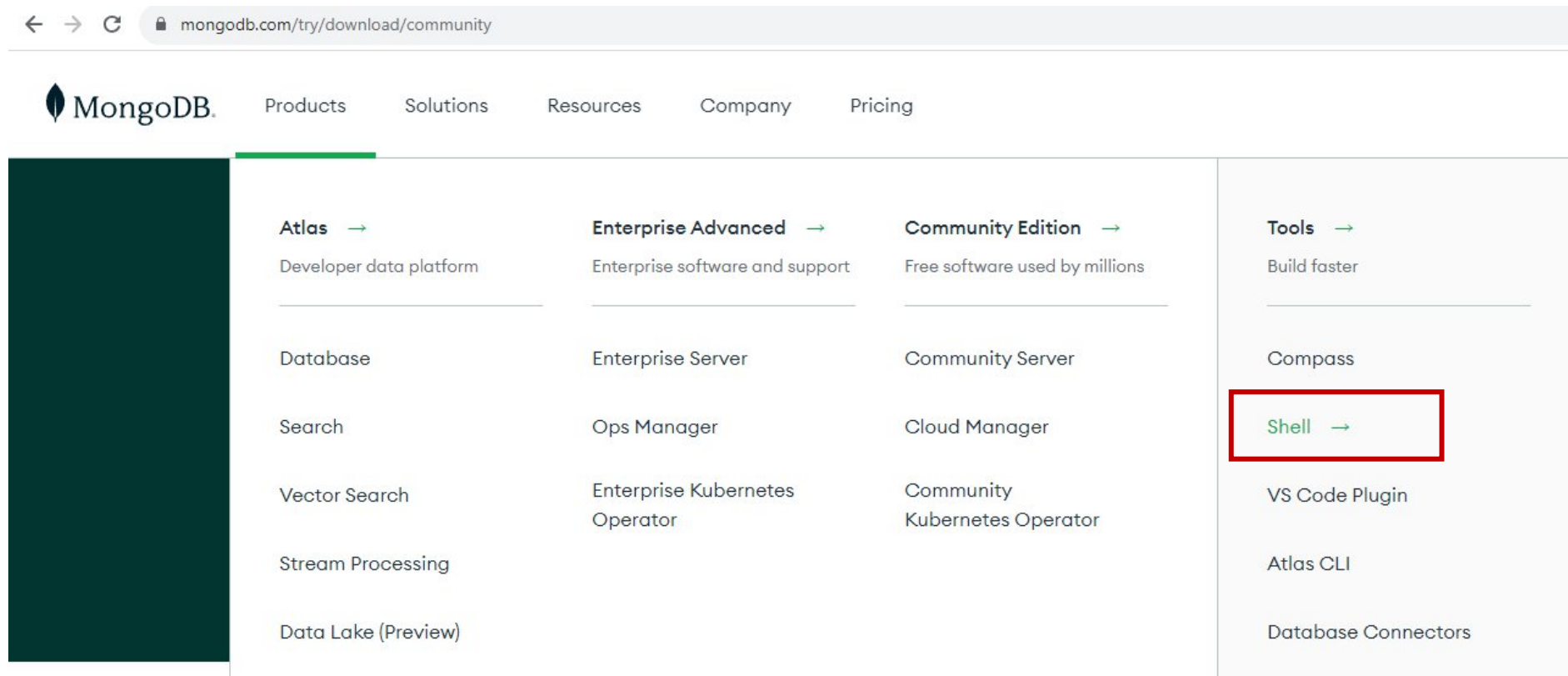
INSTALL MONGODB

 Click on Finish.



INSTALL MONGODB SHELL

🌐 Use below link to download the setup file for shell tool.
<https://www.mongodb.com/try/download/community>



INSTALL MONGODB SHELL

← → ↻ 🔒 mongodb.com/try/download/shell

Event Couldn't attend MongoDB.local London? Here's what we announced >

MongoDB Products Solutions Resources Company Pricing

🔍 Sign In Try Free

Try MongoDB Tools Free

Desktop tools, CLIs, utilities, and integrations that enhance your experience of working with MongoDB.

- MongoDB Atlas
- MongoDB Enterprise Advanced
- MongoDB Community Edition
- Tools
 - MongoDB Shell



TOOLS

MongoDB Shell Download

INSTALL MONGODB SHELL

MongoDB Shell Download

MongoDB Shell is the quickest way to connect to (and work with) MongoDB. Easily query data, configure settings, and execute other actions with this modern, extensible command-line interface – replete with syntax highlighting, intelligent autocomplete, contextual help, and error messages.

Note: MongoDB Shell is an open source (Apache 2.0), standalone product developed separately from the MongoDB Server.

Learn more

Version 2.0.1	▼
Platform Windows x64 (10+)	▼
Package msi	▼

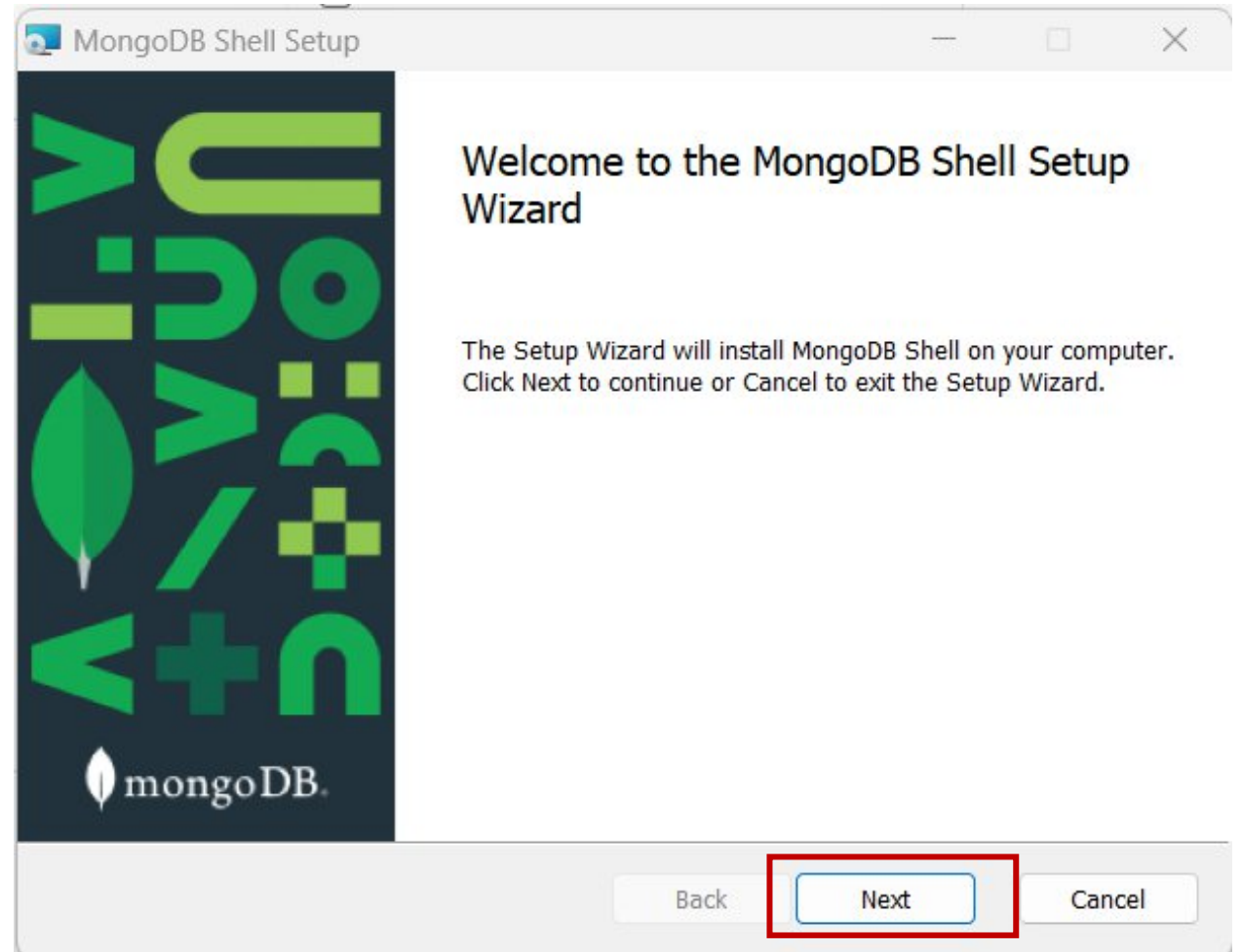
Download 

 Copy link

More Options 

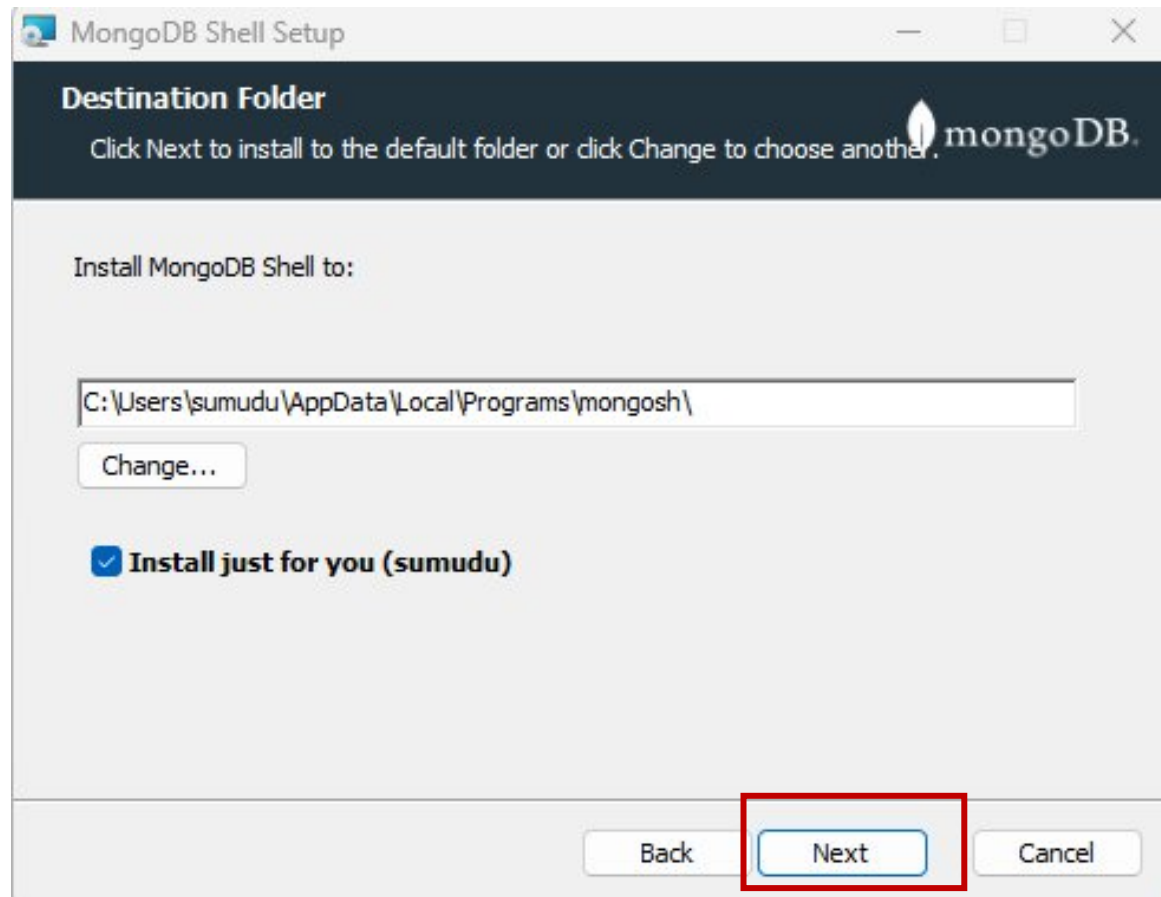
INSTALL MONGODB SHELL

 Click on Next.



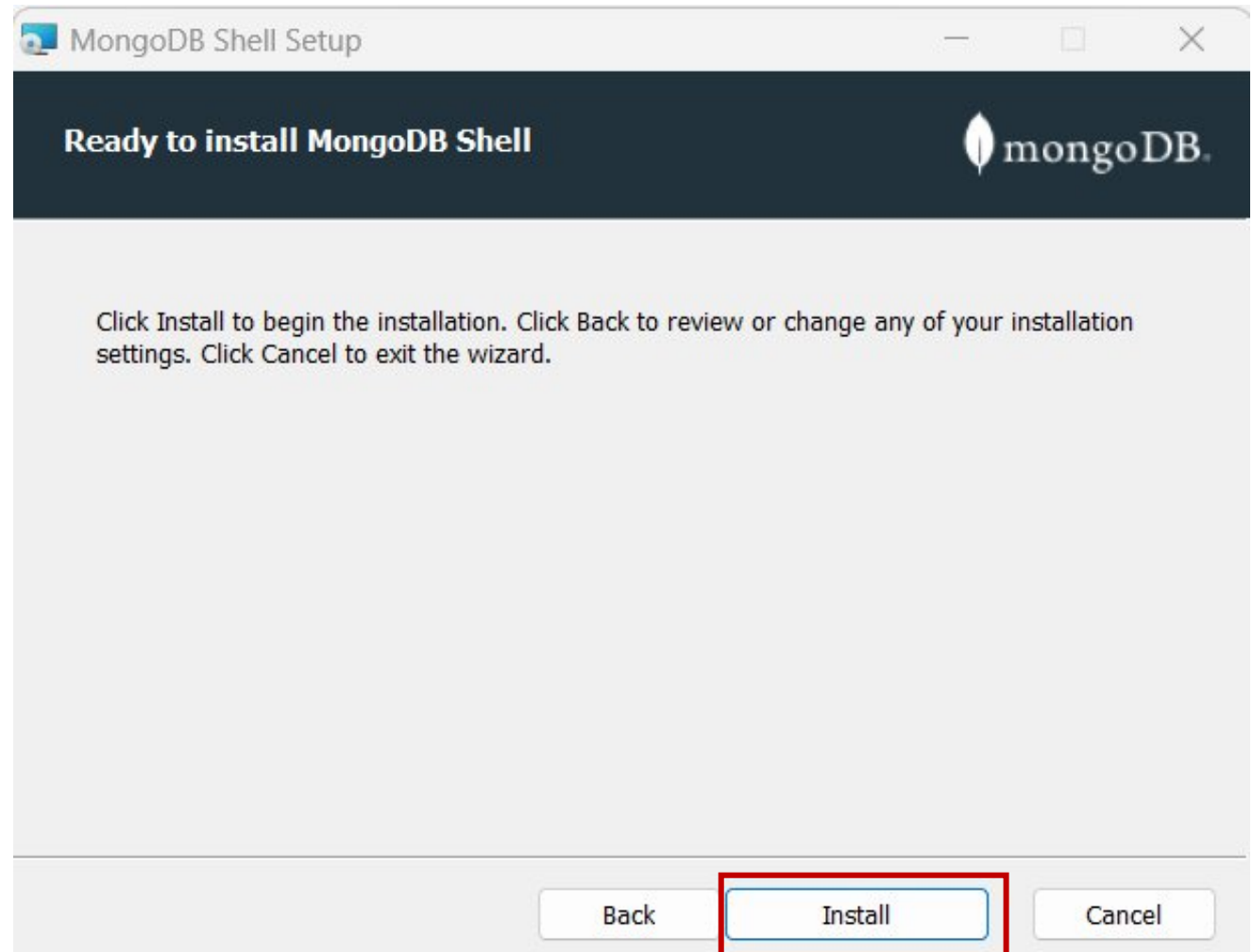
INSTALL MONGODB SHELL

 Click on Next.



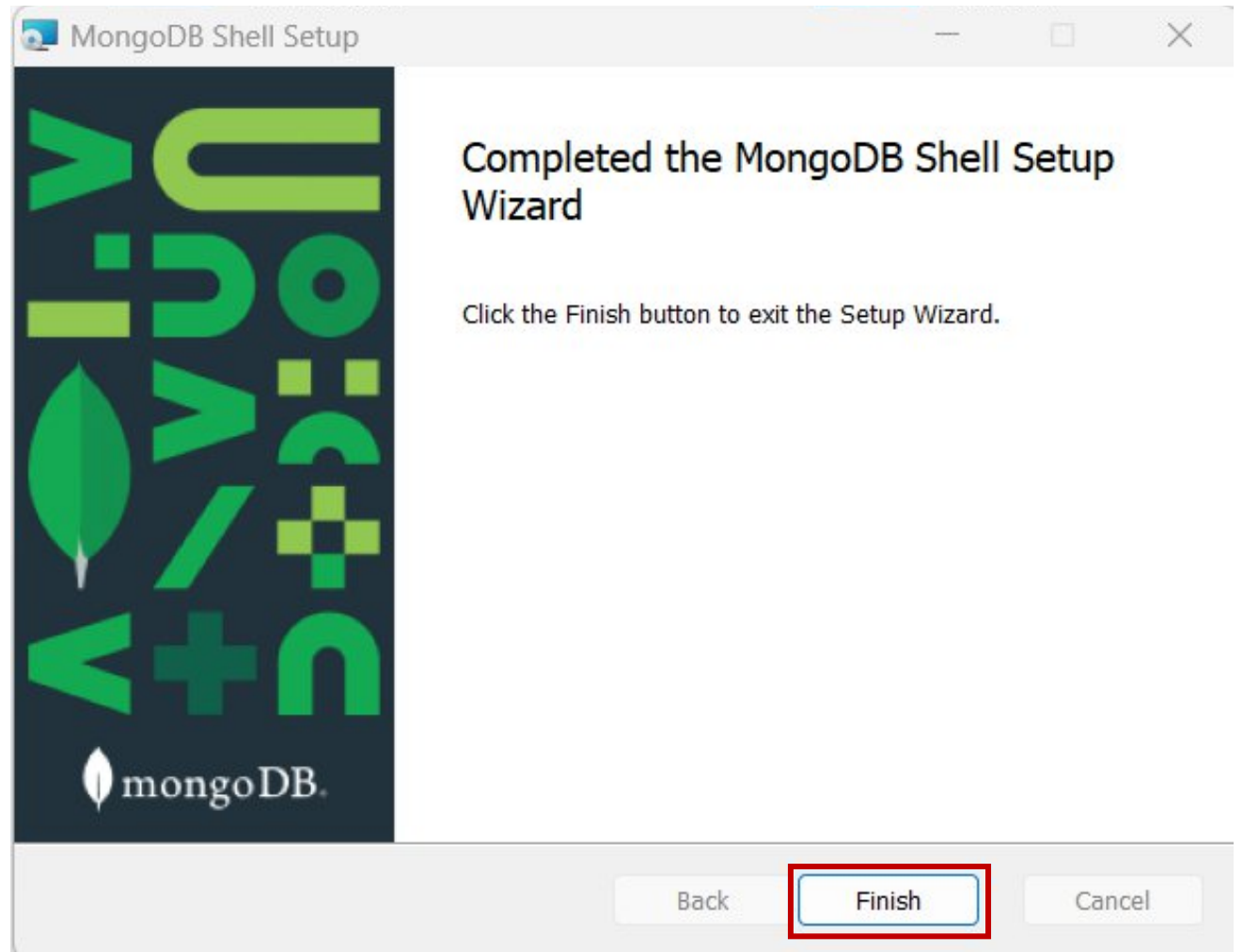
INSTALL MONGODB SHELL

 Click on Install.



INSTALL MONGODB SHELL

 Click on Finish.



INSTALL MONGODB SHELL

```
mongosh mongodb://127.0.0.1:27017/ x + v
Microsoft Windows [Version 10.0.22631.4037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\sumudu>mongosh
Current Mongosh Log ID: 66c9998687d931bd91c73bf7
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.3.1
Using MongoDB:      7.0.14
Using Mongosh:      2.3.1

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

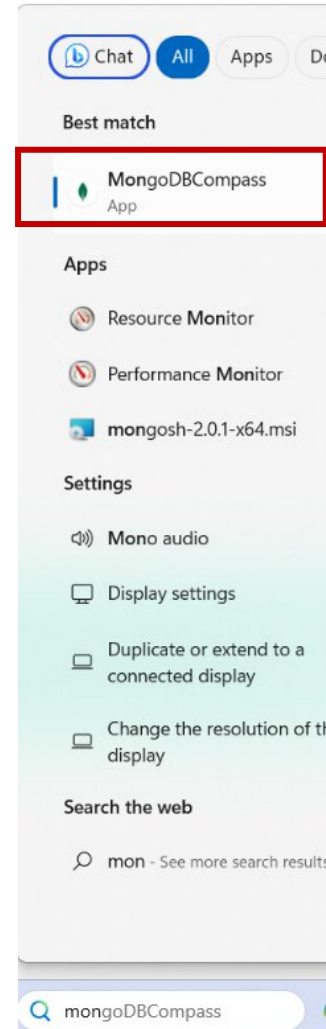
To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
The server generated these startup warnings when booting
2024-09-17T19:52:58.388+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

test>
```

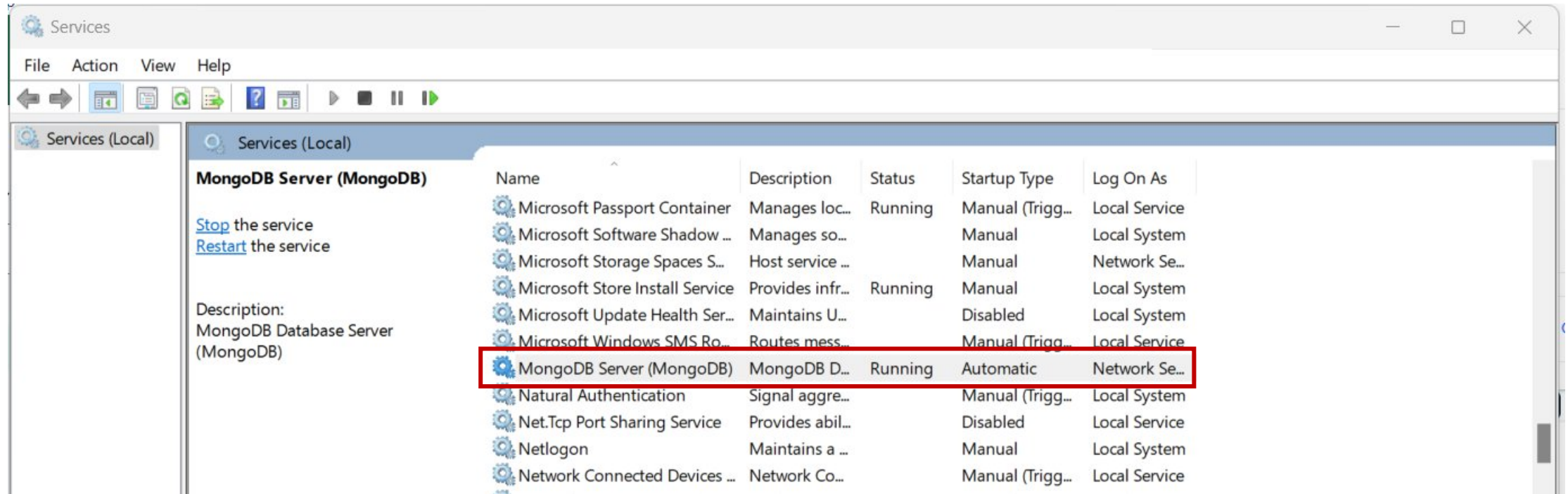
MONGODB COMPASS

 Open MongoDB Compass App



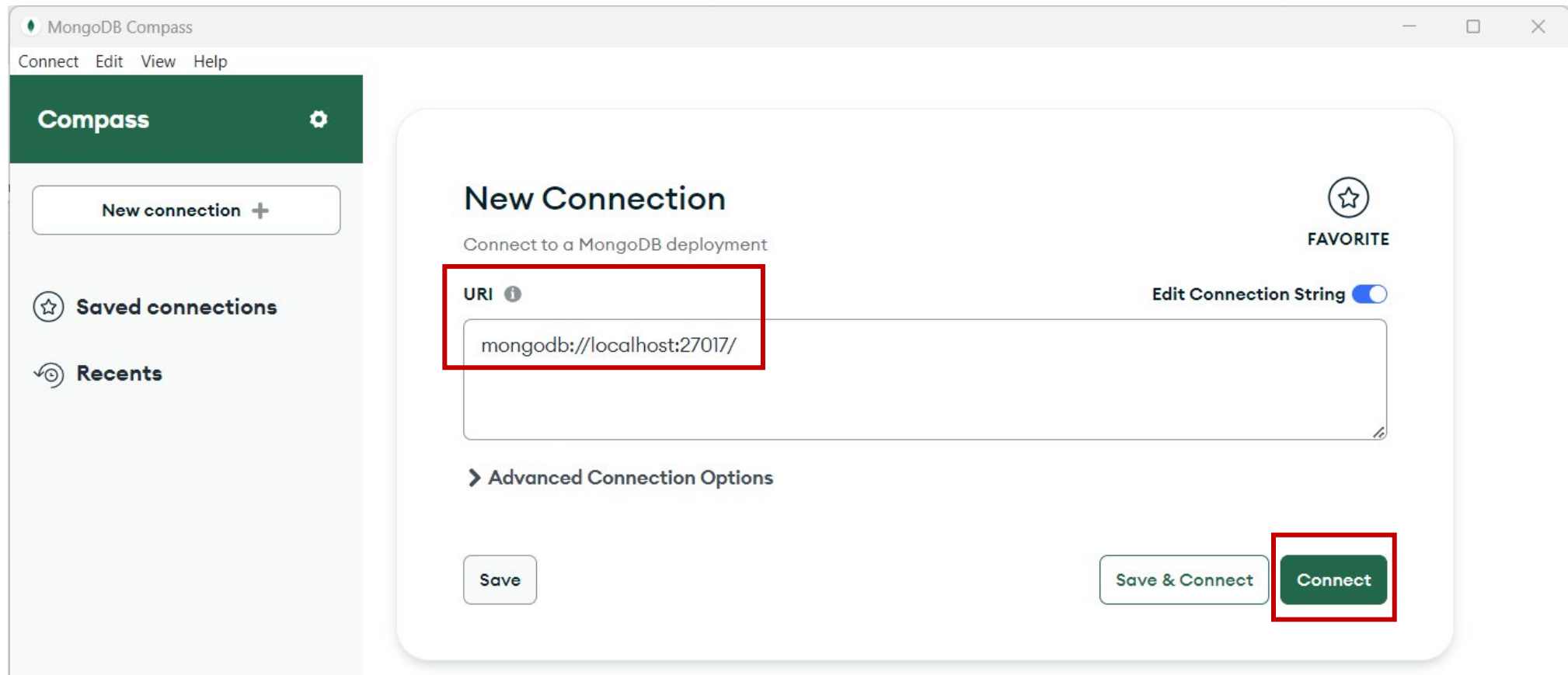
MONGODB COMPASS

Start MongoDB Services



NEW CONNECTION

🌐 Create a new connection with using URL (mongodb://localhost:27017/)



The screenshot shows the MongoDB Compass application window. The title bar reads 'MongoDB Compass'. Below it is a menu bar with 'Connect', 'Edit', 'View', and 'Help'. On the left is a sidebar with a green header 'Compass' and a settings gear icon. The sidebar contains three sections: 'New connection +' with a plus icon, 'Saved connections' with a star icon, and 'Recents' with a circular arrow icon. The main area is titled 'New Connection' with a subtitle 'Connect to a MongoDB deployment'. In the top right of this area is a 'FAVORITE' toggle with a star icon. Below the subtitle is a 'URI' label with an information icon, followed by a text input field containing 'mongodb://localhost:27017/'. To the right of the input field is an 'Edit Connection String' toggle which is turned on. Below the input field is a link '> Advanced Connection Options'. At the bottom are three buttons: 'Save', 'Save & Connect', and 'Connect'. The 'URI' label, the input field, and the 'Connect' button are highlighted with red rectangles.

MongoDB Compass

Connect Edit View Help

Compass

New connection +

Saved connections

Recents

New Connection

Connect to a MongoDB deployment

URI ⓘ

mongodb://localhost:27017/

Edit Connection String ☒

> Advanced Connection Options

Save Save & Connect Connect



CREATE AND DROP DATABASE

CREATE DATABASE



CREATE DATABASE

MongoDB Compass - localhost:27017

Connect Edit View Help

localhost:27017

My Queries Databases Performance

+ Create database Refresh

View Sort by Database Name

My Queries

Databases

Search

- admin
- config
- local

admin		
Storage size: 20.48 kB	Collections: 1	Indexes: 1
config		
Storage size: 24.58 kB	Collections: 1	Indexes: 2
local		
Storage size: 36.86 kB	Collections: 1	Indexes: 1

CREATE DATABASE AND COLLECTION

Create Database

Database Name

Bookstore

Collection Name

Books

☐ Time-Series

Time-series collections efficiently store sequences of measurements over a period of time. [Learn More](#)

➤ **Additional preferences** (e.g. Custom collation, Capped, Clustered collections)

Cancel

Create Database

MongoDB Compass - localhost:27017/Bookstore.Books

Connect Edit View Collection Help



localhost:27017

Documents

Bookstore.Books



{ } My Queries

Databases

Search

Bookstore

Books

admin

config

local

Bookstore.Books

Documents

Aggregations

Filter



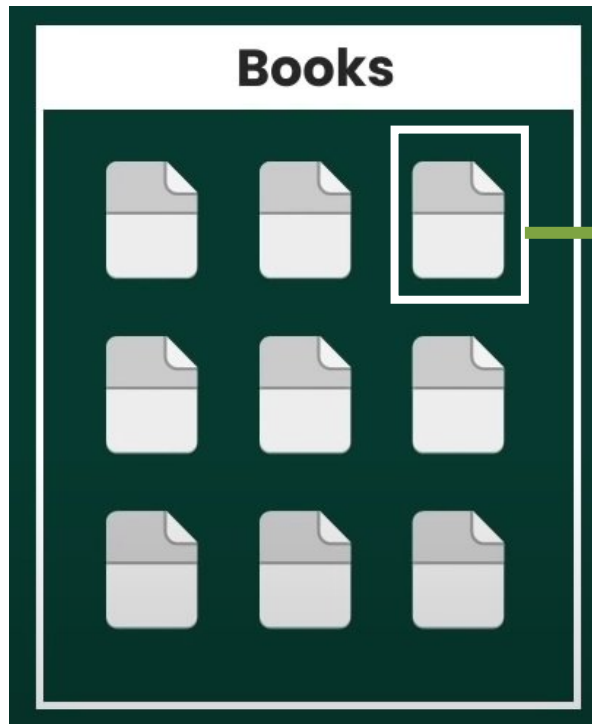
Type a query

+ ADD DATA

EXPORT

CREATE DOCUMENT

- 🌐 Records in a MongoDB database are called documents, and the field values may include numbers, strings, booleans, arrays, or even nested documents.



```
{
  "title": "The Great Gatsby",
  "author": "F. Scott Fitzgerald",
  "genre": "Fiction",
  "price": 12.99,
  "stock": 50,
  "published_year": 1925,
  "isbn": "978-0743273565",
  "publisher": "Scribner",
  "language": "English",
  "format": "Hardcover",
  "ratings": [
    { "source": "Goodreads", "rating": 4.0 },
    { "source": "Amazon", "rating": 4.5 }
  ],
  "_id": ObjectId("5f5a19231a5cf14a30198e24")
}
```

CREATE DOCUMENT - COMPASS

MongoDB Compass - localhost:27017/Bookstore.Books

Connect Edit View Collection Help

localhost:27017

Documents Bookstore.Books

My Queries

Databases

Search

Bookstore

Books

admin

config

local

Bookstore.Books

Documents Aggregations Schemas

Filter Type a query: { fi

ADD DATA EXPORT DATA

Import JSON or CSV file

Insert document

Insert Document

To collection Bookstore.Books

VIEW

```
1 /**
2  * Paste one or more documents here
3  */
4  {
5  }
6  {
7    "_id": {
8      "$oid": "65302706f11757cf97dd866c"
9    }
10 }
```

Cancel Insert

CREATE DOCUMENT

Insert Document

To collection Bookstore.Books

VIEW



```
1
2 {
3   "title": "The Great Gatsby",
4   "author": "F. Scott Fitzgerald",
5   "genre": "Fiction",
6   "price": 12.99,
7   "stock": 50,
8   "published_year": 1925,
9   "isbn": "978-0743273565",
10  "publisher": "Scribner",
11  "language": "English",
12  "format": "Hardcover"
13 }
```

Cancel

Insert

Bookstore.Books

Documents

Aggregations

Schema

Indexes

Validation

Filter



Type a query: { field: 'value' } or [Generate query](#)

+ ADD DATA

EXPORT DATA

`_id: ObjectId('6530a2c493ec1ca0fe4bb61c')`

```
title: "The Great Gatsby"
author: "F. Scott Fitzgerald"
genre: "Fiction"
price: 12.99
stock: 50
published_year: 1925
isbn: "978-0743273565"
publisher: "Scribner"
language: "English"
format: "Hardcover"
```

CREATE DOCUMENT - SHELL

MongoDB Compass - localhost:27017/Bookstore.Books

Connect Edit View Collection Help

localhost:27017 Documents Bookstore.Books

My Queries Databases Search

Bookstore Books admin config

Bookstore.Books 2 DOCUMENTS 1 INDEXES

Documents Aggregations Schema Indexes Validation

Filter {stock : 50} Generate query Explain Reset Find Options

ADD DATA EXPORT DATA 1 - 1 of 1

>_MONGOSH

> show dbs

< Bookstore	72.00 KiB
admin	40.00 KiB
config	84.00 KiB
local	72.00 KiB

test>

```
mongosh mongodb://127.0.0.1:27017/Bookstore.Books
Microsoft Windows [Version 10.0.17134.1]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kaush> mongosh
Current Mongosh Log ID: 6530dc20
Connecting to: mongodb://127.0.0.1:27017/Bookstore.Books?authSource=admin&connectTimeoutMS=30000&serverSelectionTimeoutMS=30000
Using MongoDB: 7.0.2
Using Mongosh: 2.0.1
mongosh 2.0.2 is available for download at https://www.mongodb.com/try/download/shell

For mongosh info see: https://www.mongodb.com/docs/mongosh/

-----
The server generated these startup logs when you first connected to MongoDB:
2023-10-14T09:03:30.718+05:30:
* The database was created for the database. Read and write operations are unrestricted.

test> show dbs
Bookstore 72.00 KiB
admin     40.00 KiB
config    72.00 KiB
local     72.00 KiB
test>
```

CREATE DOCUMENT - SHELL

1. Open a terminal and start the MongoDB shell by running the **mongosh** command.
2. Once connected, switch to the desired database using the **use** command, for example:

```
use Bookstore
```

3. Add a new document to the collection using the **insertOne()** method.

```
db.books.insertOne({  
  "title": "The Catcher in the Rye",  
  "author": "J.D. Salinger",  
  "genre": "Fiction",  
  "price": 10.99,  
  "stock": 30,  
});
```

EDIT DOCUMENT

 Click on edit document

Bookstore.Books

1 1
DOCUMENTS INDEXES

Documents Aggregations Schema Indexes Validation

Filter   Type a query: { field: 'value' } or [Generate query](#)  [Explain](#) [Reset](#) [Find](#)  [Options](#) 

[+ ADD DATA](#)  [EXPORT DATA](#) 

1 - 1 of 1      

 `_id: ObjectId('6530a2c493ec1ca0fe4bb61c')
title: "The Great Gatsby"
author: "F. Scott Fitzgerald"
genre: "Fiction"
price: 12.99
stock: 50
published_year: 1925
isbn: "978-0743273565"
publisher: "Scribner"
language: "English"
format: "Hardcover"`



Edit document

EDIT DOCUMENT

 Click on edit document

Bookstore.Books

1 1
DOCUMENTS INDEXES

Documents Aggregations Schema Indexes Validation

Filter   Type a query: { field: 'value' } or [Generate query](#)  [Explain](#) [Reset](#) [Find](#)  [Options](#) 

[+ ADD DATA](#)  [EXPORT DATA](#) 

1 - 1 of 1      



```
_id: ObjectId('6530a2c493ec1ca0fe4bb61c')
title: "The Great Gatsby"
author: "F. Scott Fitzgerald"
genre: "Fiction"
price: 12.99
stock: 50
published_year: 1925
isbn: "978-0743273565"
publisher: "Scribner"
language: "English"
format: "Hardcover"
```



Edit document

NESTED DOCUMENT

- In MongoDB, a nested document, also referred to as an embedded document or subdocument, is a document that is contained within another document.
- MongoDB is a NoSQL database, and it allows for flexible and hierarchical data structures, including nested documents.
- Nested documents are often used to represent complex, structured data within a single document.
- They are a powerful feature in MongoDB, and they enable you to model relationships between entities or store related data in a single document, which can be more efficient for certain use cases.

NESTED DOCUMENT

- In this example, the "author" field is a nested document within the "books" document.
- It contains information about the author, including their name, birth year, and nationality.
- Nested documents allow you to organize related data and create a more comprehensive representation of your data within a single MongoDB document.
- This is particularly useful when you have one-to-many or many-to-many relationships between entities or when you want to store additional details as part of a parent document.

```
{
  "_id": ObjectId("5f5a19231a5cf14a30198e24"),
  "title": "The Great Gatsby",
  "author": {
    "name": "F. Scott Fitzgerald",
    "birth_year": 1896,
    "nationality": "American"
  },
  "genre": "Fiction",
  "price": 12.99
}
```

FILTER OBJECT IN DOCUMENT

Bookstore.Books

2
DOCUMENT

Documents

Aggregations

Schema

Indexes

Validation

Filter



{stock : 50}



Generate query



Explain

Reset

Find



+ ADD DATA

EXPORT DATA

1 - 1 of 1



```
{
  "_id": {...},
  "title": "The Great Gatsby",
  "author": "F. Scott Fitzgerald",
  "genre": "Fiction",
  "price": 12.99,
  "stock": 50,
  "published_year": 1925,
  "isbn": "978-0743273565",
  "publisher": "Scribner",
  "language": [...],
  "format": "Hardcover",
  "ratings": [...]
}
```

FIND()

The **find()** method in MongoDB is used to retrieve documents from a collection that match a specified query or filter.

```
// Basic Find
db.collection_name.find({ /* query/filter criteria */ });

// Find All Documents
db.collection_name.find({});

// Find Documents with a Specific Field
db.collection_name.find({ field_name: field_value });

// Find Documents with Multiple Criteria
db.collection_name.find({
  field1: value1,
  field2: value2
});
```

FIND()

```
db.Books.find({}, {})
```

- The first set of empty braces {} represents the query object, indicating that you want to find all documents without any specific filtering criteria.
- The second set of empty braces {} represents the projection object, that you want to return all fields from the selected documents. Since you haven't specified any field names with values of 1 or 0, this query will return all fields for each document.

COUNT()

To count the number of documents in a MongoDB collection, you can use the **countDocuments()** method.

```
db.Books.countDocuments();
```

```
db.Books.find().count();
```

SORT()

- To retrieve documents from a MongoDB collection and sort the results, you can use the **find()** method along with the **sort()** method.
- The **sort()** method allows you to specify the field by which you want to sort the documents and the sorting order (ascending or descending).
- For example, to retrieve all books from the "Books" collection and sort them by the "stock" field in ascending order you can use the following command:

```
db.Books.find().sort({ stock: 1 });
```

- To sort in descending order, you can use:

```
db.Books.find().sort({ stock: -1 });
```

OPERATORS

- MongoDB provides a wide range of operators that allow you to perform various operations on data in your documents.
- These operators can be used for querying, updating, aggregating, and manipulating data within MongoDB collections.
- Here are some commonly used MongoDB operators:

Comparison Operators:

- **\$eq**: Matches values that are equal.
- **\$ne**: Matches values that are not equal.
- **\$gt**: Matches values that are greater than.
- **\$gte**: Matches values that are greater than or equal to.
- **\$lt**: Matches values that are less than.
- **\$lte**: Matches values that are less than or equal to.

OPERATORS

Logical Operators:

- **\$and:** Joins query clauses with a logical AND.
- **\$or:** Joins query clauses with a logical OR.
- **\$not:** Inverts the effect of a query expression.
- **\$nor:** Joins query clauses with a logical NOR.

Element Operators:

- **\$exists:** Matches documents that have the specified field.
- **\$type:** Selects documents if a field is of the specified type.

OPERATORS

Array Operators:

- **\$in:** Matches any of the values specified in an array.
- **\$nin:** Matches none of the values specified in an array.
- **\$all:** Matches an array containing all elements specified in an array.

Aggregation Operators:

- **\$group:** Groups documents based on a specified expression.
- **\$project:** Reshapes each document by specifying fields to include or exclude.
- **\$match:** Filters the documents to pass only those that match the specified condition.

UPDATE DOCUMENT

In MongoDB, you can update documents using the **updateOne()** or **updateMany()** methods.

Suppose you want to update the price of a book with the title "The Great Gatsby" in the "Books" collection:

```
db.Books.updateOne(  
  { title: "The Great Gatsby" }, // The filter to match the document  
  {  
    $set: { price: 14.99 } // The update operation to set the new price  
  }  
);
```

UPDATE DOCUMENT

- 🌱 **updateMany()** method in MongoDB to update multiple documents in a collection.
- 🌱 Let's assume you want to increase the price of all books published before the year 2000 by \$5 in the "Books" collection.

```
db.Books.updateMany(  
  { published_year: { $lt: 2000 } }, // Filter criteria to match the document  
  {  
    $inc: { price: 5 } // The update operation to increment the price by $5  
  }  
);
```

DELETE DOCUMENT

 Click on remove document

Bookstore.Books 3 DOCUMENTS 1 INDEXES

Documents Aggregations Schema Indexes Validation

Filter   Type a query: { field: 'value' } or [Generate query](#) 

[Explain](#) [Reset](#) [Find](#)  [Options](#) 

[+ ADD DATA](#)  [EXPORT DATA](#) 

1 - 4 of 4      

 `_id: ObjectId('6530a2c493ec1ca0fe4bb61c')`
`title: "The Great Gatsby"`
`author: "F. Scott Fitzgerald"`

    [Remove document](#)

DELETE DOCUMENT

In MongoDB, you can delete documents from a collection using the **deleteOne()** or **deleteMany()** methods.

Using deleteOne() to Delete a Single Document:

The deleteOne() method deletes the first document that matches the specified filter criteria.

```
db.collectionName.deleteOne({ filter_criteria });
```

For example, to delete a book with the title "The Great Gatsby" from the "Books" collection:

```
db.Books.deleteOne({ title: "The Great Gatsby" });
```

DELETE DOCUMENT

Using deleteMany() to Delete Multiple Documents:

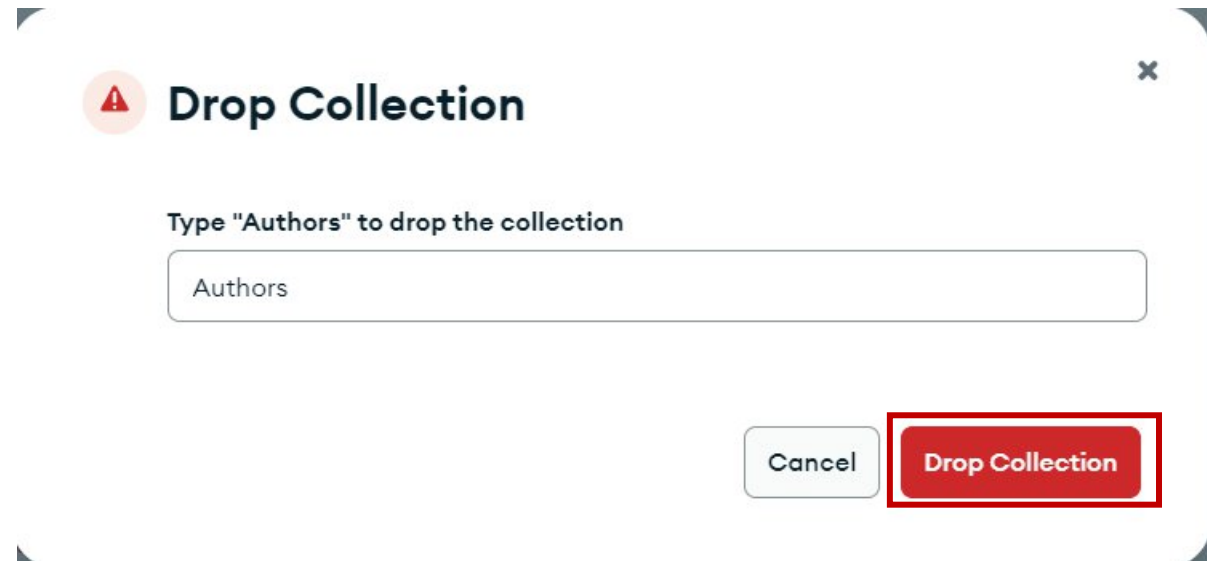
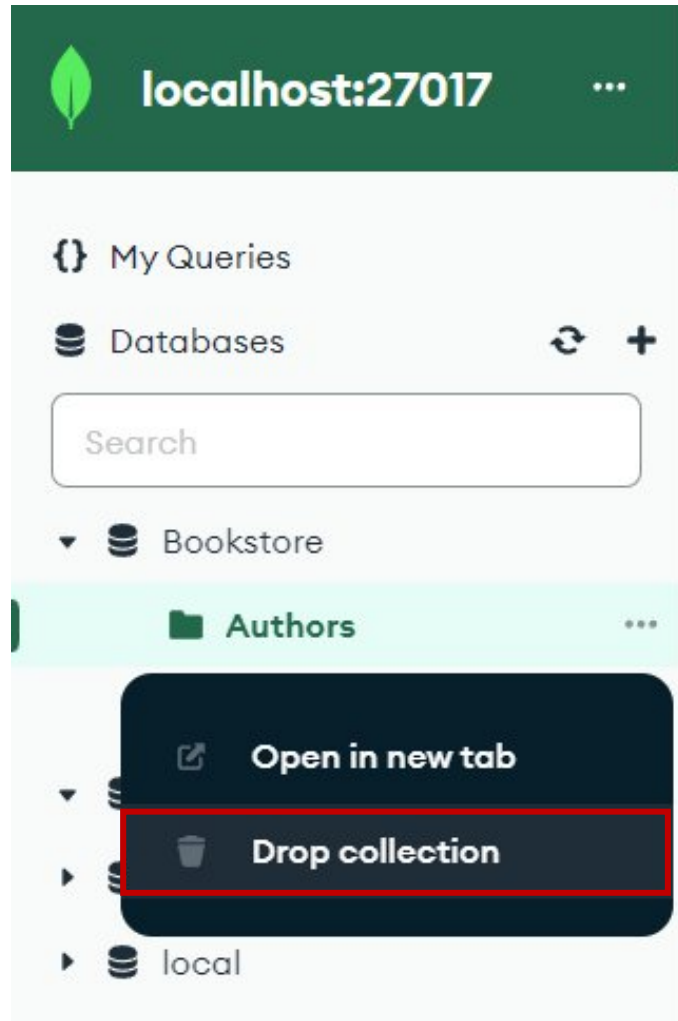
The deleteMany() method deletes all documents that match the specified filter criteria.

```
db.collectionName.deleteMany({ filter_criteria });
```

For example, to delete all books with the genre "Science Fiction" from the "Books" collection:

```
db.Books.deleteMany({ genre: "Science Fiction" });
```

DELETE COLLECTION



EXPORT COLLECTION

The screenshot illustrates the steps to export a collection in MongoDB Compass. The interface shows the 'Bookstore.Authors' collection selected in the left sidebar. The 'Documents' tab is active, displaying a query filter and a query input field. The 'EXPORT DATA' button is highlighted. A modal dialog titled 'Export' is open, showing the 'Export File Type' as 'JSON' and the 'Export...' button. A tooltip is also visible, indicating the options for exporting query results or the full collection.

localhost:27017 ...

Documents
Bookstore.Authors

Bookstore.Authors

Documents Aggregations Schema Indexes

Filter Type a query: { field: 'value' }

ADD DATA **EXPORT DATA**

Export
Collection Bookstore.Authors

Export File Type

JSON CSV

> Advanced JSON Format

Cancel **Export...**

Export query results
Export the full collection

IMPORT COLLECTION

Create Collection

Collection Name

Authors

☐ **Time-Series**

Time-series collections efficiently store sequences of measurements over a period of time. [Learn More](#)

➤ **Additional preferences** (e.g. Custom collation, Capped, Clustered collections)

Cancel

Create Collection



This collection has no data

It only takes a few seconds to import data from a JSON or CSV file.

Import Data

Import

To collection Bookstore.Authors

Import file: Bookstore.Authors.json

Options

☐ Stop on errors

Cancel

Import



MOBILE DATABASES



MOBILE DATABASES

- 📶 A Mobile database is a database that can be connected to a mobile computing device over a mobile network (or wireless network).
- 📶 Here the client and the server have wireless connections.
- 📶 In today's world, mobile computing is growing very rapidly, and it is huge potential in the field of the database.
- 📶 It will be applicable on different-different devices like android based mobile databases, iOS based mobile databases, etc. Common examples of databases are Couch base Lite, Object Box, etc.

FEATURES OF MOBILE DATABASE

- 📍 A cache is maintained to hold frequent and transactions so that they are not lost due to connection failure.
- 📍 Mobile databases are physically separate from the central database server.
- 📍 Mobile databases resided on mobile devices.
- 📍 Mobile databases are capable of communicating with a central database server or other mobile clients from remote sites.
- 📍 With the help of a mobile database, mobile users must be able to work without a wireless connection due to poor or even non-existent connections (disconnected).
- 📍 A mobile database is used to analyze and manipulate data on mobile devices.

MOBILE DATABASE

🏠 Mobile Database typically involves three parties

Fixed Hosts:

- 🏠 Fixed hosts refer to the database servers or data management systems located in a central or fixed location. These servers handle data storage, processing, and management.
- 🏠 They are responsible for processing database transactions, executing queries, and ensuring data integrity.
- 🏠 Fixed hosts provide a centralized repository for data that is accessed and manipulated by mobile units.
- 🏠 They may also handle tasks such as data backup, security, and access control.

MOBILE DATABASE

Mobile Units:

- 📱 Mobile units are mobile and transportable computing devices, such as smartphones, tablets, laptops, or other mobile devices.
- 📱 Mobile units are equipped with applications that interact with the central database. These applications can create, read, update, and delete data on the mobile device.
- 📱 Mobile units connect to the central database using wireless or cellular networks. They communicate with the database servers at fixed hosts via base stations.
- 📱 These units are used by individuals or entities to access and manipulate data, often while on the move or in different geographical areas.

MOBILE DATABASE

Base Stations:

- 📶 Base stations, also known as cell towers or base transceiver stations, are stationary communication infrastructure components.
- 📶 They are spread across a geographical area and serve as relay points for mobile units to connect to the central database at the fixed hosts.
- 📶 Base stations facilitate wireless communication between mobile units and the central database servers.
- 📶 They play a critical role in establishing and maintaining the connection between the mobile units and the fixed hosts.

ADVANTAGES OF MOBILE DATABASES

- 📶 The data in a database can be accessed from anywhere using a mobile database. It provides wireless database access.
- 📶 The database systems are synchronized using mobile databases and multiple users can access the data with seamless delivery process.
- 📶 Mobile databases require very little support and maintenance.
- 📶 The mobile database can be synchronized with multiple devices such as mobiles, computer devices, laptops etc.

LIMITATIONS OF MOBILE DATABASE

Wireless Bandwidth Restrictions:

- 🚧 Mobile databases often rely on wireless network connections, and these connections can have limited bandwidth. This can result in slower data transfer speeds, especially in areas with poor network coverage.
- 🚧 Large data transfers or complex database operations may be slower or less efficient on mobile networks compared to wired connections.

Security Concerns:

- 🚧 Mobile databases can be vulnerable to security threats. The mobility of devices makes them more susceptible to physical loss or theft.
- 🚧 Ensuring the security of data stored on mobile devices and transmitted wirelessly is a challenge. Encryption and access control measures are essential to protect sensitive data.

LIMITATIONS OF MOBILE DATABASE

Battery Power Limitations:

- 🕒 Mobile devices have limited battery life, and running database operations can consume significant power.
- 🕒 Continuous database synchronization or data-intensive operations can drain the battery quickly, affecting the usability of the device.

Wireless Communication Speed:

- 🕒 Wireless communication speed can be slower and less reliable compared to wired connections. Network congestion, signal interference, and variations in network quality can impact the speed of data transfer.
- 🕒 Slow communication can result in delays in accessing or synchronizing data.



SQLITE INSTALLATION (CLI)

STEP 1

Download and configure SQLite

URL (<https://www.sqlite.org/download.html>)

Precompiled Binaries for Windows

[sqlite-dll-win-x86-3450300.zip](#) 32-bit DLL (x86) for SQLite version 3.45.3.
(1.01 MiB) (SHA3-256: b5d01cccf1cea00b4250bc26d3941ba29b16eaaaa7ce9fe9f7776b8e4491ff2f)

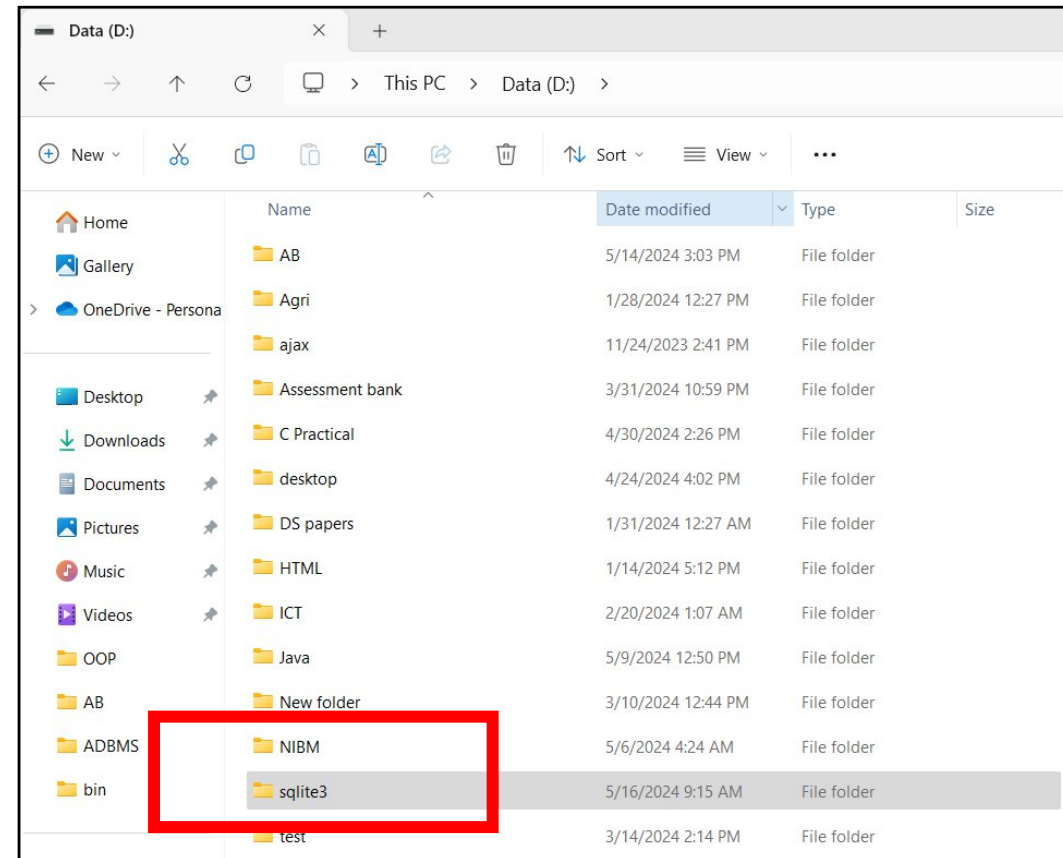
[sqlite-dll-win-x64-3450300.zip](#) 64-bit DLL (x64) for SQLite version 3.45.3.
(1.26 MiB) (SHA3-256: 9631cadef7d48498a468bebdd0dba825af3121f0980109601168f695f4273688)

[sqlite-tools-win-x64-3450300.zip](#) A bundle of command-line tools for managing SQLite database files, including the [command-line shell](#) program, the [sqldiff.exe](#) program, and the [sqlite3_analyzer.exe](#) program. 64-bit.
(4.77 MiB) (SHA3-256: 429cca813471884ff3c9b51697ecc1e6750fb947ab5cd91efb098c6fbc1f4053)

STEP 2

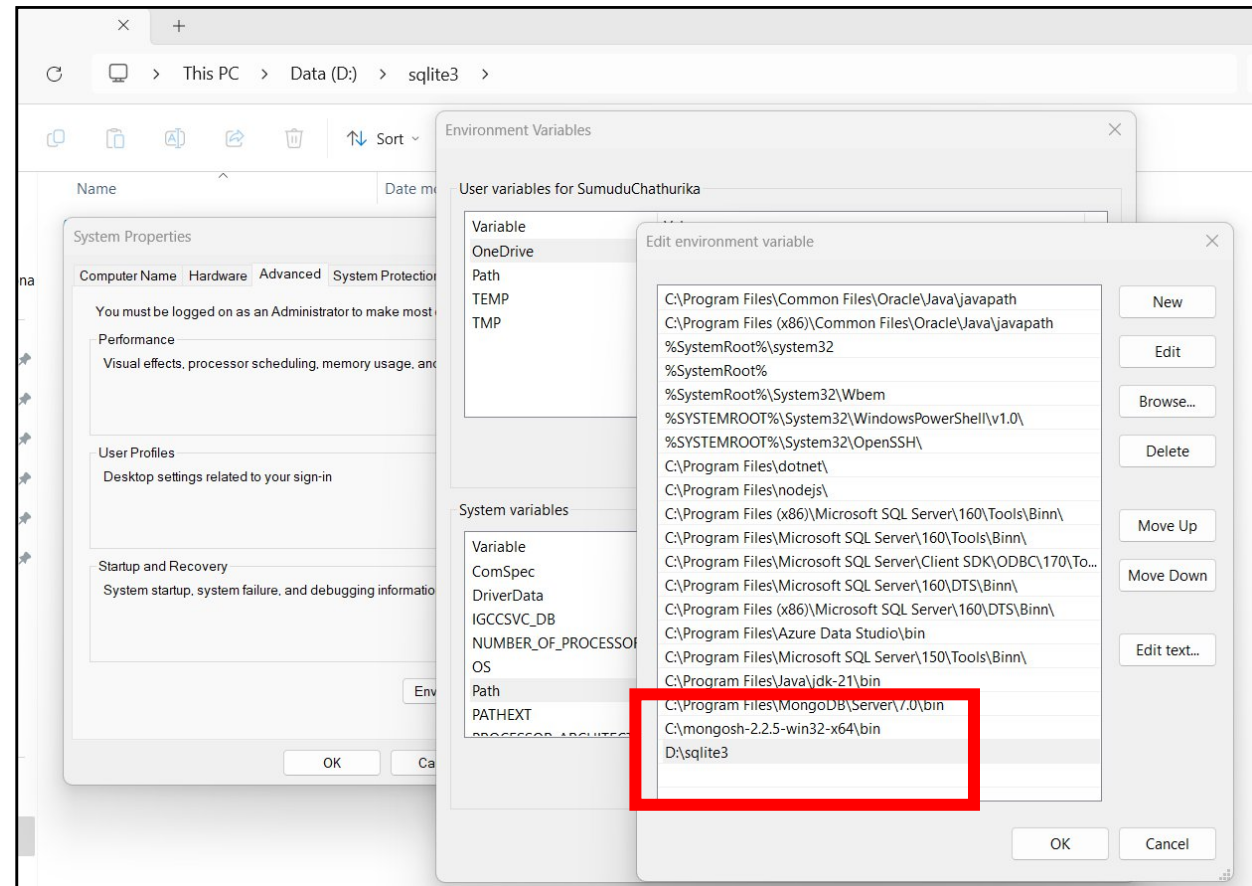
🗂️ Extract the folder into a drive and rename it (sqlite3)

 sqlite-tools-win-x64-3450300



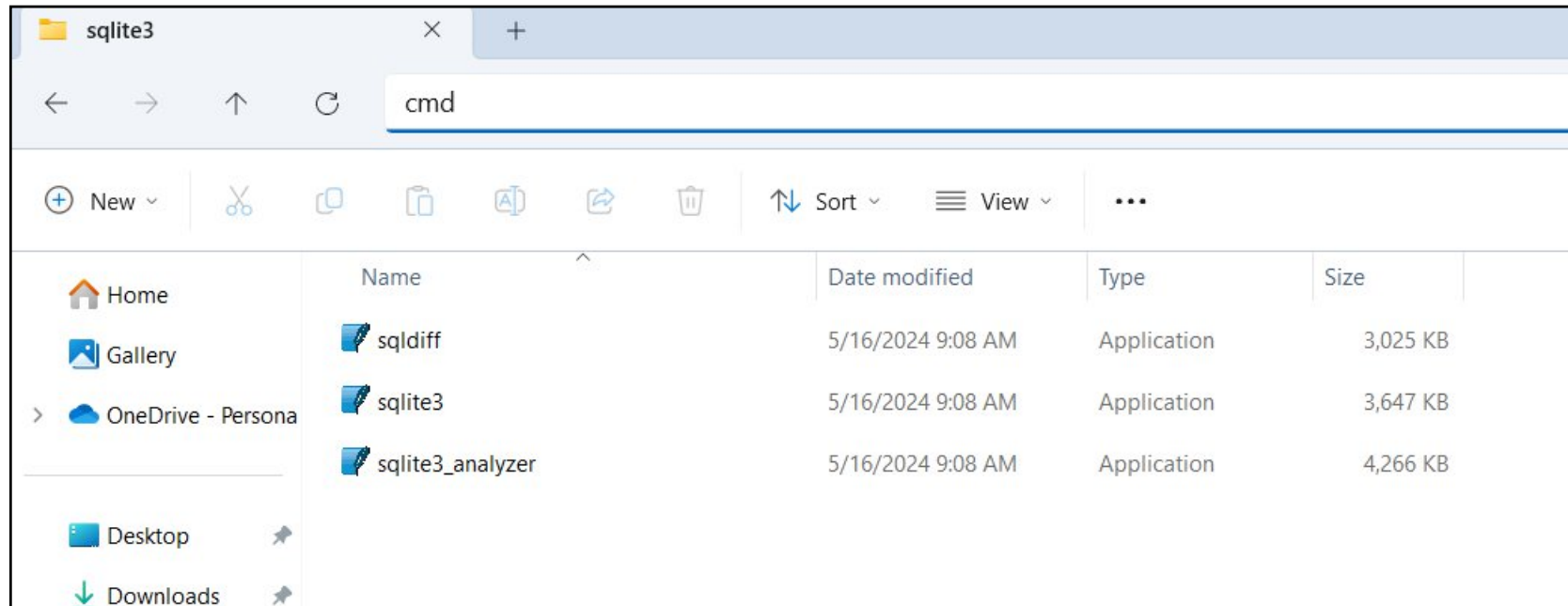
STEP 3

🔧 Set the environment variable to sqlite3 folder.



STEP 4

🕒 Open the command prompt int sqlite3 folder



USEFUL COMMANDS

 Connect to an SQLite database

```
D:\sqlite3>sqlite3
SQLite version 3.45.3 2024-04-15 13:34:05 (UTF-16 console I/O)
Enter ".help" for usage hints.
Connected to a transient in-memory database.
Use ".open FILENAME" to reopen on a persistent database.
sqlite> |
```

USEFUL COMMANDS

 Show available databases

```
sqlite> .databases  
main: "" r/w  
sqlite>
```

USEFUL COMMANDS

 Open the database and create a table

```
sqlite> .open testdb.db
sqlite> CREATE TABLE Customer(id int PRIMARY KEY, name varchar(50), city varchar(20), contact varchar(10));
sqlite>
```

USEFUL COMMANDS

 Show tables in a database

```
sqlite> .tables  
Customer  
sqlite>
```

USEFUL COMMANDS

 Insert data to the table

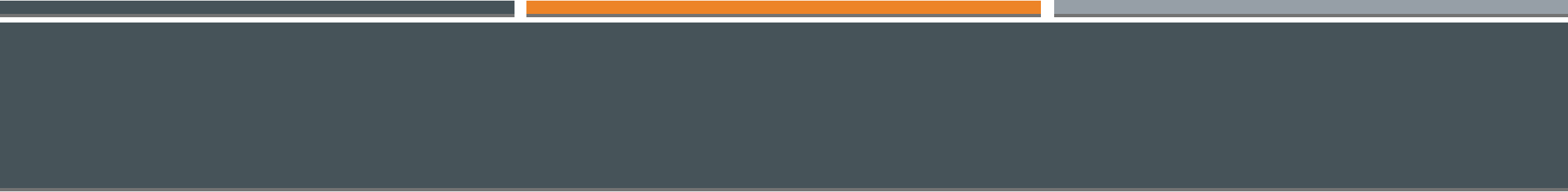
```
sqlite> INSERT INTO Customer VALUES(1, 'Sumudu' , 'Kuliyapitiya', '0778080074');  
sqlite> INSERT INTO Customer VALUES(2, 'Eshan' , 'Colombo', '0713330074');  
sqlite> INSERT INTO Customer VALUES(3, 'Ganesh' , 'Jaffna', '0767422123');
```

USEFUL COMMANDS

Select Query

```
sqlite> .header on  
sqlite> .mode column
```

```
sqlite> SELECT * FROM Customer;  
id  name      city          contact  
--  -  
1   Sumudu   Kuliyapitiya  0778080074  
2   Eshan    Colombo       0713330074  
3   Ganesh   Jaffna        0767422123  
sqlite>
```



SQLITE INSTALLATION (GUI)



Install either DB browser or Sqlite studio

STEP 1

🕒 Download and install DB Browser

URL (<https://sqlitebrowser.org/dl/>)

Downloads

(Please consider sponsoring us on Patreon 😊)

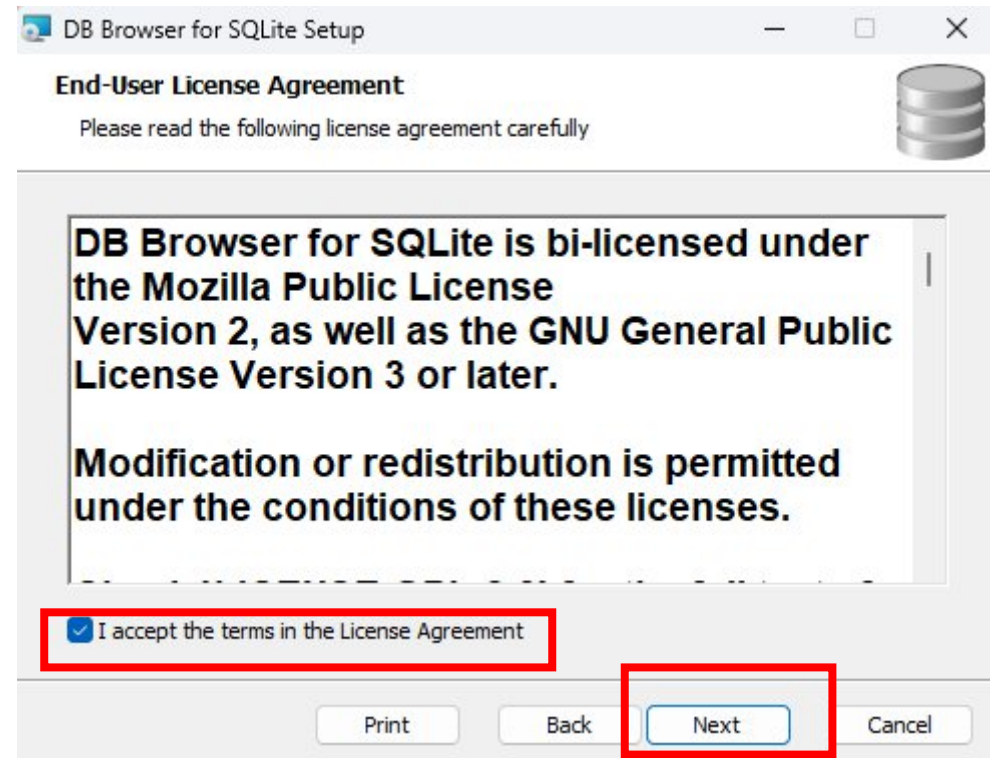
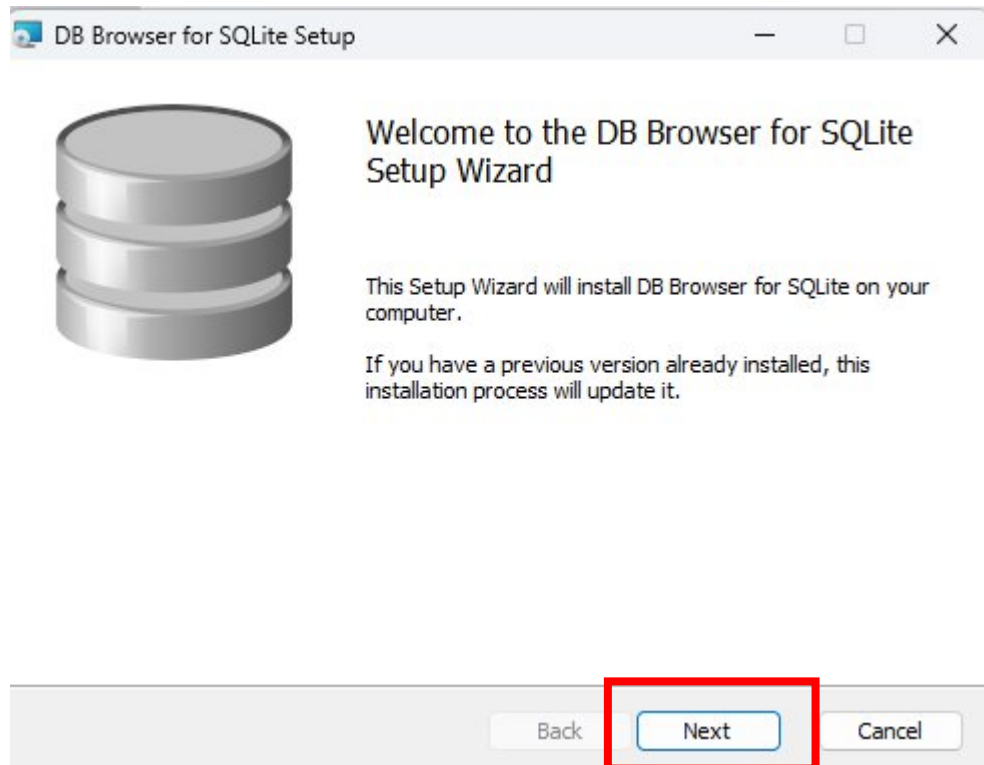
Windows

Our latest release (3.13.0) for Windows:

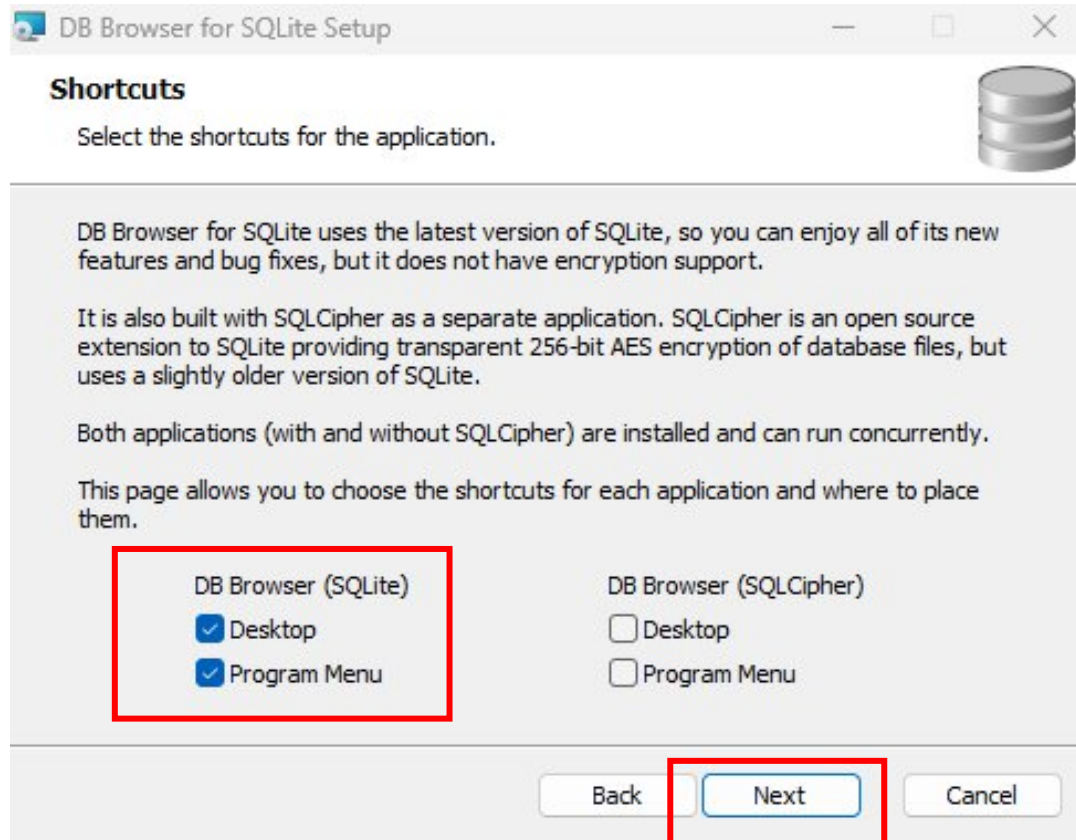
- [DB Browser for SQLite - Standard installer for 32-bit Windows](#)
- [DB Browser for SQLite - .zip \(no installer\) for 32-bit Windows](#)
- [DB Browser for SQLite - Standard installer for 64-bit Windows](#)
- [DB Browser for SQLite - .zip \(no installer\) for 64-bit Windows](#)

Free code signing provided by [SignPath.io](#), certificate by [SignPath Foundation](#).

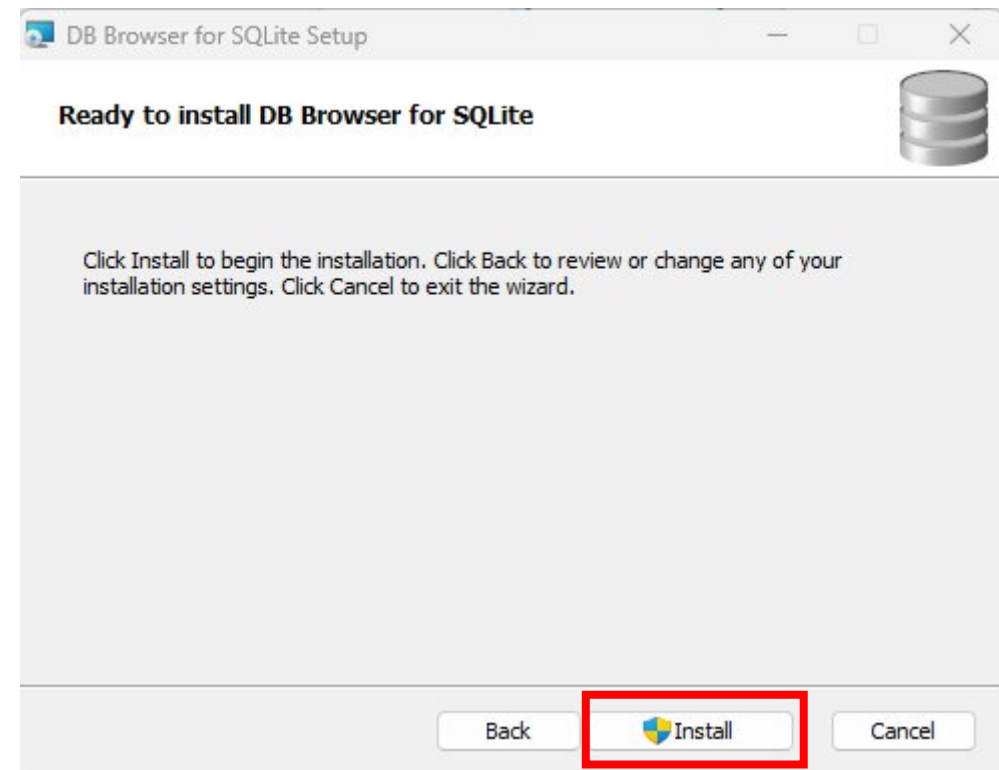
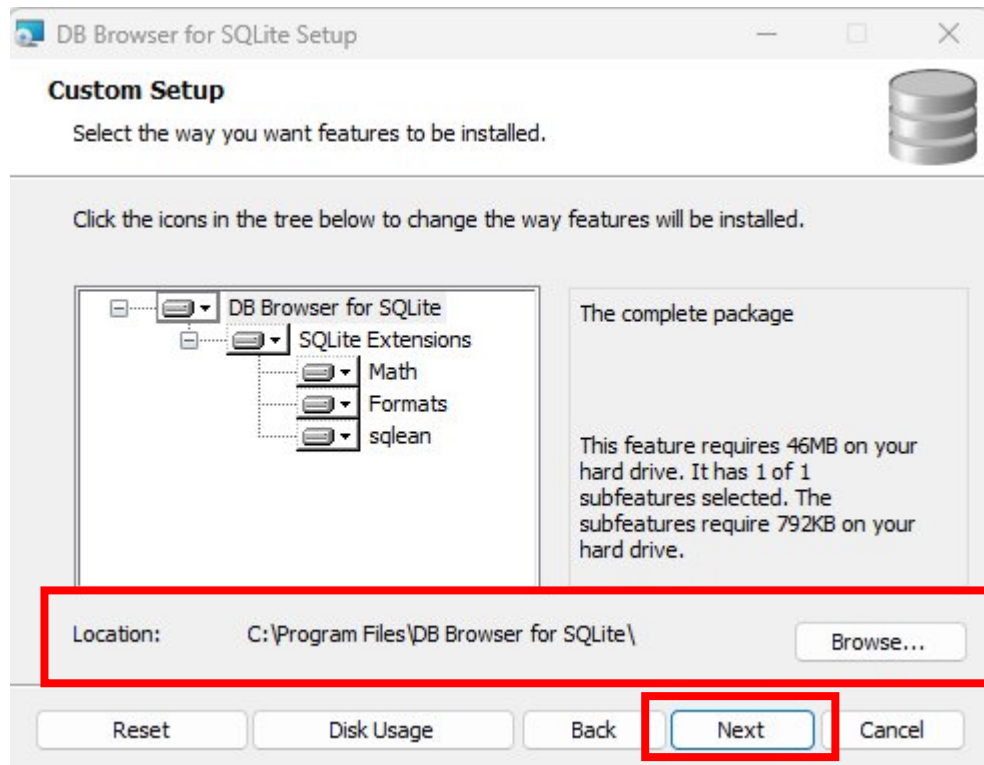
STEP 2



STEP 3

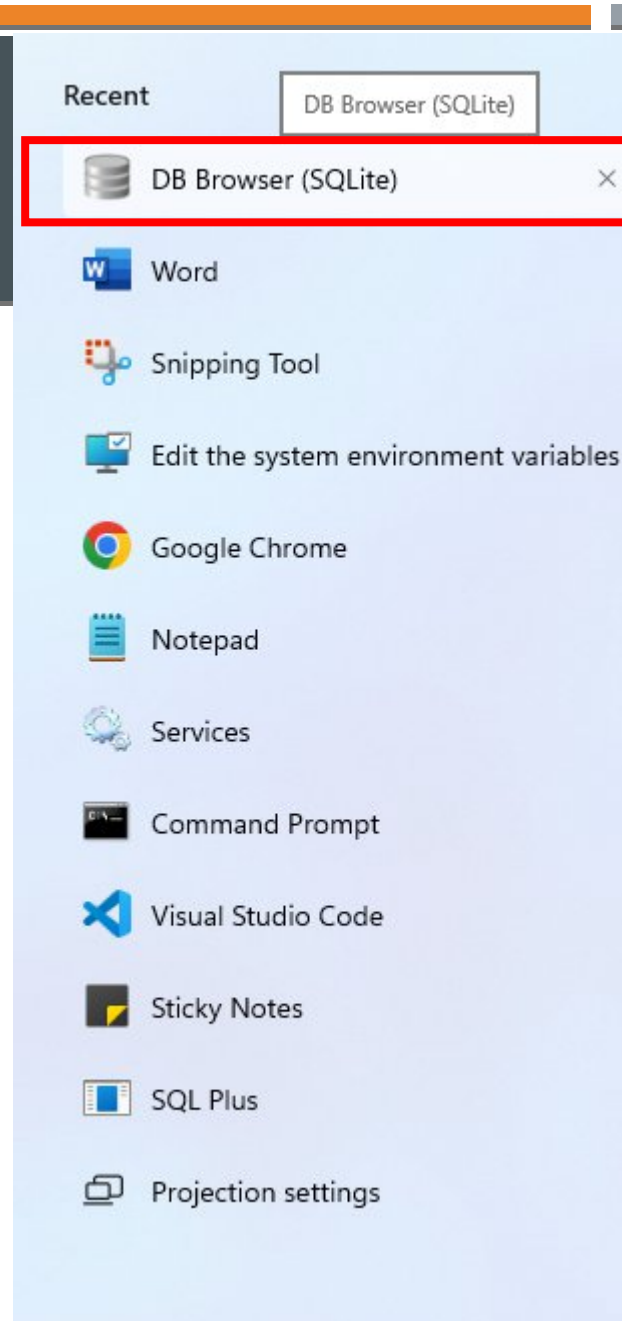


STEP 4

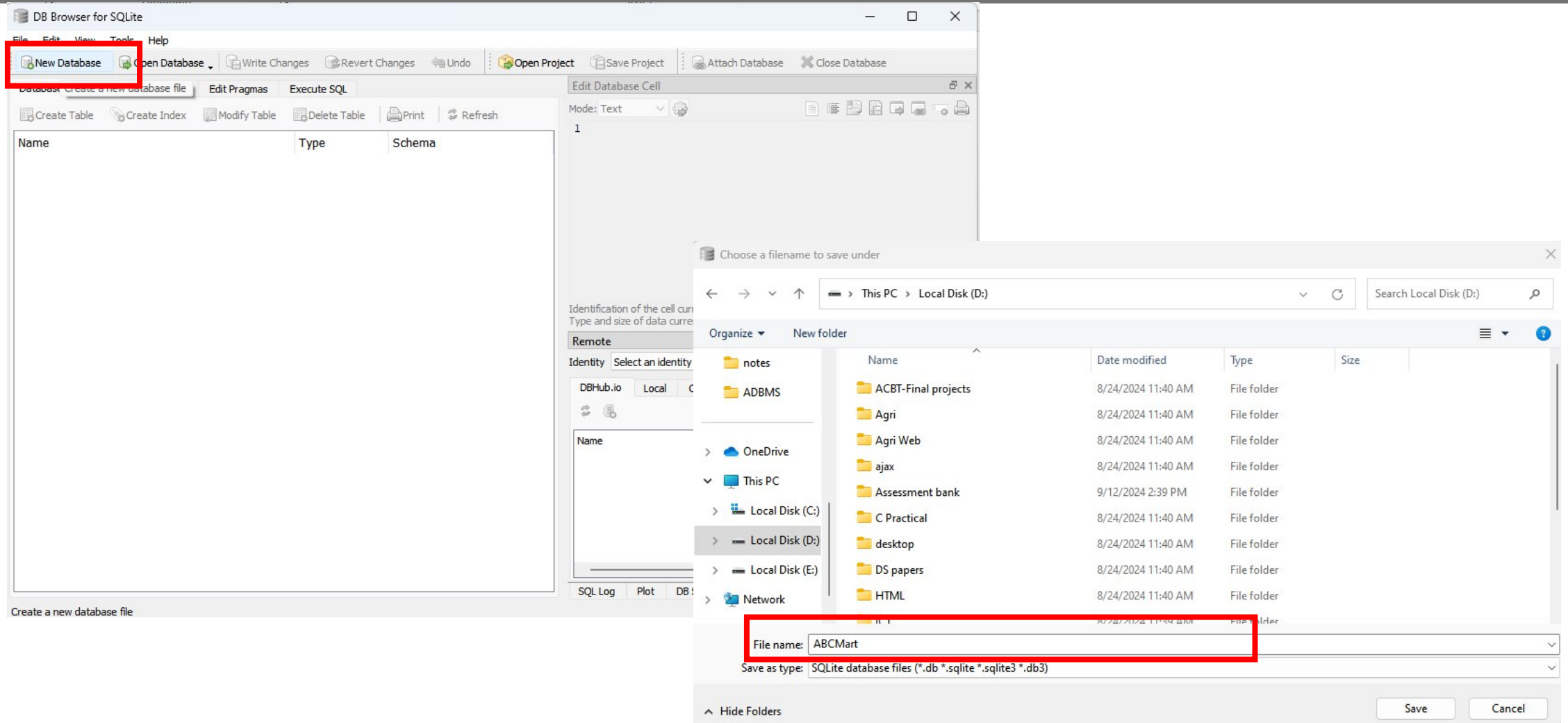


STEP 5

Open DB Browser



CREATE A NEW DATABASE



CREATE A NEW TABLE

Edit table definition

Table

Product

▼ Advanced

Fields Index Constraints Foreign Keys Check Constraints

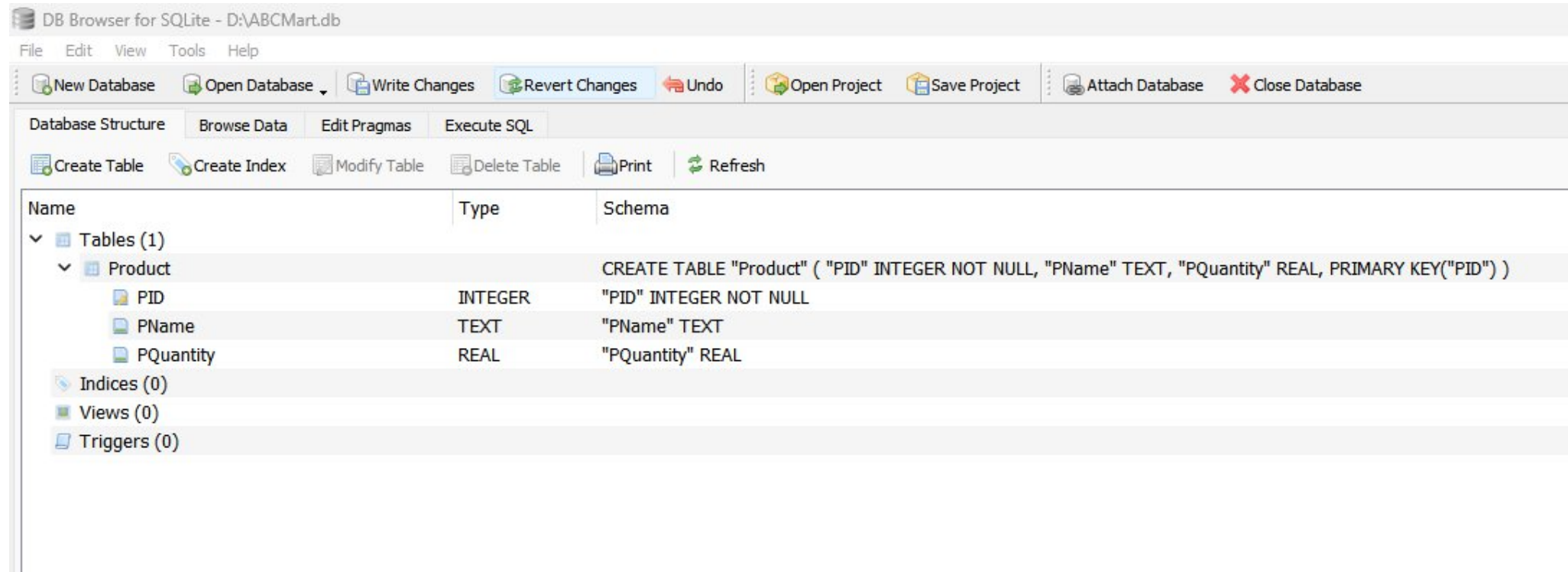
 Add  Remove  Move to top  Move up  Move down  Move to bottom

Name	Type	NN	PK	AI	U	Default	Check	Collation
PID	INTEGER	☒	☒	☐	☐			
PName	TEXT	☐	☐	☐	☐			
PQuantity	REAL	☐	☐	☐	☐			

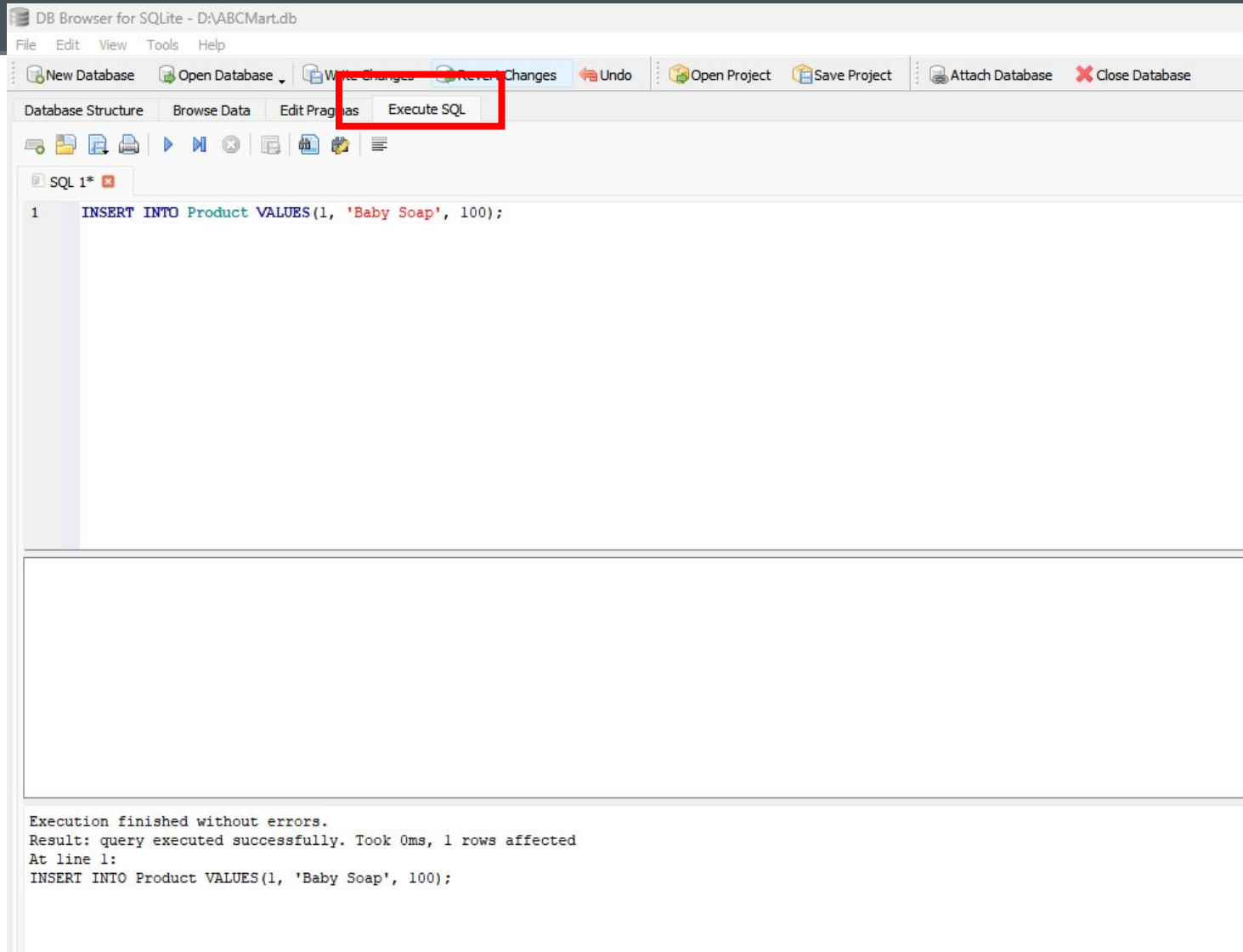
```
1 CREATE TABLE "Product" (  
2     "PID" INTEGER NOT NULL,  
3     "PName" TEXT,  
4     "PQuantity" REAL,  
5     PRIMARY KEY("PID")  
6 );
```

OK Cancel

BROWS THE TABLE



WRITE SQL QUERIES



**Thank
You**

MOBILE DATABASES



LIKE CLAUSE

- 🔗 SQLite LIKE operator is used to match text values against a pattern using wildcards. If the search expression can be matched to the pattern expression, the LIKE operator will return true, which is 1.
- 🔗 There are two wildcards used in conjunction with the LIKE operator
 - The percent sign (%)
 - The underscore (_)

EXAMPLES

- 🔔 **WHERE SALARY LIKE '200%'** - Finds any values that start with 200
- 🔔 **WHERE SALARY LIKE '%200%'** - Finds any values that have 200 in any position
- 🔔 **WHERE SALARY LIKE '_00%'** - Finds any values that have 00 in the second and third positions
- 🔔 **WHERE SALARY LIKE '2_%_ %'** - Finds any values that start with 2 and are at least 3 characters in length
- 🔔 **WHERE SALARY LIKE '%2'** - Finds any values that end with 2
- 🔔 **WHERE SALARY LIKE '_2%3'** - Finds any values that has a 2 in the second position and ends with a 3
- 🔔 **WHERE SALARY LIKE '2___3'** - Finds any values in a five-digit number that starts with 2 and ends with 3

GLOB CLAUSE

- 🔔 The GLOB operator is similar to the LIKE operator. The GLOB operator determines whether a string matches a specific pattern.
- 🔔 Unlike the LIKE operator, the GLOB operator is case sensitive and uses the UNIX wildcards. In addition, the GLOB patterns do not have escape characters.
- 🔔 The following shows the wildcards used with the GLOB operator:
 - 🔔 The asterisk (*) wildcard matches any number of characters.
 - 🔔 The question mark (?) wildcard matches exactly one character.

EXAMPLES

- 🔍 **WHERE SALARY GLOB '200*'** - Finds any values that start with 200
- 🔍 **WHERE SALARY GLOB '*200*'** - Finds any values that have 200 in any position
- 🔍 **WHERE SALARY GLOB '?00*'** - Finds any values that have 00 in the second and third positions
- 🔍 **WHERE SALARY GLOB '2??'** - Finds any values that start with 2 and are at least 3 characters in length
- 🔍 **WHERE SALARY GLOB '*2'** - Finds any values that end with 2
- 🔍 **WHERE SALARY GLOB '?2*3'** - Finds any values that have a 2 in the second position and end with 3
- 🔍 **WHERE SALARY GLOB '2????3'** - Finds any values in a five-digit number that start with 2 and end with 3

LIMIT CLAUSE

- Following is an example, which limits the row in the table according to the number of rows you want to fetch from table

```
sqlite> SELECT * FROM COMPANY;
ID      NAME      AGE      ADDRESS      SALARY
-----
1       Paul       32       California   20000.0
2       Allen      25       Texas        15000.0
3       Teddy      23       Norway       20000.0
4       Mark       25       Rich-Mond    65000.0
5       David      27       Texas        85000.0
6       Kim        22       South-Hall   45000.0
sqlite> SELECT * FROM COMPANY LIMIT 3;
ID      NAME      AGE      ADDRESS      SALARY
-----
1       Paul       32       California   20000.0
2       Allen      25       Texas        15000.0
3       Teddy      23       Norway       20000.0
sqlite>
```

LIMIT CLAUSE

- However in certain situations, you may need to pick up a set of records from a particular offset. Here is an example, which picks up 3 records starting from the 3rd position.

```
sqlite> SELECT * FROM COMPANY;
ID      NAME      AGE      ADDRESS      SALARY
-----
1       Paul       32       California   20000.0
2       Allen      25       Texas        15000.0
3       Teddy      23       Norway       20000.0
4       Mark       25       Rich-Mond    65000.0
5       David      27       Texas        85000.0
6       Kim        22       South-Hall   45000.0
sqlite> SELECT * FROM COMPANY LIMIT 3 OFFSET 2;
ID      NAME      AGE      ADDRESS      SALARY
-----
3       Teddy      23       Norway       20000.0
4       Mark       25       Rich-Mond    65000.0
5       David      27       Texas        85000.0
sqlite>
```

**Thank
You**